

CS 250: Introduction to Logic and Mathematical Reasoning

Course Notes

Contents

1	Welcome, Sets, and Functions	1
1.1	A Note on The Textbook	1
1.2	Welcome to CS 250	1
1.3	Why study logic?	2
1.4	What is logic, what is a proof?	3
1.5	Variables and formulae	4
1.6	Sets and pairs	5
1.7	Functions and relations	8
	Intermezzo: Cardinality and Function Spaces	10
1.8	Comparing set sizes	10
1.9	Countable infinity	11
1.10	Uncountable infinity—Cantor’s diagonal argument	11
1.11	Function spaces	12
1.12	The power set is a function space—the punchline	13
1.13	Exercises	14
2	Propositional Logic	15
2.1	Logical formulas, formalized	15
2.2	Tautologies and equivalences	18
2.3	Addendum: nand, xor, and how many binary operations are there?	20
3	Proof Systems for Propositional Logic	23
3.1	Tautologies as reasoning	23
3.2	A thorough derivation of proof by contradiction	24
3.3	A proof system for propositional logic	25
3.3.1	Rules for conjunction	26
3.3.2	Rules for $\vee$	27
3.3.3	Negation	27
3.3.4	T and F	28
3.3.5	Implication	28
3.3.6	Extra proof rules	28
3.4	Arguments and validity	29
3.5	Examples of doing proofs	31
3.5.1	Conjunctive normal form	31
3.5.2	Disjunctive normal form	31
3.6	Digital logic circuits	32

3.7	Addendum: differences in terminology and additional derivable proof rules for home-work	33
3.7.1	Derivable rules of inference	33
3.7.2	Differences in verbiage	34
3.7.3	Bitwise logical operators	34
Intermezzo: Inference Rules as Trees		35
3.8	A better notation	35
3.9	The rules, redux	35
3.9.1	Conjunction	35
3.9.2	Disjunction	35
3.9.3	Implication	36
3.9.4	Negation	36
3.9.5	Constants	37
3.10	Composing rules into proof trees	37
3.11	Reading proof trees	38
3.12	Why this matters	38
4	First-Order Predicate Logic	39
4.1	A logic with real types	39
4.2	Forall and exists	39
4.2.1	Free and bound variables	40
4.3	Truth sets	40
4.4	Negating quantified statements	41
4.5	Nested quantifiers	42
4.6	Necessary and sufficient conditions	43
4.7	Proofs with this logic	43
4.8	Induction, round 1	45
4.9	Addendum on notation	46
Intermezzo: More on Natural Induction		47
4.10	Proof that $0 + m = m$	47
4.11	Proof that $\text{succ } n + m = \text{succ } (n + m)$	47
4.12	Proof that addition is associative	48
4.13	Proof that addition is commutative	49
Intermezzo: Quantifiers as Games		50
4.14	The game interpretation	50
4.15	Nested quantifiers as alternating turns	51
4.16	Games and constructive logic	52
4.17	Why this matters	52
5	Informal Proofs and Groups	54
5.1	Direct proof	54
5.2	Counterexample	56
5.3	Existence proofs	56
5.4	What is a group?	57
5.5	The integers under addition form a group	58
5.6	The integers under multiplication do <i>not</i> form a group	59

5.7	Proof of uniqueness of identity	59
6	Modular Arithmetic and Proofs by Cases	60
6.1	The modulus operation	60
6.2	The quotient-remainder theorem	60
6.3	Floor and ceiling	61
6.4	$\mathbb{Z}/n\mathbb{Z}$ is a group	61
6.5	$\mathbb{Z}/p\mathbb{Z}$, where p is prime	62
6.6	Proofs by cases and the connection to or-elimination	62
	Intermezzo: What Does It Mean for Things to Be “The Same”?	64
6.7	Equivalence as abstraction	64
6.8	Quotient sets	65
6.9	The connection to types and programming	66
6.10	Weyl’s insight, and why it matters	66
7	Indirect Proof, the Euclidean Algorithm, and Hoare Logic	68
7.1	More on proofs by contradiction and proofs by contrapositive	68
7.1.1	Proof by contradiction, in practice	68
7.1.2	Proof by contrapositive	69
7.1.3	Contradiction vs. contrapositive: when to use which	70
7.2	GCD, Euclid’s algorithm, &c.	70
7.3	Programming logic	72
7.3.1	Putting it all together	74
	Intermezzo: Proofs Are Programs	77
7.4	What does a proof <i>look like</i> ?	77
7.5	The Curry–Howard dictionary	78
7.6	A worked example: the same proof, four ways	78
7.6.1	As a derivation (Module 3 style)	79
7.6.2	As a Gentzen tree	79
7.6.3	As a Python function	79
7.6.4	As Lean 4	79
7.7	Why this matters	80
	Intermezzo: Other Logics, Other Worlds	81
7.8	The zoo of logics	81
7.9	Temporal logic—logic over time	82
7.10	Expressing real properties	83
7.11	The pattern	83
7.12	Exercises	84
8	Sequences and Recursion	85
8.1	Definition by recursion	85
8.1.1	Explicit formulas	85
8.1.2	Recursive definitions	86
8.1.3	Summation and product notation	86
8.2	Induction on sequences	87
8.2.1	The standard template	87

8.2.2	Worked example: sum of the first n integers	88
8.2.3	Setting up: sum of squares	89
8.2.4	Induction is recursion	89
9	Strong Induction and Recurrences	91
9.1	Strong induction	91
9.2	Defining sequences recursively	93
9.3	Solving recurrences by iteration	94
9.4	Second-order linear homogeneous recurrences	95
10	Structural Induction	98
10.1	The pattern so far	98
10.2	Lists	98
10.2.1	Defining lists inductively	98
10.2.2	Recursive functions on lists	99
10.2.3	Worked proof: length distributes over append	100
10.3	Binary trees	101
10.3.1	Defining binary trees inductively	101
10.3.2	Recursive functions on binary trees	102
10.3.3	Worked proof: counting edges in a binary tree	102
10.4	The general recipe	103
10.4.1	Example: structural induction on propositional formulas	104
10.5	Wrapping up: the big picture	105
	Intermezzo: The Algebra of Types	107
10.6	Types are like numbers	107
10.7	The laws of algebra hold	108
10.8	Data types as polynomials	108
10.9	The Curry–Howard connection	110
10.10	Exercises	110
	Intermezzo: The Lambda Calculus	111
10.11	A mathematical model of computation	111
10.12	Computing with pure functions	112
10.13	Church encodings—building everything from nothing	112
10.14	Why this matters	114
10.15	Exercises	114
A	Notation Summary	116
B	Solutions to Exercises	118

Chapter 1

Welcome, Sets, and Functions

1.1 A Note on The Textbook

If you're having trouble finding a copy of Epp 5th edition, check the library, course reserves, or the instructor first. Older editions can also still be useful for extra practice problems.

1.2 Welcome to CS 250

Welcome to CS 250! This class is an introduction to logic and mathematical reasoning geared towards computer scientists.

A note on the structure of these notes

Between the main modules you'll find *intermezzi*—short chapters on topics that are more mathematically sophisticated, that go deeper into a subject than the main modules require, or that cover optional material you may find interesting. They are unnumbered and won't appear on exams unless your instructor says otherwise. Think of them as the bonus tracks on an album: they enrich the experience but the main story is told in the numbered modules.

By the end of this class you will

- Be able to explain what a mathematical proof is
- Know the distinction between formal and informal proofs
- Be able to prove mathematical and logical statements with direct and indirect proof methods
- Be able to calculate truth tables for Boolean expressions
- Be able to define data in BNF notation
- Be able to explain the induction principle on the natural numbers
- Know how to relate induction on other types to natural induction
- Be able to perform simple formal proofs of program correctness

1.3 Why study logic?

- understanding how to write proofs will let you reason about the *limits* of computation
- learning logic will help you reason, in general
- on a practical level, “formal methods”—the application of logic to computer science—are a lucrative and in-demand field in high-stakes computing and engineering, from embedded systems programming to chip design to the development of operating system components

To that first point, let’s consider a problem: if we want to understand not just what computers can do *today* but what computers can do *ever*, we need to start with an **abstract** model of computation, one that doesn’t depend on a particular chip architecture or instruction set.

Consider: suppose I managed to demonstrate that **no** AMD chip today could run a program that reads in the code of another program and prints 1 if the program would run in finite time and 0 if the program would go into an infinite loop. What would that tell us?

I’ve demonstrated this result. How do I know there *won’t* be a chip that *could* run such a program? How do we know we won’t invent some new kind of computer that could do this? Maybe a quantum computer could do things that seem impossible today?

Or how about this: consider two programming languages, Python and C.

If I can solve a problem in Python, do I **know** that I can always write a C program that computes the same thing? What about the other way around?

We all intuitively feel that there isn’t a real difference between what two languages can compute—only in how efficiently they compute.

But intuition alone isn’t good enough for a computer scientist. We need the ability to *know* whether two languages are essentially equivalent or two chips capable of running the same kinds of programs.

To do that we need mathematical *models* of computation that don’t depend on any of these details, like the specifics of language or hardware. In other words, we need a model that *abstracts* over these details.

But what would you do with such a model once you had it?

We’ve already determined it’s not a concrete programming language with a compiler.

No, instead we have to reason with mathematical tools: sets, symbols, syntax and semantics, proofs.

The question “what is a proof and how do I write one” is the bulk of this class. The actual mathematical modeling of computation and computer science we’ll leave for the sequel course, CS 251.

For now, I’ll say a proof is a way to convince the reader, by clearly outlining a sequence of reasoning steps, that something is true.

Our points two and three are intertwined but they involve explaining the difference between *formal* and *informal* proof. Normal proofs, like what I was talking about above, are technically considered “informal”. An informal proof is meant to communicate between mathematicians, using a mix of symbolic notation and prose. A formal proof, by contrast, lives *inside* a logic and uses only the logic’s own rules. The formal rules of a logic will feel strange and constraining, almost like writing assembly instead of a regular programming language. Each step is small, tedious, and verbose. Yet there is something important you learn by doing it. Again like assembly, you learn *why* things work the way they do.

You’re going to learn how the rules of a logic are justified, mathematically, and how these rules help us understand what is a good and bad proof. One of the hardest things when you’re first

writing informal proofs is knowing what you're allowed to say and what needs justification. Formal logic helps you build that judgment even when you're not using it directly.

Formal logic is not just a training exercise you discard once you've learned the basics. Formal proofs resist the kind of errors that are easy to make informally—errors that even professional mathematicians regularly commit. What if you could get those advantages without the tedium? That's where interactive theorem provers, sometimes called proof assistants, come in. These are like very specialized programming languages where the programs you write *are also* proofs in a formal logic! Using a proof assistant is its own skill and becoming truly competent with them is beyond the scope of this course, but I at least want to expose you to using them.

1.4 What is logic, what is a proof?

Let's make the central concept of this course precise. Logic is the study of how the truth of statements follows from their premises by means of *valid arguments*—arguments built from correct reasoning steps. In mathematics, we call a valid argument a *proof*: a way to convince the reader, through a clear sequence of reasoning steps, that something is true.

Let's see what that looks like in practice with a simple example.

Let's say you were thinking about the *natural numbers*. That is the numbers that are either zero or a positive whole number. You realize “hey, there are a lot of these. There must be infinitely many of them.” and now you want to convince yourself this is *true*, in other words you want to *prove* it.

To start, you have a rather clever guess on how to show this. First, you note that if you have a natural number you can always add one and it stays a positive whole number. Second, you decide to see what happens if there's a **finite** amount of natural numbers. You're going to *assume* that this is true and then see if you can show it leads to something weird.

The first consequence of there being a *finite* number of natural numbers is that there must be a *biggest* natural number.

Why? Because for any two natural numbers one of them must be at least as large as the other. If you compare all of these finite numbers, one of them must be the biggest.¹

Then if we have a biggest natural number we should be able to add one to it and get a new natural number. This new natural number must be bigger.

But how can this be true if we already found the biggest?

We've found a contradiction. A contradiction usually means something has gone wrong—and something has: our assumption that there are only finitely many naturals. Since the assumption leads to nonsense, we conclude that there are *infinitely many* natural numbers.

We've proved what we set out to prove, then, that there are *infinitely many natural numbers*.

This is a pretty valid proof by mathematician standards, but let's talk about some of the assumptions that went into it:

- First, we didn't actually show that you always get a new natural number by adding one to a natural number; we just appealed to the intuition that we've all done addition before and asserted it.
- Second, we didn't actually explain how we can generalize from “you can always compare two numbers” to “you can compare all the numbers in a collection of numbers.”

¹Can you implement this idea as a program?

- Third, we didn’t even technically define what comparing the size of two numbers means. Did you catch that?
- Finally, we didn’t actually justify this proof by contradiction technique. How do we know that just because we’ve found a contradiction from the assumption that the natural numbers are finite that we absolutely *know* that they must instead be infinite? We didn’t even define finite and infinite, did we?

My point here is that even when a mathematician writes a proof there is usually a body of assumptions, definitions, and facts that they’re relying on!

The world’s most niche personality test:

- If you’re completely unbothered by these omissions, you’re thinking like a mathematician: focused on getting the right intuition and letting some details slide for the sake of clarity. You want to finish the proof and move on to the next interesting thing.
- If you **are** bothered, you’re thinking like a logician: concerned about getting every detail of a proof correct and being explicit about all assumptions and definitions. You worry about whether things like “proof by contradiction” even make sense as a way of acquiring knowledge. You might like the word “epistemology.”
- If you thought “well I know there are infinitely many natural numbers because I could write a loop that runs forever enumerating them,” you’re thinking like a *constructive* mathematician: a blend of mathematician, logician, and computer scientist. You do math in your code and code with your math. You may have heard of languages like Agda and Idris that blur the line between proof and program.

None of these approaches is better than the others; each has its own strengths and weaknesses.

These notes were, however, written by a constructive mathematician, so there may be bias peeking through in how this material is presented.

1.5 Variables and formulae

We’ve given a proof in a very *informal*, natural-language way. By natural language I mean human language that evolved organically, as opposed to the language of mathematics or a programming language.

We need to start moving from this informal language into *formal* language: symbols with rigid meaning. We’ll make this journey piecemeal over time, learning how to take statements in human language—which can be ambiguous—and turn them into rigorous, unmistakable combinations of symbols.

This will both help us be precise in how we do mathematics but also help us move our mathematics from pen and paper to computer programs.

The first such formalization is the introduction of *variables*. You’ve already seen variables in algebra and programming before, although their uses are slightly different from what we’re doing here.

We’re not “solving for x” like in algebra nor are we using variables like labeled containers as in *C*, *Python*, or similar programming languages where the contents of the container can change over time.

Instead, think of variables as generalized pronouns — they work a lot like “it” or “this” in natural language.

There are *patterns* to how we introduce variables like this: we name the variable and its scope and then allow ourselves to use it, e.g. “Consider a chicken **c**, such that ...” or “For all students, **s**, in CS250 ...”

For a more concrete example, recall that in the previous section we assumed there was a largest natural number and talked through the rest of the proof from there.

With variables, we can say something like: “Assume there is a largest natural number, **n**. Then we know that $n+1$ is also a natural number and that $n+1 > n$, contradicting the assumption that **n** is largest.”

That’s a lot clearer than trying to write it in just words, right, and all we’ve done is introduce a small abstraction.

We *need* variables to help us write more complicated kinds of logical statements as well. The first kinds we’re going to talk about are what Epp calls universal, existential, and conditional.

Universal statements are things like “all dogs are mammals” or “every chicken has a common ancestor”. Universal statements are about something that is true for every possible example of a “class of thing”. We’re going to make “class of thing” more rigorous soon, but for now think “all dogs”, “every person”, “all programming languages”.

Universal statements can also include negatives like “no dogs are also cats” or “every person is not immortal”.

Existential statements are like “there is a smallest natural number” or “there is at least one rainy day each fall”. These are about showing examples of something or that a thing with certain properties *exists*.

These are the kinds of claims you make when you’re showing a problem even has a solution. For example, “the equation $3x - 29 = 0$ has a solution” is an existential statement.

The last kind of statement Epp teases out is the “conditional statement”, which express an “if-then” relationship.

For example, “if a pear can be used as a weapon, then it is not ripe” or “if a program does not compile, then it is not correct”.

We can even combine all of these concepts together into an example such as “for every equation of the form $mx + b = y$, where m is not 0, there exists a solution”. This is a universal wrapped around a conditional wrapped around an existential.

We’re now ready to make the “class of thing” concept a little more formal and talk about *sets*.

1.6 Sets and pairs

In the previous section we talked about classes of things such as “all dogs” or “all C programs”. In mathematics, we call these collections *sets*.

Definition 1.6.1 (Set). A *set* is a collection of objects, called *elements*. We denote a set with curly braces $\{ \}$, with commas separating the elements. If x is an element of set A , we write $x \in A$. If x is not an element of A , we write $x \notin A$.

Some examples:

`{red, green, blue}`

`{you, your friend, your friend’s friend}`

`{the best chicken, the worst chicken}`

These are very *abstract* collections.

How do we make sets? There are three basic ways. The first is that you can take a finite number of things and put them between the curly braces and call that a set. That’s how our first few examples worked.

The second is that you can take *other* sets and then apply a condition—a *predicate*—onto it to make a new set. The easiest example is that you can take $\mathbb{N}$, the set of all natural numbers, and then filter out all the even numbers to get the set of all odd natural numbers. If you’ve seen Python’s “list comprehensions” before, then you have seen a similar concept.

In symbols, we denote this with “set-builder notation” so we can write the above description as $\{n \mid n \in \mathbb{N}, n \% 2 = 1\}$

The general pattern is $\{x \mid x \in S, P(x)\}$, which reads “the set of all x in S such that $P(x)$ is true.” The vertical bar $\mid$ means “such that.”

But how do you get things like “the set of all natural numbers” in the first place? That brings us to the third way to define sets, which is *inductively*. We’ll introduce this informally by example until the second half of the course, where we make it all more rigorous.

With set-builder notation in hand, we can introduce some basic operations on sets.

Definition 1.6.2 (Intersection). The *intersection* of two sets A and B is the set of elements they have in common: $A \cap B := \{x \mid x \in A \text{ and } x \in B\}$.

Definition 1.6.3 (Union). The *union* of two sets A and B is the set of elements in either A or B : $A \cup B := \{x \mid x \in A \text{ or } x \in B\}$.

Definition 1.6.4 (Subset). Given sets A and B , we say A is a *subset* of B , written $A \subseteq B$, when every element of A is also an element of B .

Definition 1.6.5 (Set Difference). The *difference* of sets A and B , written $A \setminus B$, is the set of elements in A that are not in B : $A \setminus B := \{x \mid x \in A \text{ and } x \notin B\}$.

Definition 1.6.6 (Complement). Given a universal set U that contains all elements under discussion, the *complement* of A , written $\overline{A}$, is $U \setminus A = \{x \mid x \in U \text{ and } x \notin A\}$.

The complement only makes sense relative to some agreed-upon universe U . When we write $\overline{A}$ we’re implicitly assuming everyone knows what U is.

Definition 1.6.7 (Disjoint). Two sets A and B are *disjoint* if they share no elements: $A \cap B = \emptyset$.

Next is something that seems obvious but is incredibly important and—in the history of mathematics—quite divisive: equality of sets.

Two sets are equal exactly when $A \subseteq B$ and $B \subseteq A$, in other words when the two sets have *exactly* the same elements.

This has a couple of implications. First, there’s no sense of *ordering* in sets: $\{a, b, c\} = \{c, b, a\} = \{c, a, b\}$. Second, it doesn’t matter how you build the sets. You could get the “set of natural numbers divisible by four” either as $\{x \mid x \in \mathbb{N}, x \% 4 = 0\}$ or as a two-step process: first get the even numbers $E = \{x \mid x \in \mathbb{N}, x \% 2 = 0\}$ and then filter as $\{x \mid x \in E, x \% 4 = 0\}$. Either way, you get the “same” set because they have the same elements.

This is called *extensional* equality, that is equality because the two things look the same from “the outside”.

I want to hint at why this is a useful yet *controversial* definition by asking a simple question: when are two programs the same? Is producing the same output for the same input enough to call the programs the same? Why or why not?

The alternative is *intensional* equality, where two things are equal only if they’re defined or constructed the same way. Consider the sets $\{x \mid x \text{ is even and } x < 10\}$ and $\{2, 4, 6, 8\}$. These are extensionally equal—they have exactly the same elements. But intensionally they’re different: one

is defined by a predicate, the other by enumeration. The “recipes” are different even though the “dishes” come out the same.

This is exactly the question with programs: two programs that produce the same output for every input are extensionally equal. But they might use completely different algorithms, have different performance characteristics, different memory usage. Are they “the same program”? An extensionalist says yes; an intensionalist says no.

For sets, we’re choosing the extensional view in this course—two sets with the same elements are the same set, period. But be aware that this is a choice, and in some areas of computer science and mathematics (particularly type theory and constructive mathematics), intensional equality is the default. The difference becomes important when you care not just about *what* something computes but *how* it computes it.

One more consequence of extensional equality that’s worth making explicit: *duplicates don’t count*. The set $\{2, 2, 2, 2\}$ is the same as the set $\{2\}$ because they have the exact same elements—the number 2 and nothing else. Writing 2 four times doesn’t somehow put four copies of it in the set. Sets don’t work like arrays.

Definition 1.6.8 (Cardinality). The *cardinality* of a finite set A , written $|A|$, is the number of *distinct* elements in A .

For example:

- $|\{a, b, c\}| = 3$
- $|\{2, 2, 2, 2\}| = |\{2\}| = 1$
- $|\{\}| = 0$ (the empty set has no elements). The set with no elements, written $\{\}$ or $\emptyset$, is called the *empty set*.

One that trips people up: what about $\{0, \{0\}\}$? This set has two elements: the number 0 and the set $\{0\}$. These are different things! One is a number and the other is a set that *contains* a number. So $|\{0, \{0\}\}| = 2$. Don’t confuse an element with a set containing that element—0 and $\{0\}$ are not the same thing, just like a box is not the same thing as what’s inside it.²

Definition 1.6.9 (Power Set). The *power set* of a set A , written $\mathcal{P}(A)$, is the set of all subsets of A : $\mathcal{P}(A) := \{S \mid S \subseteq A\}$.

For example, $\mathcal{P}(\{1, 2\}) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$. Notice that the power set always includes both $\emptyset$ (the empty set is a subset of everything) and A itself ($A \subseteq A$).

How big is the power set? For a set with n elements, $|\mathcal{P}(A)| = 2^n$. Each element of A is either in a given subset or not—two choices per element, so $2 \times 2 \times \cdots \times 2 = 2^n$ total subsets. We’ll see where this 2^n comes from more deeply in the next chapter.

The final concept to discuss with sets is another operation: the *product* of sets. Epp’s approach is to derive products from more basic set operations, which is a worthwhile exercise and worth reading once. In these notes, we’ll instead take tuples as a basic building block, which is more common in the kind of foundational mathematics you’ll encounter as a computer scientist.

You’ve seen tuples before: coordinates! $(3, 4)$ is three units in the x-direction, four units in the y-direction. Tuples are, in a sense, the opposite of sets. They have *order* and a *fixed size*. If you want an analogy, think of arrays in C/C++: you can have arrays of arbitrary size but any *given* array has a size that you decided on in advance.

²If you’re a Python programmer: `0 == {0}` is **False**. If that seems obvious to you, good. But it’s the kind of thing that causes errors on homework.

Definition 1.6.10 (Cartesian Product). The *Cartesian product* of sets A and B is the set of all tuples made from A and B : $A \times B = \{(a, b) \mid a \in A, b \in B\}$.

With all of that, we can move on to the last topic of this first module: functions and relations.

Exercise 1. Let $A = \{1, 2, 3, 4\}$ and $B = \{3, 4, 5, 6\}$. Compute $A \cap B$, $A \cup B$, and $|A \times B|$.

Exercise 2. What is $|\{1, \{1\}, \{1, \{1\}\}|$? Be careful—count distinct elements.

1.7 Functions and relations

As with the last section we’ll diverge a little from Epp here.

Definition 1.7.1 (Function). A *function* f from a set A to a set B , denoted $f : A \rightarrow B$, is an assignment that sends *every* element of A to *exactly one* element of B . That is, every $a \in A$ has an output $f(a) \in B$, and if $f(a) = b$ and $f(a) = c$ then $b = c$.

There are two requirements packed into this definition: *totality* (every element of A must have an output) and *uniqueness* (each element of A maps to exactly one element of B , not two or more). If we drop the totality requirement—allowing some elements of A to have no output—we get a *partial function*. Partial functions come up naturally in computer science (think of a program that might crash or loop forever on certain inputs), but unless we say “partial” explicitly, “function” always means the total version.

Epp defines a function as a subset of $A \times B$. This is related, but in general “a function as a gadget that assigns elements of one set to another” and “the subset of tuples in $A \times B$ that describes the action of the function” are two different concepts that we need to keep distinct. This connects back to the question about when two programs are the same that we raised in the previous section.

It’s important to note that functions in mathematics include both things we know how to calculate, like $f(x) = x + 1$, and things we don’t. For example, consider a function that maps each C program implementing a function from integers to integers to the smallest input that doesn’t cause it to go into an infinite loop, if one exists. That’s a weird partial function, but it’s a well-defined assignment, and it’s definitely not something you can calculate.

Definition 1.7.2 (Relation). A *relation* between sets A and B is a subset of $A \times B$ — a collection of pairs (a, b) recording which elements of A are related to which elements of B . Unlike a function, a relation allows an element of A to be related to zero, one, or many elements of B .

We’ve already seen one important relation: $\subseteq$ is a relation on sets. You can have $A \subseteq B$ and $A \subseteq C$ without B and C being the same, because $\{1\} \subseteq \{1, 2\}$ and $\{1\} \subseteq \{1, 3\}$.

A few more examples of relations you already know: equality on a set (is a the same as b ?), “less than” on the integers ($3 < 5$ but not $5 < 3$), and “is a prerequisite of” on courses (CS 250 is a prerequisite of CS 251, but not the other way around).

When we have a relation R on a single set A (that is, R relates elements of A to other elements of A), we can ask what *properties* it has. Three come up constantly.

Definition 1.7.3 (Reflexive). A relation R on a set A is *reflexive* if every element is related to itself: for all a in A , $a R a$.

Definition 1.7.4 (Symmetric). A relation R on a set A is *symmetric* if whenever a is related to b , then b is related to a : for all a, b in A , if $a R b$ then $b R a$.

Definition 1.7.5 (Transitive). A relation R on a set A is *transitive* if relations chain: for all a, b, c in A , if $a R b$ and $b R c$, then $a R c$.

Definition 1.7.6 (Equivalence Relation). A relation that is reflexive, symmetric, *and* transitive is called an *equivalence relation*.

Equality is the canonical equivalence relation—it's reflexive ($a = a$), symmetric ($a = b$ implies $b = a$), and transitive ($a = b$ and $b = c$ implies $a = c$). We'll see other equivalence relations later in the course, such as modular congruence.

Which properties do our other examples satisfy? “Less than” on the integers is transitive ($a < b$ and $b < c$ implies $a < c$) but not reflexive ($3 \not< 3$) and not symmetric ($3 < 5$ but $5 \not< 3$). The subset relation $\subseteq$ is reflexive ($A \subseteq A$) and transitive but not symmetric ($\{1\} \subseteq \{1, 2\}$ but $\{1, 2\} \not\subseteq \{1\}$).

Exercise 3. For each of the following relations on $\mathbb{Z}$, determine whether it is reflexive, symmetric, and/or transitive: (a) $a R b$ iff $a + b$ is even; (b) $a R b$ iff $a \leq b$; (c) $a R b$ iff $|a - b| \leq 1$.

This has been a foundational week: sets, tuples, functions, and relations are the vocabulary we'll use for the rest of the course. In the next module we start doing *logic*.

Intermezzo: Cardinality and Function Spaces

In Module 1 we introduced sets, functions, and the power set. We said the cardinality of a finite set is just the number of distinct elements it contains. But we also mentioned sets like $\mathbb{N}$ and $\mathbb{Z}$ that are infinite. How do you compare the sizes of infinite sets? Do all infinite sets have the “same amount” of infinity?

The answer is *no*, and the story of how we know that—and the beautiful connection between power sets and function spaces that falls out of it—is what this chapter is about.

1.8 Comparing set sizes

For finite sets, cardinality is easy: count the elements. $|\{a, b, c\}| = 3$ and $|\{0, 1\}| = 2$, so $\{a, b, c\}$ is bigger. Done.

But what about infinite sets? You can’t count to infinity and compare the totals. We need a different strategy. The key idea, due to Georg Cantor in the 1870s, is to use *functions* to compare sizes.

If you can pair up every element of A with a unique element of B and vice versa—a perfect matching with nothing left over on either side—then A and B have “the same size.” The function that does this pairing is called a *bijection*, and it’s built from two simpler properties.

Definition 1.8.1 (Injection (one-to-one)). A function $f : A \rightarrow B$ is *injective* (or *one-to-one*) if no two inputs map to the same output: whenever $f(a_1) = f(a_2)$, we have $a_1 = a_2$.

An injection maps distinct inputs to distinct outputs—no two inputs share an output. Every input gets its own unique output. The function $f(x) = 2x$ on the integers is injective because different inputs always produce different outputs. The function $g(x) = x^2$ on $\mathbb{Z}$ is *not* injective because $g(3) = g(-3) = 9$ —two different inputs, same output.

Definition 1.8.2 (Surjection (onto)). A function $f : A \rightarrow B$ is *surjective* (or *onto*) if every element of B is hit: for every $b \in B$, there exists some $a \in A$ with $f(a) = b$.

A surjection means nothing in the codomain is left out. Every possible output actually gets produced by some input. If $f : \mathbb{Z} \rightarrow \mathbb{Z}$ is defined by $f(x) = 2x$, then f is *not* surjective because odd numbers are never hit. But $g : \mathbb{Z} \rightarrow \mathbb{Z}$ defined by $g(x) = x + 1$ is surjective because every integer n is $g(n - 1)$.

Definition 1.8.3 (Bijection). A function $f : A \rightarrow B$ is a *bijection* if it is both injective and surjective. Equivalently, f is a bijection if it has an inverse: a function $f^{-1} : B \rightarrow A$ such that $f^{-1}(f(a)) = a$ for all $a \in A$ and $f(f^{-1}(b)) = b$ for all $b \in B$.

A bijection is a perfect pairing—every element of A matches exactly one element of B , and every element of B matches exactly one element of A . When a bijection $f : A \rightarrow B$ exists, we say A and B have the same cardinality, written $|A| = |B|$.

For finite sets this lines up with our intuition: $\{a, b, c\}$ and $\{1, 2, 3\}$ have the same cardinality because we can pair them up (say $a \mapsto 1, b \mapsto 2, c \mapsto 3$). But the power of this definition is that it also works for infinite sets, where counting fails.

1.9 Countable infinity

$\mathbb{N} = \{0, 1, 2, 3, \dots\}$ is infinite. Are there other infinite sets that are “the same size” as $\mathbb{N}$?

Consider $\mathbb{Z}$, the integers: $\dots, -3, -2, -1, 0, 1, 2, 3, \dots$. At first glance $\mathbb{Z}$ looks “twice as big” as $\mathbb{N}$ because it extends in both directions. But that intuition is misleading. We can construct a bijection $f : \mathbb{N} \rightarrow \mathbb{Z}$ by zigzagging:

$$0 \mapsto 0, \quad 1 \mapsto -1, \quad 2 \mapsto 1, \quad 3 \mapsto -2, \quad 4 \mapsto 2, \quad 5 \mapsto -3, \quad 6 \mapsto 3, \quad \dots$$

More precisely:

$$f(n) = \begin{cases} n/2 & \text{if } n \text{ is even} \\ -(n+1)/2 & \text{if } n \text{ is odd} \end{cases}$$

This is a bijection. Every integer appears exactly once in the list $0, -1, 1, -2, 2, -3, 3, \dots$, so $|\mathbb{N}| = |\mathbb{Z}|$. Infinity is weird: a set that looks “twice as big” turns out to be exactly the same size.

It turns out that $\mathbb{Q}$, the rational numbers, is also the same size as $\mathbb{N}$. The proof uses a clever diagonal enumeration often called Cantor’s pairing function—you arrange the rationals in a grid and then zigzag through them. We won’t go through the details here, but the punchline is the same: you can list all the rationals, one by one, without missing any.

Definition 1.9.1 (Countably infinite). A set S is *countably infinite* if there is a bijection $f : \mathbb{N} \rightarrow S$. A set is *countable* if it is either finite or countably infinite.

Intuitively, a set is countable if you can write its elements as a (possibly infinite) list: a first element, a second element, a third, and so on, eventually hitting everything. The integers can be listed $(0, 1, -1, 2, -2, \dots)$, so they’re countable. The rationals can be listed too (with some cleverness), so they’re also countable.

Hilbert’s Hotel. Here’s a fun thought experiment that helps build intuition for why infinity behaves so strangely. Imagine a hotel with infinitely many rooms, numbered $1, 2, 3, \dots$, and every room is occupied. A new guest arrives. Can you fit them in?

Yes! Move every current guest from room n to room $n+1$. Room 1 is now free for the new guest. In fact, you can fit infinitely many new guests the same way: move guest n to room $2n$, freeing up all the odd-numbered rooms. This is essentially the same bijection trick we used above—infinity plus one is still the same infinity. It’s not that the hotel “got bigger”; it’s that the pairing between guests and rooms changed.

1.10 Uncountable infinity—Cantor’s diagonal argument

So $\mathbb{N}$, $\mathbb{Z}$, and $\mathbb{Q}$ are all the same size. You might start to wonder: is *every* infinite set countable? Maybe infinity just comes in one flavor?

No. The set $\mathbb{R}$ of real numbers is *not* countable. There are strictly more real numbers than natural numbers. This was proved by Cantor in 1891 with one of the most famous arguments in all of mathematics: the *diagonal argument*.

Theorem 1.10.1 (Cantor). The set of real numbers in the interval $[0, 1)$ is uncountable.

Proof. We proceed by contradiction. Assume, for the sake of argument, that we *can* list all the real numbers in $[0, 1)$. That means there's a bijection from $\mathbb{N}$ to $[0, 1)$, giving us an enumeration $r_1, r_2, r_3, \dots$ that includes every real number in $[0, 1)$.

Write each r_i in its decimal expansion:

$$\begin{aligned} r_1 &= 0.d_{11} d_{12} d_{13} d_{14} \dots \\ r_2 &= 0.d_{21} d_{22} d_{23} d_{24} \dots \\ r_3 &= 0.d_{31} d_{32} d_{33} d_{34} \dots \\ r_4 &= 0.d_{41} d_{42} d_{43} d_{44} \dots \\ &\vdots \end{aligned}$$

Now construct a new real number $d = 0.e_1 e_2 e_3 e_4 \dots$ by choosing each digit to *differ* from the corresponding diagonal entry. Specifically, set

$$e_n = \begin{cases} 5 & \text{if } d_{nn} \neq 5 \\ 6 & \text{if } d_{nn} = 5 \end{cases}$$

The number d is in $[0, 1)$, so it should appear somewhere in our list. But d differs from r_1 in the first digit, from r_2 in the second digit, from r_3 in the third digit, and in general from r_n in the n th digit. So $d \neq r_n$ for every n . This means d is *not* in our list—contradicting the assumption that the list was complete. $\square$

Notice the proof technique: we assumed something and derived a contradiction, so the assumption must be false. This is *proof by contradiction*, the same technique we used in Module 1 to show there are infinitely many natural numbers. We'll formalize this technique properly in Module 3.

The deeper lesson is the *method*: construct something that systematically disagrees with every entry in a supposed complete list. This “diagonalization” idea is incredibly powerful and shows up in several places across mathematics and computer science.

Forward look. This diagonal argument has the same structure as the proof that the halting problem is undecidable, which you'll see in CS 251. The idea of constructing something that disagrees with every entry in a list—a function that does the opposite of whatever the n th program does on input n —is exactly the same trick. If you understand Cantor's argument, you already understand the skeleton of one of the most important results in theoretical computer science.

1.11 Function spaces

Let's change gears. Given two sets A and B , it's natural to ask: how many functions are there from A to B ?

The set of *all* functions from A to B is written B^A (read “ B to the A ”).

Why the exponential notation? Because for finite sets, $|B^A| = |B|^{|A|}$. Here's why: to define a function $f : A \rightarrow B$, you need to decide, for each element of A , which element of B it maps to. That's $|B|$ choices for each of the $|A|$ elements, giving $|B|^{|A|}$ total functions.

Example 1.11.1. Let $A = \{a, b\}$ and $B = \{0, 1\}$. Then $|B^A| = 2^2 = 4$. The four functions are:

Function	$f(a)$	$f(b)$
f_1	0	0
f_2	0	1
f_3	1	0
f_4	1	1

That’s all of them. Each row is a complete function from A to B —for each input in A , it specifies exactly one output in B .

If you’re a Python programmer, you can think of B^A as the type `Dict[A, B]` where every key in A is required to be present. A function is just a dictionary with no missing keys.

You might also write the function space as $A \rightarrow B$ instead of B^A . Both notations are common. The exponential notation emphasizes the counting formula; the arrow notation emphasizes that we’re talking about functions.

1.12 The power set is a function space—the punchline

Here is the connection that ties this chapter together. Recall from Module 1 that the power set $\mathcal{P}(A)$ is the set of all subsets of A , and that $|\mathcal{P}(A)| = 2^{|A|}$ for finite sets. We said each element is either “in” a subset or “not in” it—two choices per element. But now we can say something much more precise.

Every subset $S \subseteq A$ can be described by a function $\chi_S : A \rightarrow \{0, 1\}$ called its *characteristic function* (or *indicator function*):

$$\chi_S(x) = \begin{cases} 1 & \text{if } x \in S \\ 0 & \text{if } x \notin S \end{cases}$$

Conversely, every function $f : A \rightarrow \{0, 1\}$ determines a subset $\{x \in A \mid f(x) = 1\}$. These two operations are inverses of each other: going from subset to characteristic function and back gives you the same subset, and going from function to subset and back gives you the same function.

In other words, there is a *bijection* between $\mathcal{P}(A)$ and $\{0, 1\}^A$. We write this as

$$\mathcal{P}(A) \cong 2^A$$

where 2 is shorthand for the two-element set $\{0, 1\}$.

Example 1.12.1. Let $A = \{a, b, c\}$. The subset $\{a, c\}$ corresponds to the characteristic function $\chi_{\{a, c\}} : A \rightarrow \{0, 1\}$ defined by:

$$a \mapsto 1, \quad b \mapsto 0, \quad c \mapsto 1$$

And the function $g : A \rightarrow \{0, 1\}$ with $g(a) = 0, g(b) = 1, g(c) = 0$ corresponds to the subset $\{b\}$.

Every subset gets a unique function, every function gets a unique subset, and nothing is left over on either side. That’s a bijection.

This explains *why* $|\mathcal{P}(A)| = 2^{|A|}$. It’s not just a coincidence or a counting trick—it’s because the power set literally *is* the function space 2^A , and function spaces have exponential cardinality:

$$|\mathcal{P}(A)| = |2^A| = |\{0, 1\}^A| = |\{0, 1\}|^{|A|} = 2^{|A|}$$

Subsets, characteristic functions, and bit strings of length $|A|$ are three descriptions of the same object. When you pick a subset of A , you're making a binary choice for each element—in or out, 1 or 0. That *is* a function to $\{0, 1\}$. That *is* a bit string. Three descriptions, one object.

Forward look: predicates. When we introduce predicates in Module 4, a predicate on a set A will be defined as a function $A \rightarrow \text{Bool}$. The truth set of the predicate—the set of elements making it true—is exactly the subset corresponding to this characteristic function. So predicates, subsets, and functions to Bool are three views of a single concept.

Cantor's theorem and the hierarchy of infinities. Here's something to chew on. Cantor proved that $|\mathcal{P}(A)| > |A|$ for *any* set A —even infinite ones. There is no bijection between a set and its power set; the power set is always strictly bigger. (The proof is another diagonal argument!)

Combined with $\mathcal{P}(\mathbb{N}) \cong 2^{\mathbb{N}}$, this gives us an infinite hierarchy of infinities, each strictly larger than the last:

$$|\mathbb{N}| < |2^{\mathbb{N}}| < |2^{2^{\mathbb{N}}}| < \dots$$

The first level is the countable infinity we've been working with. The second, $|2^{\mathbb{N}}|$, turns out to equal $|\mathbb{R}|$ —the cardinality of the real numbers. There are strictly more real numbers than natural numbers (we proved this!), and strictly more subsets of the reals than reals, and so on forever. Infinity isn't one thing — it's an entire hierarchy, and it goes up forever.

1.13 Exercises

Exercise 1. List all elements of $\mathcal{P}(\{1, 2, 3\})$. How many are there?

Exercise 2. How many functions are there from $\{a, b, c\}$ to $\{0, 1\}$? List them by giving the characteristic function (that is, the values $f(a), f(b), f(c)$) for each subset of $\{a, b, c\}$.

Exercise 3. Prove that the set of even natural numbers $E = \{0, 2, 4, 6, \dots\}$ is countably infinite by giving an explicit bijection $f : \mathbb{N} \rightarrow E$. Verify that your function is both injective and surjective.

Chapter 2

Propositional Logic

2.1 Logical formulas, formalized

With all our mathematical preliminaries out of the way, we're ready to start dealing with logic more formally. Early attempts at talking about logic were focused on rhetoric and how to make arguments in human language. They were still about reasoning but didn't treat it in the *symbolic* way we do in modern times, where we turn logic into formulas and equations and symbols.

In this class, I'm not only going to take a symbolic approach to logic like Epp but I'm going to take a very "computer sciencey" approach, discussing logic as though it were a programming language.

There are many possible logics — yes, plural, because there are many possible logics just as there are many possible programming languages, each serving its own purpose. The standard way to introduce any logic is to define its *syntax* and its *semantics*.

The *syntax* of the logic is a lot like when we talk about the syntax of any programming language: what combinations of symbols make a "valid" program/formula. The syntax tells us how we're allowed to build formulas in our logic.

The *semantics* of a logic tells us what the formulas of the logic **mean**. In ordinary programming languages, the compiler or interpreter implements the semantics: it defines what a syntactically valid program does when executed. For (most) logics, the semantics tells us how to take a formula and turn it into a mathematical function that takes in arguments and returns either "true" or "false".

What do true and false mean here? That question could fill a PhD thesis, but for our purposes: we can define "true" as "reflects reality" and "false" as "contradicts reality". For most of this class, you can think of "true" and "false" a lot like the boolean types you find in every programming language.

The first logic we're going to deal with is really simple. It only has booleans as data, variables, and a few logical connectives: and ($\wedge$), or ($\vee$), not ($\sim$), and implication ($\rightarrow$). (We'll use $\neg$ for negation in our mathematical notation; some textbooks, including Epp, write $\sim$ instead. They mean the same thing.) The idea of this logic is that it encompasses basic operations for how to connect statements about the world and their relationship to each other. It's all verbs, no nouns, but it still helps give some intuition of how reasoning works.

We're going to express this using something called Backus-Naur form (https://en.wikipedia.org/wiki/Backus-Naur_form) which is a standard notation for describing grammars of formal languages, like this one, and something you'll see more of as you study computer science.

Definition 2.1.1 (Propositional Logic Syntax). The syntax of propositional logic is given by the following BNF grammar:

$$L := T \mid F \mid L \wedge L \mid L \vee L \mid \neg L \mid L \rightarrow L \mid x$$

BNF is a way to describe how to build things recursively. It tells us, in this case, that to build an “L” you have a bunch of possibilities separated by the $|$ symbol. You can follow this recursively, continuing to substitute in things for any L symbols that appear until you only have base cases (here: variables, T, and F).

When a formula is ambiguous—like $p \wedge q \vee r$, which could be read as $(p \wedge q) \vee r$ or $p \wedge (q \vee r)$ —we follow the standard precedence: $\neg$ binds tightest, then $\wedge$, then $\vee$, then $\rightarrow$. We use parentheses to override precedence when needed. This is similar to how $*$ binds tighter than $+$ in arithmetic.

Here are some examples to help clarify how this is used. This grammar tells us that

- T is a valid formula
- F is a valid formula
- $T \wedge (x \vee y)$ is a valid formula
- $T \rightarrow L$ is not a valid formula because there’s still an L
- $T + F$ is not a valid formula because $+$ isn’t an allowed symbol in the syntax
- heck is a valid formula, a single variable named heck

That’s the syntax. We need to define the *semantics* of this logic. We’ve already named the pieces, so let’s give an informal semantics before presenting the formal one:

- T is just T, the boolean value for true
- F is just F, the boolean value for false
- $p \wedge q$ is *and* and is true when p and q are both true
- $p \vee q$ is *or* and is true when **either** p or q are true, but **both** can be true
- $\neg p$ is *not* and is true when p isn’t
- $p \rightarrow q$ is *implication* and captures the idea that p being true forces q to be true. Specifically, it’s false only when p is true and q is false
- variables are true when you plug in a true value into them

That’s the informal semantics. What’s the *formal* semantics? We’re going to explain it in terms of **truth tables**.

A truth table is a way of describing the *graph* of the function that a formula represents. Since all the arguments to these functions are just booleans and can be enumerated easily, the graph can be represented as a table.

Let’s start with variables.

x	x
T	T
F	F

A variable is just an identity function—it returns its input unchanged.

From here, we can build up the meaning of any formula piecemeal and describe the meaning of connectives in terms of “p” and “q” as subformulas, so when we write something like this:

p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

what we’re saying is that if we’ve already evaluated the meaning of “p” and “q” we can plug in their values and get the meaning of when they’re connected by an *and*. So we can read this table off and it matches the informal meaning we described above: $p \wedge q$ is true exactly when p is true and q is true and false otherwise.

Next, $p \vee q$:

p	q	$p \vee q$
T	T	T
T	F	T
F	T	T
F	F	F

Which tells us that $p \vee q$ is true when either p or q is true (including both of them).

Next, negation:

p	$\neg p$
T	F
F	T

Now we get to the counterintuitive one: implication. The intuition is that we want $p \rightarrow q$ to be true if p being true makes q be true—in other words, I can take the knowledge that p is true and then conclude that q is true.

(Sidenote: there’s a sense, that we might get into later in this course, that implication really is a function that takes in a proof that p is true and turns it *into* a proof that q is true. Keep that idea in mind.)

Do I care if q is true without p being true? In our first choice of logic, the answer is “no”. We’re going to make the decision that $F \rightarrow p$ is true no matter what p is, because the case where the premise is false is irrelevant to what we want implication to capture. This has some odd implications! From a false premise you can conclude anything. We’ll dig into that more as we go.

For now, the truth table:

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

With these truth tables in hand, let's show how to combine them together in an example:

x	y	z	$((x \vee y) \wedge z) \rightarrow x$
T	T	T	$((T \vee T) \wedge T) \rightarrow T \implies T$
T	T	F	$((T \vee T) \wedge F) \rightarrow T \implies F \rightarrow T \implies T$
T	F	T	$((T \vee F) \wedge T) \rightarrow T \implies T$
T	F	F	$((T \vee F) \wedge F) \rightarrow T \implies F \rightarrow T \implies T$
F	T	T	$((F \vee T) \wedge T) \rightarrow F \implies T \rightarrow F \implies F$
F	T	F	$((F \vee T) \wedge F) \rightarrow F \implies F \rightarrow F \implies T$
F	F	T	$((F \vee F) \wedge T) \rightarrow F \implies F \rightarrow F \implies T$
F	F	F	$F \rightarrow F \implies T$

Whew! That was a lot, but hopefully you can see that, in principle, you could use truth tables to calculate the meaning of any formula as a function.

Exercise 1. Build the truth table for $(p \rightarrow q) \wedge (q \rightarrow p)$. This combination will come up later when we define the biconditional ($\leftrightarrow$)—you'll see why.

2.2 Tautologies and equivalences

Having talked about our very first logic, which is usually called “propositional logic”, we can start discussing some of its implications and structure.

Definition 2.2.1 (Tautology). A *tautology* is a formula whose meaning is **always** true, no matter what values you put into all the variables.

For example, for any formula p we have that $p \vee \neg p$ is a tautology! Let's check the truth table:

p	$p \vee \neg p$
T	$T \vee F = T$
F	$F \vee T = T$

No matter what value the sub-formula p takes, you get true.

Definition 2.2.2 (Contradiction). A *contradiction* is a formula whose meaning is **always** false, no matter what values you put into the variables.

For example, $p \wedge \neg p$ is a contradiction—there's no assignment that makes it true. Contradictions are the dual of tautologies and will play a key role when we discuss proof by contradiction in the next module.

Since propositional logic is a really simple logic, one that only has booleans as data, the tautologies of propositional logic aren't individually profound, but they tell us important things about valid reasoning. The above tautology tells us that for any formula—for any statement about the world—either the claim is true or it is false.

Maybe that doesn't seem profound, but this is called “the law of the excluded middle” and is actually pretty controversial in the history of logic! In *this* propositional logic and *this* notion of “truth”, however, we can always conclude that if something isn't false it must be true.

Why controversial? There's a tradition in logic—called *constructive* or *intuitionistic* logic—that refuses to accept $p \vee \neg p$ as an axiom. The idea is that to prove “ A or B ” you should have to

actually show *which one* is true, not just argue that one of them must be. If you think about it from a programming perspective, this makes a lot of sense: a constructive proof of “ A or B ” is like a program that actually *computes* which case holds and hands you the evidence. Classical logic, the kind we’re using, lets you say “well, one of them *has* to be true” without ever telling you which—you get an existence result but not an algorithm. That connection between proofs and programs turns out to be really deep and beautiful, and it’s one of the reasons logic matters so much to computer science. In this course we’ll stick with classical logic (with excluded middle) because that’s what Epp uses and what most intro courses assume, but fair warning: your instructor has a constructive bias, and it will probably peek through from time to time.

We’ll come back to tautologies a bit later when we want to talk about reasoning and proof a little more explicitly, but for now just keep in mind the idea that they provide templates for what valid reasoning looks like.

Another thing we can do with truth tables is calculate *equivalences*—we can figure out when formulas are equivalent to each other.

Definition 2.2.3 (Logical Equivalence). Two formulas are *logically equivalent* exactly when they have the same truth table.

Since we’re defining the meaning of a formula **as its truth table**, this is a natural notion of equivalence.

We can also express equivalence *inside* the logic by defining a shorthand: $p \leftrightarrow q$ means $(p \rightarrow q) \wedge (q \rightarrow p)$. This is called the *biconditional* and reads “if and only if.” It is **not** a primitive connective in our grammar—it’s just an abbreviation for a conjunction of two implications. You can compute its truth table by expanding the definition:

p	q	$p \leftrightarrow q$ (i.e. $(p \rightarrow q) \wedge (q \rightarrow p)$)
T	T	T
T	F	F
F	T	F
F	F	T

So the biconditional is true exactly when p and q have the same truth value—you can think of it like an equality test on booleans. Saying “if and only if” is the same as saying the implication goes both ways.

As an example, we’re going to prove two formulas that end up being useful in some optimizations: de Morgan’s laws! (https://en.wikipedia.org/wiki/De_Morgan%27s_laws)

Let’s start with the law that $\neg(p \wedge q) \equiv (\neg p) \vee (\neg q)$. We’ll calculate both of their truth tables and compare!

p	q	$\neg(p \wedge q)$
T	T	F
T	F	T
F	T	T
F	F	T

and

p	q	$(\neg p) \vee (\neg q)$
T	T	F
T	F	T
F	T	T
F	F	T

These are clearly the same!

Exercise 2. Prove the second De Morgan’s law $\neg(p \vee q) \equiv (\neg p) \wedge (\neg q)$ by computing both truth tables.

De Morgan’s laws tell us how negation interacts with $\wedge$ and $\vee$: pushing $\neg$ inside a conjunction or disjunction swaps one connective for the other.

This raises a natural question: how few operations do you actually need? This is where we introduce the concept of “functional completeness”: how few operations do you need to recreate all the logical operations on booleans?

For example, we have the operation $p \rightarrow q$. To recall, this operation has the following truth table:

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

Now it turns out this is identical to the truth table of $(\neg p) \vee q$

p	q	$(\neg p) \vee q$
T	T	$F \vee T = T$
T	F	$F \vee F = F$
F	T	$T \vee T = T$
F	F	$T \vee F = T$

The truth tables match! Technically, we don’t even need implication in this particular logic—though this equivalence is specific to classical propositional logic and does not hold in general if we have $\neg$ and $\vee$.

This means that just having $\vee$, $\wedge$, and $\neg$ is “functionally complete” because it can describe all the operations of this logic.

Exercise 3. Show that XOR ($p \oplus q$) can be expressed using only $\wedge$, $\vee$, and $\neg$. Hint: compare truth tables.

2.3 Addendum: nand, xor, and how many binary operations are there?

There are some additional operations that you *can* define that we haven’t defined yet.

The first of these we’ll call xor (sometimes shown with the $\oplus$ symbol), the *exclusive* or and it has the following truth table:

p	q	$p \oplus q$
T	T	F
T	F	T
F	T	T
F	F	F

This is true if *either* p or q is true but not when *both* are true.

The second of these is *nand*, which is just “not-and” and it’s rather important to the design of circuits:

p	q	$p \text{ nand } q$
T	T	F
T	F	T
F	T	T
F	F	T

An obvious question, and one that comes up in the homework of the accompanying class to these notes, is *how many of these operations are there and what are they?*

We can approach this *exhaustively*. A binary operation takes two (2) arguments, each of which is a boolean and can take two values. That’s where we get the form of the truth tables above!

p	q	$f(p, q)$
T	T	?
T	F	?
F	T	?
F	F	?

How many different binary operations are even definable? That’s equivalent to asking how many different truth tables are possible. There’s a distinct truth table for every choice of how we fill in those four output entries in the table above. Well, okay, we know how to do this! Each of those four slots involves making a choice between one of two things: T and F. So the total number of choices must be $2 \times 2 \times 2 \times 2 = 2^4$ which is 16.

To prove *functional completeness* of a set of boolean operators, you need to show that for any possible truth table you can find a way to replicate it using those operators. Let’s do a concrete example:

Here’s our truth table for implication

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

Now let’s show that with just or, and, and not we can replicate the implication operator:

p	q	$(\neg p) \vee q$
T	T	T
T	F	F
F	T	T
F	F	T

Solving these little puzzles is the point of the homework this week.

Exercise 4. How many *unary* boolean operations are there? List them all.

Chapter 3

Proof Systems for Propositional Logic

3.1 Tautologies as reasoning

We’ve talked through a few different things about the *syntax* and the *semantics* of our first logic, propositional logic, and we’ve alluded to this idea that tautologies—formulas whose truth tables are always true—tell us **something** about what counts as valid reasoning.

Now it’s time to make that truly concrete and help explain some of these connections between natural human intuition and reasoning, informal mathematics, and formalized reasoning in a logic.

Consider the formula $p \wedge q \rightarrow p$. You can read this as “if p and q are true then p is true”. A reasonable assertion!

But reasonable isn’t good enough. We need to show that this is always the case, and we can do that by calculating its truth table.

p	q	$p \wedge q \rightarrow p$
T	T	T
T	F	$F \rightarrow T \implies T$
F	T	$F \rightarrow F \implies T$
F	F	$F \rightarrow F \implies T$

This is, in fact, a *tautology*.

But we can also interpret this tautology as saying something like

If we start with the premises p and q then we can conclude p . This represents a shift between a logical and a metalogical notion of reasoning. In one, we use the literal implication operator $\rightarrow$ and in the other we use the intuitive notion of *if* and *then*.

This goes the other way too. For example, consider that most famous of reasoning steps “modus ponens”—¹ Modus ponens is simply this:

If p is true, and I know that p *implies* q , then I can conclude q .

It’s a pretty intuitive concept because it just formalizes the meaning of “implication”: that p being true *forces* q to be true.

A different way to write modus ponens is to take the approach the book takes, which will look remarkably like the way statements of theorems and proofs will look once you get to the Natural Number Game later in the class.

¹I swear it’s a red flag if someone says “modus ponens” in casual conversation, even when that conversation is about mathematics and logic.

(a) p
 (b) $p \rightarrow q$
 show: q

This form is convenient because it makes clear what formula corresponds to the theorem. To build it: conjoin all the premises with $\wedge$, then put a $\rightarrow$ between that conjunction and the conclusion.

Here the corresponding formula, which we hope to show is a tautology, would be $(p \wedge (p \rightarrow q)) \rightarrow q$.

Let's justify this by taking a look at its truth table

p	q	$(p \wedge (p \rightarrow q)) \rightarrow q$
T	T	T
T	F	$(T \wedge (T \rightarrow F)) \rightarrow F \implies T$
F	T	$(F \wedge (F \rightarrow T)) \rightarrow T \implies T$
F	F	$(F \wedge (F \rightarrow F)) \rightarrow F \implies T$

This truth table is all Ts, so it *is* in fact a tautology! This means that we've described a valid method of proof after all.

This isn't surprising — we defined implication to behave exactly this way.

3.2 A thorough derivation of proof by contradiction

Before we cover the remaining proof methods, let's dig into *proof by contradiction* in detail, which you might recognize from our very first unit in this class when we proved that there were an infinite number of natural numbers. To do that we need to explore the *other* Latin proof method: modus tollens.

Modus tollens is a way of dealing with implication and things that *aren't* true. In the semi-formal proof structure, modus tollens says that

(a) $p \rightarrow q$
 (b) $\sim q$
 shows: $\sim p$

Let's talk about what this means before we jump into the truth table: if we know that p implies q , but that q isn't true, we then know that p *can't* be true because if it was true it would force q to be true. For example, if you know that if you get over 80% on the final exam you'll get an A in the class, but you see that your final grade was an AB, then you *know* that you got less than 80% on the final.

Again, this is just a restatement of our intuition about what implication must mean, but in the negative rather than positive sense.

But let's check that it's a tautology and thus valid reasoning

p	q	$((p \rightarrow q) \wedge \neg q) \rightarrow \neg p$
T	T	$((T \rightarrow T) \wedge F) \rightarrow F \implies F \rightarrow F \implies T$
T	F	$((T \rightarrow F) \wedge T) \rightarrow F \implies F \rightarrow F \implies T$
F	T	$((F \rightarrow T) \wedge F) \rightarrow T \implies F \rightarrow T \implies T$
F	F	$((F \rightarrow F) \wedge T) \rightarrow T \implies T \rightarrow T \implies T$

Good — this is firmly a tautology.

What about proof by contradiction? This is where it's important to remember that the p and q aren't necessarily variables but are generalized sub-formulas—you can stick another formula inside there and the result should be valid. Since the tautologies are *always* true it doesn't even matter what values the sub-formula take.

Proof by contradiction is, essentially,

(a) $\sim p \rightarrow r \wedge \sim r$

shows: p

In other words, if assuming p is false leads to a contradiction (that you can conclude—for some r —that r is true *and* $\sim r$ is true) then you can conclude p . How is this modus tollens? Well we take modus tollens and substitute in $\sim p$ for the p in the formula and $r \wedge \sim r$ instead of q .

We then get the formula—which again, we know is a tautology—

$((\sim p \rightarrow (r \wedge \sim r)) \wedge \sim(r \wedge \sim r)) \rightarrow \sim\sim p$

This is almost the right form but look at that goofy $\sim(r \wedge \sim r)$ in there! By De Morgan's laws we have that this should actually become

$\sim r \vee \sim\sim r$

which still contains $\neg\neg r$. We should verify double negation with a truth table rather than taking it for granted.

p	$\neg\neg p$
T	$\neg F \implies T$
F	$\neg T \implies F$

Okay, so not-not p is p . Neat. In which case the $\sim(r \wedge \sim r)$ part of our expression becomes $\sim r \vee r$, and we recognize that one as a tautology we saw earlier! The thing about it being a tautology is that it means we can just substitute T for it instead so our formula for proof by contradiction becomes

$((\sim p \rightarrow (r \wedge \sim r)) \wedge T) \rightarrow \sim\sim p$

which can simplify even further to

$(\sim p \rightarrow (r \wedge \sim r)) \rightarrow \sim\sim p$

and since $\sim\sim p$ is equivalent to p , this is exactly the proof-by-contradiction rule we wanted.

And since we did this entirely by substituting formula into a tautology and simplifying, we don't even need to check the truth table to guarantee correctness!

Notice that this derivation relied on double negation elimination—the step where we replaced $\sim\sim p$ with p —which is equivalent to excluded middle. This makes full proof by contradiction a *classical* technique; it doesn't work in constructive logic, where you can't just erase double negations. What about proof by contrapositive? The situation is subtler than it might seem. Modus tollens—from $P \rightarrow Q$ and $\neg Q$, conclude $\neg P$ —is constructively valid. But “proof by contrapositive” as a technique means proving $\neg Q \rightarrow \neg P$ to establish $P \rightarrow Q$, and that equivalence also requires excluded middle. So both contradiction and contrapositive, as proof techniques, are classical. The difference is that contrapositive proofs tend to be more *informative*—they show you the structure of the argument rather than just ruling out alternatives. We'll revisit this distinction more carefully in Module 7.

3.3 A proof system for propositional logic

Do we *always* have to prove things within the logic by appealing to truth tables? Truth tables work fine for propositional logic, because in this logic all our variables are booleans and can take one of two values: T or F.

In the next module, though, we’re going to add one of the simplest possible new types of data to our logic that aren’t just booleans: the natural numbers, i.e. 0, 1, 2, &c.

As we proved together in our very first week of class there are an infinite number of natural numbers so already we can no longer check “truth tables” of every possible value of every variable in order to determine whether the statement is true.

No, instead, we need to have a system for doing *proofs* within the logic, methods of reasoning that we know are “sound”—or in other words will always produce logically valid results no matter how they’re chained and combined together.

Now, we can’t just make up these proof rules out of nothing: we need to have some way of checking that these are valid in the first place, and that’s where the *semantics* of a logic are necessary. We can check that the rules of proof work with the semantics and don’t do anything wild like let you conclude false things from true ones.

With that motivation in hand, let’s list out the proof rules and check them against the truth-table semantics, which in this case means checking truth tables.

We’ve already talked about a couple of the rules: modus ponens and modus tollens, the rules for implication.

But for every binary operator we’ve seen there are rules for how to build it and how to *use* it. These are called introduction and elimination rules, respectively.

Psst: dear reader, if you’re wondering what the introduction rule for implication is, we’ll come back to that at the end of this section.

3.3.1 Rules for conjunction

Introduction:

(a) p
 (b) q
 shows $p \wedge q$

In other words, if you know that p is true and q is true then you know that p and q is true. Unsurprising!

In terms of the validation of this rule this means that we’d need to show that

$p \wedge q \rightarrow p \wedge q$
 is a tautology

But this is obvious because we can easily show that $a \rightarrow a$ for any subformula a that we choose to substitute in.

a	$a \rightarrow a$
T	$T \rightarrow T = T$
F	$F \rightarrow F = T$

Elimination: There are two elimination rules for $\wedge$:

(a) $p \wedge q$
 shows p

(a) $p \wedge q$
 shows q

These are unsurprising.

Exercise 1. Validate that $p \wedge q \rightarrow q$ is a tautology by computing its truth table.

3.3.2 Rules for $\vee$

Since $\vee$ is the mirror of $\wedge$, it's not surprising that there are going to be two introduction rules and one elimination rule.

Introduction:

(a) p
shows $p \vee q$

(a) q
shows $p \vee q$

These just follow from the very definition of what $\vee$ is as an operator and you can verify that $p \rightarrow p \vee q$ and $q \rightarrow p \vee q$ are both tautologies.

Elimination: What does it *mean* to prove that $(p \vee q) \rightarrow r$? You don't know which of p and q is true, so what you're saying is "whether p or q is true I know r is true" which we can break down like this:

(a) $p \vee q$
(b) $p \rightarrow r$
(c) $q \rightarrow r$
shows r

And we can validate this with a truth table like so (this is going to be a little ugly so hold on)

p	q	r	$((p \vee q) \wedge (p \rightarrow r) \wedge (q \rightarrow r)) \rightarrow r$
T	T	T	$(T \wedge T \wedge T) \rightarrow T \implies T$
T	T	F	$(T \wedge F \wedge F) \rightarrow F \implies T$
T	F	T	$(T \wedge T \wedge T) \rightarrow T \implies T$
T	F	F	$(T \wedge F \wedge F) \rightarrow F \implies T$
F	T	T	$(T \wedge T \wedge T) \rightarrow T \implies T$
F	T	F	$(T \wedge T \wedge F) \rightarrow F \implies T$
F	F	T	$(F \wedge T \wedge T) \rightarrow T \implies T$
F	F	F	$(F \wedge T \wedge T) \rightarrow F \implies T$

What does this rule actually tell us? It is, as the book points out, the formal logic of *proof by cases*. Proof by cases is something we've already informally encountered in the class and it's implicitly what works behind homework 2: for each of the 16 possible truth tables of binary operators, you show that you can prove the proposition, so you know it's true for all of them (all in this case functioning like a Big Or).

And here's something worth flagging for later: when we eventually see the meaning of $\forall$ and $\exists$, there is a sense in which $\forall$ really is an "infinitely big $\wedge$ " and $\exists$ is an "infinitely big $\vee$."

3.3.3 Negation

This is a fun one because we've essentially seen the rules for this already.

Introduction:

(a) assuming p leads to a contradiction
shows $\sim p$

In other words, if assuming p lets you derive both q and $\neg q$ for some q , then you can conclude $\neg p$. This is sometimes called “proof by negation” to distinguish it from full proof by contradiction. It’s worth noting that in a lot of logics negation is *defined* as $\sim p = p \rightarrow F$. One such logic is the logic behind Lean 4, which you’ll be playing with as part of learning The Natural Number Game.

Elimination:

(a) $\sim\sim p$
shows p

This is double negation elimination, and as we discussed in the proof-by-contradiction section, it requires excluded middle. In constructive logic, you can always go from p to $\neg\neg p$, but not the other way.

3.3.4 T and F

There are also introduction and elimination rules for the constants T and F as well. What’s fun about this is that there’s *only* an introduction rule for T and *only* an elimination rule for F.

Introduction (T):

shows T

That’s it — you can just conclude T. It needs no premises. And there’s no elimination rule because knowing T gives you no new information.

Elimination (F):

(a) F
shows p

Wait, what? What is p here? Well it’s literally anything. Because if you assume false you can conclude anything. To understand this, follow our rules for how to convert a proof in the above form into its corresponding tautology and then look at what the truth table would be.

3.3.5 Implication

Okay, we saved implication for last because it’s... a little weird.

Elimination: The elimination rule for implication is just modus ponens. That’s easy.

Introduction: It’s the introduction that’s weird. Because it’s more like:

(a) has a proof that assuming p , shows q
show $p \rightarrow q$

Why is this weird? Because we have two layers: *implication* inside the logic, and *proofs* which take a set of premises and derive a conclusion, which is kind of a *metalogical* activity.

The introduction rule of implication lets us take a proof—a metalogical thing—and turn it into an implication *within* the logic. Then the elimination rule lets us take an implication within the logic and use it inside a proof.

3.3.6 Extra proof rules

There are a handful of proof rules we’re going to introduce to help us out, based on tautologies: an important one

shows $(p \vee \sim p)$

with no assumptions

This is going to help us deal with a bunch of proofs by giving us the ability to, essentially, use subformulas without any assumptions.

We're going to call this rule "excluded middle"

It's worth pausing to note that this rule is a *choice*. The introduction and elimination rules for conjunction, disjunction, and implication are all constructively valid, as is negation *introduction* (proof by negation). But negation *elimination* (double negation elimination) requires excluded middle. Excluded middle is the specific axiom you add on top of those structural rules to get *classical* logic. Without it, you have what's called intuitionistic or constructive logic, where every proof of a disjunction $p \vee q$ must actually demonstrate *which* of p or q holds. We're choosing to include excluded middle in this course, but it's worth knowing that this is precisely the thing that separates classical from constructive reasoning.

Also we have the maybe obvious rule of proof by assumption:

(a) p
shows p

This is just a rule that lets us formally use our assumptions in our proofs.

3.4 Arguments and validity

Now we need to actually talk about what it means for an argument to be *valid*, because this is something that the assignments lean on heavily and it's worth being really precise about.

An *argument form* is just a sequence of statements called *premises* followed by a *conclusion*. We write the premises on top, draw a little horizontal line, and then write the conclusion below it with a $\therefore$ symbol. For example, here's the argument form for modus ponens:

$p \rightarrow q$
 p

therefore q

When is an argument form *valid*? An argument form is valid if whenever **all** the premises are true, the conclusion is also true. Another way to say this: take the conjunction of all the premises, form its implication to the conclusion, and check whether *that* whole formula is a tautology. If it is, the argument is valid. If it's not, the argument is invalid and there exists some assignment of truth values where all the premises hold but the conclusion doesn't.

How do we actually *test* validity? You build a truth table for all the variables involved, find the rows where every single premise evaluates to true—these are the *critical rows*—and then check whether the conclusion is also true in those rows. If the conclusion is true in every critical row, valid. If there's even one critical row where the conclusion is false, invalid.

Let's work through an example. Consider the argument $p \vee q, \sim q \therefore p$ —this is called *disjunctive syllogism* and we listed it in the derivable rules section below, but let's actually verify it.

p	q	$p \vee q$	$\neg q$	p
T	T	T	F	T
T	F	T	T	T
F	T	T	F	F
F	F	F	T	F

The critical rows are the ones where *both* premises are true, i.e. where $p \vee q$ is T *and* $\sim q$ is T. That's only the second row ($p=T, q=F$). And in that row the conclusion p is T. So the argument is valid!

Now here's where things get interesting, because there are two extremely common *fallacies*² that trip people up constantly:

Converse error (also called “affirming the consequent”): this is the argument form

```
p -> q
q
-----
therefore p
```

This is **invalid**. Here's the intuition: “if it's raining then the ground is wet; the ground is wet; therefore it's raining.” But the sprinklers could be on! Just because the ground is wet doesn't mean it rained. The problem is that you're confusing $p \rightarrow q$ with its *converse* $q \rightarrow p$, and an implication and its converse are **not** logically equivalent (you can verify this by comparing their truth tables).

Inverse error (also called “denying the antecedent”): this is the argument form

```
p -> q
~p
-----
therefore ~q
```

Also **invalid**. “If it's raining then the ground is wet; it's not raining; therefore the ground is not wet.” Again, sprinklers. You're confusing $p \rightarrow q$ with its *inverse* $\sim p \rightarrow \sim q$, which again is not logically equivalent to the original.

One nice thing about all of this machinery is that you can *chain* argument forms together to build longer deductions. This is what a formal proof really is: you start with your premises, apply rules of inference one step at a time, and each step produces a new known fact that you can use in later steps. Think of it like writing a program where each line is a function call that's justified by one of our rules—modus ponens, disjunctive syllogism, &c.—and the “return value” feeds into the next line. You keep going until you reach the conclusion.

Here's a quick example of chaining. Suppose our premises are $p \rightarrow q$, $q \rightarrow r$, and p . We want to conclude r . The deduction looks like:

```
1. p -> q premise
2. q -> r premise
3. p premise
4. q modus ponens on 3, 1
5. r modus ponens on 4, 2
```

Each line is justified by exactly one rule and refers back to earlier lines. That's the whole game. It's very much like a sequence of statements in a program where each variable depends on previously computed values.

Exercise 2. Determine whether the following argument is valid or invalid. If invalid, find a counterexample row. Premises: $p \rightarrow q$, $r \rightarrow \neg q$, r . Conclusion: $\neg p$.

²A fallacy is just an argument form that looks plausible but is actually invalid. The whole point of formal logic is to catch these things before they sneak into your reasoning.

3.5 Examples of doing proofs

Now let's dig into some proofs and see how they actually work. For example, let's see if we can *prove* some laws:

Proof systems—formal proofs in a logic—are going to look a little different than the kind of thing we've done before.

Let's make an important derived rule: transitivity of implication

Theorem: Assuming $p \rightarrow q$, $q \rightarrow r$, show $p \rightarrow r$
Proof:
(a) $p \rightarrow q$, by assumption
(b) $q \rightarrow r$, by assumption
(c) proof that assuming p , shows r
 subproof: (i) p , by assumption
 (j) q , by modus ponens with (i) and (a)
 show r , by modus ponens with (j) and (b)
shows $p \rightarrow r$ by introduction of implication

Theorem: Assuming $p \rightarrow q$, show $\sim p \vee q$
Proof:
(a) $p \rightarrow q$, by assumption
(b) $p \vee \sim p$, by excluded middle
(c) $q \rightarrow \sim p \vee q$, by introduction of $\rightarrow$ and $\vee$
(d) $p \rightarrow \sim p \vee q$, by transitivity of (a) and (c)
(e) $\sim p \rightarrow \sim p \vee q$, by introduction of $\rightarrow$ and $\vee$
shows $\sim p \vee q$, by elimination of $\vee$ with (b), (d), (e)

3.5.1 Conjunctive normal form

A formula is in **conjunctive normal form** (CNF) when it is a conjunction of *clauses*, where each clause is a disjunction of *literals*. A literal is either a variable or its negation (e.g. p or $\sim p$).

For example

$$(p \vee \sim q) \wedge (\sim p \vee q \vee r) \wedge (q)$$

is in conjunctive normal form: it is a conjunction of three clauses, each of which is a disjunction of literals.

On the other hand,

$$\sim(p \vee q)$$

is *not* in conjunctive normal form because $\sim(p \vee q)$ is not a disjunction of literals. However, by De Morgan's laws it can be rewritten as

$$(\sim p) \wedge (\sim q)$$

and *this* is in CNF (a conjunction of two single-literal clauses).

3.5.2 Disjunctive normal form

A formula is in **disjunctive normal form** (DNF) when it is a disjunction of *terms*, where each term is a conjunction of *literals*. So the following is in disjunctive normal form

$$(p \wedge \sim q) \vee (\sim p \wedge q \wedge r) \vee (q)$$

but the following is not because $\sim(q \wedge r)$ is not a conjunction of literals

$$(p \wedge q) \vee (\sim(q \wedge r))$$

3.6 Digital logic circuits

Here's where things get fun and we connect all this abstract boolean algebra stuff to the real physical world. It turns out that the logical operations we've been studying—AND, OR, NOT—have direct physical implementations called *logic gates*. An AND gate is a little electronic component that takes two input signals and produces an output that's 1 (high voltage) only when both inputs are 1. An OR gate outputs 1 when at least one input is 1. A NOT gate (also called an *inverter*) takes one input and flips it. There are also NAND gates, which are just an AND followed by a NOT, and it turns out these are particularly important because you can build literally every other gate out of just NAND gates.³

A *circuit* is just a composition of gates—you wire the output of one gate into the input of another. If you think about it this is exactly function composition! The output of gate f becomes the input of gate g , so you're computing $g(f(x))$. It's functions all the way down.

To *evaluate* a circuit you trace signals from the inputs through each gate to the output, applying the truth table of each gate as you go. Let's do an example: suppose we have inputs P and Q, and the circuit works like this. Q branches into two paths: one path goes to an OR gate together with P, and the other path goes through a NOT gate. Then the output of the OR gate and the output of the NOT gate both feed into an AND gate, and that's our final output.

With P=1 and Q=0: the OR gate computes $\text{OR}(1,0) = 1$, the NOT gate computes $\text{NOT}(0) = 1$, and the AND gate computes $\text{AND}(1,1) = 1$. So the circuit outputs 1 for that input.

We can fill out the whole truth table for this circuit by tracing through all four input combinations:

P	Q	$P \vee Q$	$\neg Q$	$(P \vee Q) \wedge \neg Q$	Output
1	1	1	0	0	0
1	0	1	1	1	1
0	1	1	0	0	0
0	0	0	1	0	0

This circuit computes the function that's true exactly when P is true and Q is false—which is just $P \wedge \sim Q$. Interesting that our three-gate circuit could have been built with two gates! This is the kind of optimization that boolean algebra helps us find.

But here's the really cool part: how do you *build* a circuit from a truth table? This is where disjunctive normal form comes back in a big way. Remember DNF? Here's the recipe:

1. Look at every row of the truth table where the output is 1.
2. For each such row, build an AND of all the input variables—but negate any variable that's 0 in that row. This AND gate “recognizes” exactly that one row.
3. OR all of those ANDs together.

³This is called functional completeness and it's honestly one of the coolest facts in all of computer science. Your entire computer could, in principle, be built out of nothing but NAND gates.

For instance, if you have a truth table with inputs P , Q and the output is 1 only when $P=1$, $Q=0$ or $P=0$, $Q=1$, you'd get the DNF expression $(P \wedge \sim Q) \vee (\sim P \wedge Q)$ —which, hey, that's XOR! And now you have a direct recipe for wiring up the gates: two AND gates (one for each term), some NOT gates for the negations, and one OR gate to combine them.

This is why DNF matters—it gives you a completely mechanical way to turn *any* truth table into a working circuit. And this is literally how hardware design works at the lowest level. Computer chips are just enormous networks of logic gates, millions and millions of them wired together. The boolean algebra we learned in Module 2 *is* the mathematics of chip design. When hardware engineers want to make a circuit smaller or faster, they're essentially doing the same kind of simplification we did with De Morgan's laws and distributivity—reducing the number of gates needed to compute the same function.

Notice how this connects back to functional completeness from the previous module. The DNF construction is actually a *proof* that AND, OR, and NOT are functionally complete: given any truth table whatsoever, DNF hands you a circuit built entirely from those three gates. And since we showed in that module that NAND alone can simulate AND, OR, and NOT, it follows that NAND alone is functionally complete—which is exactly why real hardware can be (and often is) built entirely from NAND gates.

Exercise 3. Given the truth table where the output is 1 only for inputs $(P=1, Q=0, R=1)$ and $(P=0, Q=1, R=1)$, write the DNF expression and describe the circuit.

Exercise 4. Find the error in the following “proof” that $p \rightarrow q$ is logically equivalent to $q \rightarrow p$: “Consider any assignment. If p and q are both true, then both $p \rightarrow q$ and $q \rightarrow p$ are true. If both are false, then both implications are true (since $F \rightarrow F = T$). So they always have the same truth value, and are logically equivalent.”

What case does this argument miss?

3.7 Addendum: differences in terminology and additional derivable proof rules for homework

3.7.1 Derivable rules of inference

There are a number of extra rules you can derive that are useful for proofs, some of them are so useful they get extra special names:

Note: double negation elimination ($\neg\neg q$ shows q) was already listed as the elimination rule for negation in Section 3.3 above.

modus tollens:

(a) $p \rightarrow q$
 (b) $\sim q$
 shows $\sim p$

We derived this as a tautology in Section 2. It can also be derived from implication elimination and negation introduction.

disjunctive syllogism (there's two versions of it)

(a) $p \vee q$
 (b) $\sim q$
 shows p

(a) $p \vee q$
(b) $\sim p$
shows q

3.7.2 Differences in verbiage

The webwork problems might refer to “hypothetical syllogism”; this is just a different way of saying “transitive property of implication” which we derived above

In other words,

(a) $p \rightarrow q$
(b) $q \rightarrow r$
shows $p \rightarrow r$

3.7.3 Bitwise logical operators

There is a question in the webwork assignment that refers to bitwise logical operators like you may have seen in C or C++. However! Not everyone’s classes actually **covered** bitwise operators, so it would be a mistake on our part to not cover them here: if you see something like

$0101 \wedge 1100$

The way you calculate this is to go bit by bit and do the operation, gluing the pieces together at the end. So in this case we have a bitwise and, so calculate this as doing the $\wedge$ operator to each pair of bits—treating 0 as false and 1 as true.

$0 \wedge 1 = 0$
 $1 \wedge 1 = 1$
 $0 \wedge 0 = 0$
 $1 \wedge 0 = 0$

0100

Similarly

$0101 \vee 1100$

becomes

$0 \vee 1 = 1$
 $1 \vee 1 = 1$
 $0 \vee 0 = 0$
 $1 \vee 0 = 1$

1101

Intermezzo: Inference Rules as Trees

3.8 A better notation

In Module 3 we wrote our inference rules in a semi-formal style that looked like little programs—premises on numbered lines, a conclusion at the bottom. That format is fine for getting started, and it has the nice property that it looks like code. But there’s a standard notation used in logic textbooks, type theory, and programming language theory that’s compact, composable, and worth learning: *Gentzen-style* inference rules, sometimes called “natural deduction” style after the proof system Gerhard Gentzen introduced in the 1930s.⁴

The idea is simple. You draw a horizontal line. Above the line go the *premises*—the things you need to know. Below the line goes the *conclusion*—the thing you get to conclude. Off to the right of the line you write the name of the rule. That’s it.

3.9 The rules, redux

Let’s revisit the rules from Module 3, but now in this new notation. We’ll use L^AT_EX’s `\dfrac` command, so these are literally just fractions where the numerator is what you know and the denominator is what you conclude.

3.9.1 Conjunction

Introduction: If you know p and you know q , you can conclude $p \wedge q$:

$$\frac{p \quad q}{p \wedge q} \wedge \text{I}$$

This is the same rule as the `lstlisting` version from Module 3—premises (a) p and (b) q , conclusion $p \wedge q$ —just written more compactly.

Elimination: If you know $p \wedge q$, you can pull out either piece:

$$\frac{p \wedge q}{p} \wedge \text{E}_1 \qquad \frac{p \wedge q}{q} \wedge \text{E}_2$$

3.9.2 Disjunction

Introduction: If you know one disjunct, you can conclude the disjunction:

$$\frac{p}{p \vee q} \vee \text{I}_1 \qquad \frac{q}{p \vee q} \vee \text{I}_2$$

⁴Gentzen was a remarkable logician who also invented the sequent calculus, proved the consistency of Peano arithmetic, and basically invented modern proof theory. All before the age of 30.

Notice that you get to “make up” the other disjunct—if you know p , you can conclude $p \vee q$ for *any* q . Remember from Module 3 that $p \rightarrow (p \vee q)$ really is a tautology.

Elimination (proof by cases):

$$\frac{p \vee q \quad p \rightarrow r \quad q \rightarrow r}{r} \vee E$$

This is the rule we worked through with the eight-row truth table. If you know $p \vee q$ and you can get to r from either side, you’ve got r .

3.9.3 Implication

Elimination (modus ponens):

$$\frac{p \quad p \rightarrow q}{q} \rightarrow E$$

Our old friend. If you know p and you know $p \rightarrow q$, you get q .

Modus tollens:

$$\frac{p \rightarrow q \quad \neg q}{\neg p} MT$$

If you know $p \rightarrow q$ and you know $\neg q$, you can conclude $\neg p$. This is the “backdoor” version of modus ponens: instead of affirming the antecedent, you deny the consequent. Unlike the contrapositive *technique*, modus tollens as a single inference step is constructively valid.

Introduction: This is the rule that was “a little weird” in Module 3, and the tree notation actually makes it *clearer*. The idea is: if you can derive q while *temporarily assuming* p , then you can discharge that assumption and conclude $p \rightarrow q$. We write the discharged assumption in square brackets:

$$\frac{\begin{array}{c} [p] \\ \vdots \\ q \end{array}}{p \rightarrow q} \rightarrow I$$

The $[p]$ means “ p was assumed for the sake of this derivation, and now we’re done with it.” The dots indicate that there’s a subderivation—possibly several steps—that gets from p to q . Once you apply $\rightarrow I$, the assumption p is *discharged*: it’s no longer something you’re assuming, because you’ve packaged it up inside the implication.

This is exactly the same as the two-layer structure from Module 3 where we had “has a proof that assuming p , shows q ”—the bracket notation is just the standard way of writing that in tree form.

3.9.4 Negation

Introduction: If assuming p lets you derive a contradiction ($\perp$), then you can discharge that assumption and conclude $\neg p$:

$$\frac{\begin{array}{c} [p] \\ \vdots \\ \perp \end{array}}{\neg p} \neg I$$

This works exactly like $\rightarrow I$: the brackets around p mean that p was a temporary assumption, and it gets discharged when you apply the rule. In fact, in many formulations of logic $\neg p$ is simply *defined* as $p \rightarrow \perp$, so $\neg I$ is literally a special case of $\rightarrow I$.

Elimination (double negation):

$$\frac{\neg\neg p}{p} \neg\text{E}$$

If you know $\neg\neg p$, you can conclude p . Note that this rule requires the law of excluded middle—it is not valid in constructive logic. In a proof assistant like Agda or Coq, you would need to explicitly invoke a classical axiom to use it. The direction $p \rightarrow \neg\neg p$ is constructively valid, but eliminating the double negation is the part that requires classical reasoning (see the discussion in Module 3).

3.9.5 Constants

For completeness:

$$\frac{}{T} \text{TI} \qquad \frac{}{\perp} \perp\text{E}$$

T can always be concluded (no premises needed—note the empty space above the line). We write $\perp$ (pronounced “bottom” or “absurdity”) for the constant false—this is the same as F from Module 2, just in standard natural deduction notation. From $\perp$ you can conclude literally anything, which is the “ex falso” principle.

3.10 Composing rules into proof trees

Here’s where this notation really shines. Because the conclusion of one rule is just a formula, you can plug it in as the premise of another rule. The result is a *tree* that grows upward from the final conclusion at the root.

Let’s redo the transitivity-of-implication proof from Module 3. We want to show:

Given $p \rightarrow q$ and $q \rightarrow r$, prove $p \rightarrow r$.

Here’s the tree:

$$\frac{\frac{\frac{[p] \quad p \rightarrow q}{q} \rightarrow\text{E} \quad q \rightarrow r}{r} \rightarrow\text{E}}{p \rightarrow r} \rightarrow\text{I}$$

Read it from the bottom up: we want $p \rightarrow r$, so we use $\rightarrow\text{I}$, which means we need to derive r from the assumption $[p]$. To get r we use modus ponens with $q \rightarrow r$, but we need q . To get q we use modus ponens with $[p]$ and $p \rightarrow q$. Done.

The assumptions $p \rightarrow q$ and $q \rightarrow r$ are *undischarged*—they sit at the leaves of the tree as open assumptions. The assumption $[p]$ is discharged by the $\rightarrow\text{I}$ at the root.

Compare this to the `lstlisting` version:

```
(a) p -> q, by assumption
(b) q -> r, by assumption
(c) proof that assuming p, shows r
    subproof: (i) p, by assumption
                (j) q, by modus ponens with (i) and (a)
                show r, by modus ponens with (j) and (b)
shows p -> r by introduction of implication
```

Same proof, same logic — just a different way to lay it out on the page. The tree version makes the structure—which things depend on which—immediately visible.

3.11 Reading proof trees

You can read these trees in two directions, and both are useful.

Bottom-up (goal-directed): start at the conclusion and ask “what rule could produce this?” That tells you what you need to prove next, and you work your way up to the leaves. This is the way you typically *construct* a proof—you start with what you want and work backwards until everything is justified.

Top-down (forward reasoning): start at the leaves (your assumptions) and follow the rules downward, seeing what you can derive. This is the way you typically *verify* a proof—you check that each step follows from the ones above it.

This dual reading mirrors a distinction you’ll keep encountering: the difference between “working backwards from what you want to prove” and “working forwards from what you know.” If you’ve ever debugged a program by starting from the error message and tracing back through the code, you were doing goal-directed reasoning. If you’ve ever traced through a program with a debugger, stepping forward line by line, you were doing forward reasoning. Both are useful; neither is always better.

3.12 Why this matters

If this is your only logic or discrete math course, you can get by just fine with the `lstlisting` format from Module 3. But if you go on to take a course in compilers, programming language theory, formal verification, or type theory—and as CS majors, there’s a decent chance you will—you’re going to see Gentzen-style rules *everywhere*. The typing rules for a programming language are written this way. The operational semantics of a language are written this way. The rules of a type checker, a proof assistant, a static analyzer—all trees, all the time. It’s also the notation used in most logic textbooks beyond the intro level, so if you ever crack open a graduate-level logic text, you’ll be glad you’ve seen it before.

Think of this intermezzo as a Rosetta Stone: you now know two ways to write the same rules and can translate between them.

Chapter 4

First-Order Predicate Logic

4.1 A logic with real types

Now we need to start dealing with a logic that has actual types beyond just booleans: first-order predicate logic! This is going to look *a lot* like propositional logic—it still has our old friends: $\rightarrow$, $\wedge$, $\vee$, $\sim$, T , and F —but now it’s also going to have $\forall$ and $\exists$ as well as a notion of predicates.

We need these things in order to add *sets* (or in other words *types*) to our logic. Unlike booleans, when it comes to sets we need to be able to quantify our propositions about elements.

- are we making a logical claim about a particular element? (i.e. that zero is the identity of addition)
- are we making a logical claim about all elements in A ? ($\forall x : A. \dots$)
- are we making a logical claim that we know that there is an element with a certain property, although we’re not naming a particular element? ($\exists x : A. \dots$)

We also need to be able to make statements about these types, which is where our predicates come in. With the tools we’ve seen, we don’t have the ability to define predicates *inside* the logic. Instead we need to add the predicates *to* the logic and define them externally as functions: a predicate on the type/set A is really a function $A \rightarrow \text{Bool}$ where Bool is the two-point set $\{\text{True}, \text{False}\}$.

A really special predicate is the equality predicate on each set, which is given by the diagonal relation where $a=a$ is true and $a=b$ is false for any b that isn’t a .

4.2 Forall and exists

We’ve seen, in informal mathematics, the meaning of $\forall$ and $\exists$ a few times. As we discussed above, these are used to *quantify* what elements of a set/type¹ a proposition is covering, which is why they’re referred to as “quantifiers”.

We need to describe what the semantics of these quantifiers are. There are two answers in this logic. The first is that the meaning of a quantifier can be described in terms of the underlying sets and functions on sets.

¹I’m going to keep saying set/type because in some logics there is a notion of “types” that are more general than sets. For the purposes of this class, think of them as essentially the same thing.

When we write “ $\forall x : A. P(x)$ ” this is going to be true exactly when the function $P : A \rightarrow \text{Bool}$ sends every element of A to `True`. Equivalently, if you know the language of inverse images of functions, “ $\forall x : A. P(x)$ ” is true when $P^{-1}(\text{True}) = A$.

Similarly, when we write “ $\exists x : A. P(x)$ ” this is going to be true exactly when the function $P : A \rightarrow \text{Bool}$ sends at least one element of A to `True` or, again in the language of inverse images of functions when the set $P^{-1}(\text{True})$ is not empty.

The *other* way we can define the meaning, at least when the set is finite (so that we can literally write out the conjunction or disjunction), is that we can define $\forall$ as a kind of Big And and $\exists$ as a kind of Big Or, where

“ $\forall x : A. P(x)$ ” becomes

$P(x_1) \wedge P(x_2) \wedge \dots$

for every x in the set A . Similarly, “ $\exists x : A. P(x)$ ” becomes

$P(x_1) \vee P(x_2) \vee \dots$

for every x in A .

You can play with these transformations for some small finite sets to prove to yourself that these definitions match up exactly.²

4.2.1 Free and bound variables

When a quantifier introduces a variable, that variable is *bound*—it belongs to the quantifier the way a function parameter belongs to the function. In “ $\forall x : A. P(x)$ ” the variable x is bound by the $\forall$. A variable that isn’t captured by any quantifier is called *free*.

If you’re a programmer, you already know the distinction: bound variables are like loop variables or function parameters—they’re local to their scope and you can rename them without changing anything. Free variables are like globals. Just as `def f(x): return x + 1` and `def f(y): return y + 1` define the same function, the formulas $\forall x. P(x)$ and $\forall y. P(y)$ say exactly the same thing. The bound variable is a placeholder; its name doesn’t matter.

Consider a formula with both kinds: $\forall x. P(x, y)$. Here x is bound (it’s under the scope of $\forall x$) and y is free. This formula is *not* a complete statement—its truth depends on what y is. Think of it as a function with one argument still unresolved: the $\forall$ “closed over” x but y is still dangling. Only once you fix a value for y (or bind it with another quantifier) do you get something with a definite truth value.

Why does this matter? When we get to the proof rules later in this module, the $\forall$ -elimination rule *substitutes* a concrete term for the bound variable, and the $\forall$ -introduction rule requires that the variable you’re generalizing over is truly arbitrary—meaning it isn’t free anywhere it shouldn’t be. Keeping track of which variables are free and which are bound is what makes these rules safe.

Exercise 1. In the formula $\exists x. \forall y. P(x, y, z)$, identify which variables are free and which are bound. Explain how you determined the status of each variable.

4.3 Truth sets

This is a quick concept, but it comes up often enough to deserve a name.

Definition 4.3.1 (Truth set). Given a predicate P on a set A , the *truth set* of P is the set

$$\{x \in A \mid P(x) \text{ is true}\}$$

²There’s a secret third way to define $\forall$ and $\exists$ in dependent type theory, a logical framework where $\forall$ becomes a kind of function definition and $\exists$ becomes a kind of pair, like a pair in Python.

i.e. the set of all elements that make the predicate true.

If you were paying attention in the last section you'll notice that this is just the inverse image $P^{-1}(\text{True})$ that we already talked about, but now it has a name. Think of it like a type alias in Python or Haskell—same thing, different label.³

Let's do a couple of worked examples.

Example 1. Let $P(d) = "6/d \text{ is an integer}"$ and let our set be $\mathbb{Z}^+$ (the positive integers). So we're asking: which positive integers divide 6 evenly? The truth set is $\{1, 2, 3, 6\}$ because those are exactly the positive divisors of 6.

Example 2. Let $P(x) = "1 \leq x^2 \leq 4"$ and let our set be $\mathbb{Z}$ (all integers). We need $x^2 \geq 1$, so $x \neq 0$, and $x^2 \leq 4$, so $x \in \{-2, -1, 0, 1, 2\}$. Combining these: the truth set is $\{-2, -1, 1, 2\}$.

Here's how this connects back to quantifiers:

- $\forall x : A. P(x)$ is true **if and only if** the truth set of P equals A . Every element makes the predicate true, so the truth set is the whole set.
- $\exists x : A. P(x)$ is true **if and only if** the truth set of P is nonempty. At least one element makes it true, so there's something in there.

So you can think of $\forall$ as asking "is the truth set everything?" and $\exists$ as asking "is the truth set nonempty?" which is a pretty clean way to think about it.

4.4 Negating quantified statements

This is *crucial* and it comes up constantly—in proofs, in writing specifications, in checking whether your code is correct, &c. You need to be able to negate statements that have quantifiers in them.

The good news is that the rules are really clean and they're basically just De Morgan's laws but for quantifiers instead of $\wedge$ and $\vee$:

$\neg(\forall x : A. P(x)) \equiv \exists x : A. \neg P(x)$ $\neg(\exists x : A. P(x)) \equiv \forall x : A. \neg P(x)$

Why do these make sense? Let's think about it. "It's *not* the case that everything satisfies P " means "there's something that *doesn't* satisfy P ." And "it's *not* the case that something satisfies P " means "*nothing* satisfies P ," which is the same as saying everything fails to satisfy P .

And hey, remember the Big And / Big Or interpretation from the last section? This is just De Morgan's laws from Module 2 in disguise! Negating a big conjunction (which is what $\forall$ is) gives you a big disjunction (which is what $\exists$ is), and vice versa. The same principle applies at a larger scale.⁴

Example 1. Negate "every computer has a CPU."

This is $\forall c : \text{Computers}. \text{HasCPU}(c)$. The negation is $\exists c : \text{Computers}. \neg \text{HasCPU}(c)$, which in English is: "there exists a computer that does not have a CPU."

Example 2. Negate "there exists a band that has won at least 10 Grammys."

This is $\exists b : \text{Bands}. \text{Grammys}(b) \geq 10$. The negation is $\forall b : \text{Bands}. \text{Grammys}(b) < 10$, which says: "for every band, that band has not won at least 10 Grammys"—or equivalently, "no band has won at least 10 Grammys."

³Think of it as applying a filter: in Python, `truth_set = {x for x in A if P(x)}`.

⁴If you squint at it in exactly the right way, the entirety of first-order logic is just propositional logic with extra steps. Which is kind of the point.

One more pattern that shows up all the time: negating an implication *inside* a quantifier. Recall from Module 2 that $\neg(P \rightarrow Q) \equiv P \wedge \neg Q$ —the negation of “if P then Q” is “P and not Q.” So when you negate a universally quantified implication you get:

$$\neg(\forall x. P(x) \rightarrow Q(x)) \equiv \exists x. P(x) \wedge \neg Q(x)$$

which reads as “there’s some x where $P(x)$ holds but $Q(x)$ doesn’t.” This is a really common pattern in counterexample-based reasoning: to disprove “all P’s are Q’s” you find a P that isn’t a Q.

Exercise 2. Negate the following statement and simplify: “For every student s in CS 250, there exists a homework h such that s completed h on time.” Write the negation in both symbols and plain English.

4.5 Nested quantifiers

Sometimes one quantifier just isn’t enough and you need to stack them. This is where things get interesting.

Example. Consider the statement “there exists a real number u such that for all real numbers v , $uv = v$.” In our notation this is:

$$\exists u : \mathbb{R}. \forall v : \mathbb{R}. uv = v$$

This is claiming that there’s a multiplicative identity. Is it true? Yes! Take $u = 1$ and you get $1 \cdot v = v$ for all v . So we’ve *witnessed* the existential by providing a concrete value.⁵

Here’s what you really need to internalize: **order matters**. The statements $\forall x. \exists y. P(x, y)$ and $\exists y. \forall x. P(x, y)$ are *very different*.

Let’s look at a concrete case. Consider $P(x, y) = “y > x”$ over $\mathbb{R}$:

- $\forall x : \mathbb{R}. \exists y : \mathbb{R}. y > x$ says “for every number there’s a bigger one.” This is **true**—just pick $y = x + 1$.
- $\exists y : \mathbb{R}. \forall x : \mathbb{R}. y > x$ says “there’s a number bigger than all numbers.” This is **false**—there’s no largest real number.⁶

If you’re a programmer, here’s a useful way to think about it. The statement $\forall x. \exists y. P(x, y)$ is like saying “for every input x , we can find an output y such that $P(x, y)$ holds.” In other words, a function exists that maps inputs to valid outputs—like writing `def f(x): return x + 1` in Python. The y you pick can *depend* on x .

But $\exists y. \forall x. P(x, y)$ is like saying “there’s a single y that works for ALL inputs.” You have to pick one constant value up front that satisfies every possible x . That’s way harder—like trying to write `y = ???` at the top of your program and having it work for every input. In C terms, it’s the difference between a function and a global constant.

Negating nested quantifiers is actually straightforward: you just flip each quantifier as you push the negation through, and then negate the predicate at the end. So:

$$\neg(\exists u : \mathbb{R}. \forall v : \mathbb{R}. uv = v) \equiv \forall u : \mathbb{R}. \exists v : \mathbb{R}. uv \neq v$$

⁵This is exactly what the $\exists$ -introduction rule does: you hand it a concrete element and a proof that the predicate holds on that element.

⁶If you think ∞ is a real number, we need to have a talk.

Each $\exists$ becomes a $\forall$, each $\forall$ becomes an $\exists$, and the predicate gets negated. That’s it. You just walk through the quantifiers one at a time applying the rules from the previous section.

Exercise 3. Let $P(x, y)$ mean “ $x + y = 0$ ” over $\mathbb{Z}$. Determine whether each of the following is true or false, and justify your answer:

- (a) $\forall x. \exists y. P(x, y)$
- (b) $\exists y. \forall x. P(x, y)$

4.6 Necessary and sufficient conditions

This section is about translating some very common mathematical and CS jargon back into the logic we already know. The phrases “necessary condition” and “sufficient condition” show up *everywhere* in textbooks, specifications, API docs, &c. and you need to know what they mean.

Definition 4.6.1 (Necessary and sufficient conditions).

- “ P is *sufficient* for Q ” means $P \rightarrow Q$. If P happens, that’s *enough* to guarantee Q . P is sufficient—it suffices.
- “ P is *necessary* for Q ” means $Q \rightarrow P$, or equivalently $\neg P \rightarrow \neg Q$. You *can’t* have Q without P . P is a prerequisite.
- “ P is *necessary and sufficient* for Q ” means $P \leftrightarrow Q$, that is, $(P \rightarrow Q) \wedge (Q \rightarrow P)$. They’re logically equivalent—each one implies the other.

A common point of confusion: “ P is necessary for Q ” is $Q \rightarrow P$, **not** $P \rightarrow Q$. The necessary thing goes on the *right* of the arrow when the thing that needs it is on the left. Think of it as: Q requires P , so if you have Q you must have P . This trips people up more than almost anything else in the course, so take a moment to make sure it sits right.

Example. “Being a square is sufficient for being a rectangle.” This means: if something is a square, then it’s a rectangle. Every square is a rectangle. That’s $\text{Square}(x) \rightarrow \text{Rectangle}(x)$.

Flipping it around: “Being a rectangle is necessary for being a square.” This means: if something isn’t a rectangle, it can’t be a square. You need rectangle-ness in order to have square-ness. That’s $\text{Square}(x) \rightarrow \text{Rectangle}(x)$ —the same implication, just described from the other direction.

The connection to everything else we’ve been doing is pretty direct: these are just different ways of talking about implication. The reason we bring this up now is that math texts and CS specs use this language *all the time* and if you don’t know how to translate “necessary” and “sufficient” into arrows, you’re going to have a bad time reading formal documentation.⁷

4.7 Proofs with this logic

Proofs in this logic are going to be really similar to proofs in propositional logic but there are going to be five additional rules: the intro and elimination rules of our new quantifiers plus a special rule for our $=$ operator.

We’ll start with the easiest, which is the rule for equality.

Reflexivity:

⁷Real example from a spec I once read: “Authentication is necessary but not sufficient for authorization.” Translation: $\text{Authorized} \rightarrow \text{Authenticated}$ but $\text{Authenticated} \not\rightarrow \text{Authorized}$. You need to be logged in, but being logged in doesn’t mean you have permission.


```

assumes x : A
-----
shows x = x

```

which is just that you’re always allowed to conclude that a thing is equal to itself, unsurprisingly! If you’re also playing the natural number game or if you ever try other interactive theorem provers you probably will see this rule called “reflexivity”, or “rfl”/“refl” for short.

Next we present the rules for $\forall$. We start with elimination because it is the simpler of the two.

Elimination:

```

assumes  $\forall x : A. P(x)$ 
       a : A
-----
shows P(a)

```

This says that if P is true of all elements of A and a is an element of A , then $P(a)$ holds. It’s a lot like applying a function, isn’t it?

Introduction: The introduction rule is going to look a little like the introduction rule for implication.

```

assumes
  proof of:
    assumes a : A
    -----
    shows P(a)
  -----
shows  $\forall x : A. P(x)$ 

```

Side condition: the variable a must really be *arbitrary*. In practice that means a should be fresh for the subproof and should not appear free in any undischarged assumption that the subproof depends on. Otherwise you’d be proving $P(a)$ for some *special* a and then incorrectly generalizing to all elements of A .

What we’re doing is taking evidence that given an *arbitrary* element of A we can always show that $P(a)$ is true and internalizing that into the $\forall$, the same way that we internalize the ability to prove Q assuming P into $P \rightarrow Q$.

The rules for $\exists$ are the mirror image of $\forall$ ’s.

Elimination:

```

assumes
   $\exists x : A. P(x)$ 
  subproof:
    assumes c : A
    P(c)
    -----
    shows Q
  -----
shows Q

```

Side condition: the witness variable c must be *fresh*. In particular, it must not appear free in Q or in any undischarged assumption outside the subproof. Otherwise the particular name chosen for the witness could leak out into the conclusion, which would make the rule unsound.

This rule says: if you know something satisfying P exists, and you can show that Q follows from having *any* element c satisfying $P(c)$ —without relying on which specific c it is—then you can

conclude Q . Think of it as opening a box: you know something is inside, you don't know exactly what, but you show that no matter what it turns out to be, the conclusion holds. This is why the subproof introduces a *fresh* variable c —it represents an arbitrary witness, not a specific one.

Introduction:

```

assumes
  c : A
  P(c)
-----
shows  $\exists x : A. P(x)$ 

```

4.8 Induction, round 1

Here we're going to introduce the concept of the natural numbers and also the idea of natural induction. Why? We've shown how to prove things about predicates in general, but if I were to just *hand* you a predicate on a set it's not like you'd know how to actually do anything with it. You don't have any handles, so to speak, for how to start your proof other than brute forcing the set-based semantics. But as we've discussed a couple of times now this won't work for infinite sets where you'd need to check the formula against potentially infinite values.

Induction is a way of getting a “handle” on the data inside a particular set/type by defining types in a way that has structure to it.

Remember earlier how we talked about BNF syntax? The syntax where we recursively defined a set of things?

Like how we defined all propositional logic formulas as

$L := L \wedge L \mid L \vee L \mid L \rightarrow L \mid \sim L \mid T \mid F \mid x$

We can define the natural numbers the same way:

$\text{Nat} := 0 \mid \text{succ Nat}$

Every natural number is either 0 or the successor of another natural number, so checking a predicate on infinitely many values reduces to just *two cases*.

That's what induction buys us.

Now if you're playing the natural number game you'll have already seen what the induction rule based on this definition is but let's spell it out:

natural induction

```

assumes P(0)
  subproof:
    assumes P(n)
    -----
    shows P(succ n)
-----
shows  $\forall n : \text{Nat}. P(n)$ 

```

What this is saying is that if you can show that P is true on 0 and if you can show that with evidence that $P(n)$ is true you can prove $P(\text{succ } n)$, then you can prove that P is true on all natural numbers.

Note how the base case is the same as the base case of the recursion and that the second part follows the recursive structure of the BNF syntax.

4.9 Addendum on notation

Sometimes you might see the “: A” part left out of quantifiers, like

“ $\forall x. P(x)$ ”

instead. This is valid syntax *if* we’ve defined in advance what our “domain of discourse” is. Then and only then can we leave out naming what set we’re talking about because there’s only *one* set.

Intermezzo: More on Natural Induction

To illustrate natural induction, let's work through some proofs similar to those in The Natural Number Game, but informally.

For the purposes of these proofs we'll assume that the definition of addition by recursion on the second argument is

$$\begin{aligned}n + 0 &= n \\ n + (\text{succ } m) &= \text{succ } (n + m)\end{aligned}$$

4.10 Proof that $0 + m = m$

We have automatically from the definition of addition by recursion that $n + 0 = n$, but what about the other way? You might wonder why this needs a proof at all—isn't it obvious? The subtlety is that our definition of addition recurses on the *second* argument. The equation $n + 0 = n$ is the base case of that recursion, so it holds by definition. But $0 + m = m$ puts zero in the *first* argument, which is not the recursive position. The definition doesn't directly tell us what happens there, so we need induction.

Theorem: $\forall m. 0 + m = m$

Proof by induction on m .

Case $m = 0$: Need to prove that $0 + 0 = 0$, but this follows immediately by the definition of addition.

Case $m = \text{succ } m'$:

Assuming $0 + m' = m'$.

Shows $0 + \text{succ } m' = \text{succ } m'$.

By the definition of addition,

$$0 + \text{succ } m' = \text{succ } (0 + m')$$

but

$$\text{succ } (0 + m') = \text{succ } m' \qquad \text{by the induction hypothesis that } 0 + m' = m'$$

QED

4.11 Proof that $\text{succ } n + m = \text{succ } (n + m)$

Theorem: $\forall n m. \text{succ } n + m = \text{succ } (n + m)$

Proof by induction on m .

Case $m = 0$: $\text{succ } n + 0 = \text{succ } (n + 0)$.

Immediate from the definition of addition.

Case $m = \text{succ } m'$:

Assuming: $(\text{succ } n) + m' = \text{succ } (n + m')$.

Shows: $(\text{succ } n) + (\text{succ } m') = \text{succ } (n + (\text{succ } m'))$.

$$\begin{aligned} (\text{succ } n) + (\text{succ } m') &= \text{succ } (\text{succ } n + m') && \text{by the definition of addition} \\ &= \text{succ } (\text{succ } (n + m')) && \text{by the induction hypothesis} \end{aligned}$$

but the right hand side tells us

$$\text{succ } (n + (\text{succ } m')) = \text{succ } (\text{succ } (n + m')) \quad \text{by the definition of addition}$$

so we end up with

$$\text{succ } (\text{succ } (n + m')) = \text{succ } (\text{succ } (n + m'))$$

which is true by reflexivity.

QED

4.12 Proof that addition is associative

Theorem: $\forall n m r. n + (m + r) = (n + m) + r$

Proof by induction on n .

Case $n = 0$: Need to prove that $\forall m r. 0 + (m + r) = (0 + m) + r$.

Both sides reduce to $m + r$ by the left and right identity properties of 0 (proved in Section 1 and given by definition, respectively):

$$m + r = m + r$$

which is true by reflexivity.

Case $n = \text{succ } n'$: Need to prove that **assuming** $\forall m r. n' + (m + r) = (n' + m) + r$ we can derive $\text{succ } n' + (m + r) = (\text{succ } n' + m) + r$.

$$\text{succ } n' + (m + r) = \text{succ } (n' + (m + r)) \quad \text{by our theorem above}$$

and the right hand side similarly transforms

$$\begin{aligned} (\text{succ } n' + m) + r &= \text{succ } (n' + m) + r && \text{by our theorem above} \\ &= \text{succ } ((n' + m) + r) && \text{by the same theorem} \\ &= \text{succ } (n' + (m + r)) && \text{by the induction hypothesis} \end{aligned}$$

so we have

$$\text{succ } (n' + (m + r)) = \text{succ } (n' + (m + r))$$

which is true by reflexivity.

QED

4.13 Proof that addition is commutative

Theorem: $\forall n m. n + m = m + n$

Proof by induction on n .

Case $n = 0$: Need to prove that $\forall m. 0 + m = m + 0$.

This is immediately solved by the fact that 0 is the left and right identity for addition, so this simplifies to $m = m$ which is true by the definition of $=$ (i.e. by reflexivity).

Case $n = \text{succ } n'$: Need to prove that **assuming** $\forall m. n' + m = m + n'$ we can show $\forall m. (\text{succ } n') + m = m + (\text{succ } n')$.

So by the definition of addition we have

$$m + (\text{succ } n') = \text{succ } (m + n')$$

and by one of the above theorems we have that

$$(\text{succ } n') + m = \text{succ } (n' + m)$$

but

$$\text{succ } (n' + m) = \text{succ } (m + n') \quad \text{by the induction hypothesis}$$

and thus

$$\text{succ } (m + n') = \text{succ } (m + n')$$

which is true by reflexivity.

QED

Intermezzo: Quantifiers as Games

In Module 4 we defined quantifiers two ways: as functions into `Bool`, and as Big And / Big Or over a set. Both of those descriptions are perfectly correct, but they’re also a bit static—they tell you what quantifiers *mean* but not what it *feels like* to work with them. There’s a third perspective that is more dynamic, more intuitive, and connects beautifully to both programming and to modern cryptography. It’s the idea that logic is a *game*.

4.14 The game interpretation

Imagine two players sitting across from each other. One is the **Prover** (that’s you—the person trying to show the statement is true). The other is the **Challenger** (think of them as Nature, or an Opponent, or a very skeptical friend). The logical formula being evaluated determines the rules of the game. Prover wins if, at the end of the game, the resulting atomic statement is true. Challenger wins if it’s false.

Here’s how each connective translates into a game move:

- $\exists x. P(x)$: **Prover’s move.** Prover picks a witness x , and then the game continues with $P(x)$. Prover wins if $P(x)$ turns out to be true for the chosen x . This is the “I can find one” quantifier—Prover gets to choose.
- $\forall x. P(x)$: **Challenger’s move.** Challenger picks *any* x they want—the nastiest, most adversarial x they can think of—and then the game continues with $P(x)$. Prover wins if $P(x)$ is true *regardless* of what Challenger picked. This is the “it works for everything” quantifier—Challenger gets to choose, and Prover has to survive.
- $A \wedge B$: **Challenger’s move.** Challenger picks which of A or B to verify. Prover must be able to win *both* games, so it doesn’t matter which one Challenger chooses. Think of it as: “I claim both—go ahead, quiz me on either one.”
- $A \vee B$: **Prover’s move.** Prover picks which of A or B to demonstrate. Only one needs to hold, and Prover gets to pick the one they can actually win. Think of it as: “I claim at least one of these—let me show you which.”
- $A \rightarrow B$: **Challenger provides, Prover responds.** Challenger hands over evidence for A , and Prover must use it to produce evidence for B . If you’re a programmer, this should look familiar: it’s a function. Challenger provides the input, Prover computes the output.
- $\neg A$: **Role swap.** The players switch seats. Prover becomes Challenger, Challenger becomes Prover, and they play the game for A with reversed roles. Negation flips who’s trying to do what.

This might seem like just a cute metaphor, but it’s not—it’s a genuine mathematical framework called *game semantics*⁸ and it gives us the exact same truth values as the set-based semantics from Module 4. A formula is true precisely when Prover has a *winning strategy*—a way to play that wins no matter what Challenger does.

4.15 Nested quantifiers as alternating turns

The game interpretation really shines when we look at nested quantifiers, because nested quantifiers correspond to a game with *multiple rounds*—the players alternate turns, and the whole formula describes a back-and-forth.

Let’s revisit the example from Module 4 where we saw that quantifier order matters. Let $P(x, y) = “y > x”$ over $\mathbb{R}$.

Game 1: $\forall x : \mathbb{R}. \exists y : \mathbb{R}. y > x$

1. Challenger picks any real number x . (Challenger’s move—this is $\forall$.)
2. Prover sees what x was chosen and picks y in response. (Prover’s move—this is $\exists$.)
3. Prover wins if $y > x$.

Can Prover always win? Yes! Prover’s strategy is simple: whatever x Challenger picks, respond with $y = x + 1$. Since $x + 1 > x$ for every real number, this strategy works against every possible challenge. The formula is **true**.

The key thing to notice: Prover’s choice of y gets to *depend on* x . Prover moves second, so they have information. Their winning strategy is a *function* from challenges to responses: $f(x) = x + 1$.

Game 2: $\exists y : \mathbb{R}. \forall x : \mathbb{R}. y > x$

1. Prover picks a real number y . (Prover’s move—this is $\exists$.)
2. Challenger sees what y was chosen and picks x in response. (Challenger’s move—this is $\forall$.)
3. Prover wins if $y > x$.

Can Prover win? No. Whatever y Prover commits to, Challenger can just pick $x = y + 1$, and then $y > y + 1$ is false. Prover has to commit *before* seeing the challenge, and no single constant y can be bigger than every real number. The formula is **false**.

See the difference? In Game 1, Prover has a *strategy*—a function that responds to each challenge. In Game 2, Prover needs a *constant*—a single value that works universally. Strategies are much more powerful than constants, which is why $\forall x. \exists y$ is a weaker claim than $\exists y. \forall x$ (the second implies the first, but not vice versa).

If you’re a programmer, the distinction is really clean:

- $\forall x. \exists y. y > x$ is like writing a function:

```
def respond(x):
    return x + 1 # always bigger than the input
```

⁸Game semantics was developed seriously in the 1990s by Abramsky, Jagadeesan, Malacaria, Hyland, Ong, and others. It provides a fully compositional semantics for programming languages and logics—not just a teaching analogy.

You get to see the input before choosing your output. Easy.

- $\exists y. \forall x. y > x$ is like declaring a global constant:

$y = ???$ # must be bigger than ALL possible inputs

You have to commit before seeing any input. Impossible (for the reals).

This is the same intuition we built in Module 4, but the game framing makes the “why” visceral: it’s about *who moves first*. The order of quantifiers determines the order of play, and the order of play determines how much information each player has when they make their choice.

4.16 Games and constructive logic

There’s a philosophical point lurking here that connects back to something we touched on in Module 5: the distinction between constructive and non-constructive proofs.

The game interpretation is inherently *constructive*. To win a game, Prover can’t just argue abstractly that a winning move exists somewhere—Prover has to actually *make the move*. When the formula says $\exists x. P(x)$, Prover has to put a concrete x on the table. When it says $A \vee B$, Prover has to point to one of A or B and win that game. No hand-waving allowed.

This is exactly the spirit of constructive mathematics: to prove something exists, you must build it. To prove a disjunction, you must know which side is true.

Classical logic, by contrast, gives Prover an extra power: *strategy-stealing arguments*. In classical logic, Prover is allowed to reason like this: “Suppose I can’t win. Then Challenger must have a winning strategy. But if Challenger has a winning strategy, then [some contradiction arises], so actually I *can* win.” The conclusion is that Prover wins, but the argument never actually described a move for Prover to make. It proved existence without providing a witness.

This connects directly to the law of excluded middle, $p \vee \neg p$. In game terms, this says: “Either Prover can demonstrate p , or Prover can demonstrate $\neg p$.” Classically, this is just assumed—one of the two must be the case. But constructively, Prover has to actually play one of the two games and win it. For many propositions (especially undecidable ones from computability theory) there’s no general way to know which side to pick, so $p \vee \neg p$ isn’t something you get for free.

If you remember the discussion of constructive existence proofs from Module 5—where we said that a constructive proof gives you an algorithm while a non-constructive proof gives you a theorem but no code—this is the same idea wearing different clothes. Game semantics makes it vivid: constructive logic is the version of the game where Prover actually has to play. Classical logic is the version where Prover is sometimes allowed to win by saying “if I couldn’t win, that would be absurd.”⁹

4.17 Why this matters

You might be wondering: is this just a nice way to think about quantifiers, or does it actually *go* somewhere? It goes a lot of places.

Interactive proof systems. In theoretical computer science and cryptography, the Prover-vs-Verifier model is the literal foundation of *interactive proof systems*. A Prover tries to convince

⁹If that sounds like cheating, welcome to the intuitionist club. If it sounds perfectly reasonable, welcome to the classical club. Both clubs have cookies; they just disagree about whether the cookies have been constructed or merely shown to exist.

a Verifier that a statement is true, and the Verifier is allowed to ask questions (challenges). The statement is “provable” if there’s a Prover strategy that convinces the Verifier, and “sound” if no cheating Prover can fool the Verifier into accepting a false statement. This is essentially the same two-player game we’ve been discussing, just taken very seriously.

Zero-knowledge proofs. An especially remarkable variant: what if Prover wants to convince Verifier that a statement is true *without revealing any information about why it’s true*? This is a zero-knowledge proof, and it’s the cryptographic technology behind privacy-preserving blockchain protocols, anonymous credentials, and a growing chunk of modern cryptography. The game-theoretic structure is essential—the “zero knowledge” property is defined in terms of what Verifier can learn from the interaction.

Complexity theory. One of the landmark results in theoretical computer science is that $IP = PSPACE$: the class of problems solvable by an interactive proof system (where a polynomial-time Verifier interacts with an all-powerful Prover) is exactly the class of problems solvable with polynomial space. This was a shocking result when it was proved in 1992, and the entire framework rests on formalizing computation as a game between Prover and Verifier.

The point is that the two-player game isn’t just a pedagogical metaphor. It’s a lens through which serious mathematics, computer science, and cryptography are actually done. When you see quantifiers as games, you’re not just learning a trick for reading formulas—you’re learning the conceptual vocabulary of a significant part of modern theory.

Exercises

Exercise 1. Consider the statement

$$\forall \epsilon > 0. \exists \delta > 0. \forall x. (|x - 3| < \delta \rightarrow |x^2 - 9| < \epsilon).$$

This is the epsilon-delta definition of continuity for $f(x) = x^2$ at $x = 3$.

Describe the game: who moves first? What does each player choose at each turn? What must Prover achieve in order to win? (You do not need to find Prover’s actual strategy—just describe the structure of the game.)

Exercise 2. In the game for $\exists x. \forall y. (x + y = y)$, what is Prover’s winning move? (Hint: Prover moves first. What value of x makes the inner statement true no matter what Challenger picks for y ?)

Chapter 5

Informal Proofs and Groups

This module is, in a sense, a breather. We’re not introducing new concepts here — instead we’re getting practice with informal proofs after several modules of formal logic.

The relevant sections of Epp give us a lot of time to practice proofs with numbers. They’re very well written and even call out some topics I think are important: a games-based approach to quantifiers, constructive vs. non-constructive proofs of $\exists$, and the relationship between writing proofs and writing computer programs (framing the connection as analogy rather than mathematical identity).

What I think might be useful as an addition to the text, rather than attempting to recapitulate its content, is to give lots of examples of proofs in a different context than numbers.

But first, let’s make sure we have the number-theory toolkit down solid, since that’s what Epp 4.1–4.4 is all about.

One thing worth saying up front: the informal proof techniques we’re about to practice are not a departure from the formal rules we built up in Modules 3 and 4—they’re implementations of them. A direct proof of “for all x , if $P(x)$ then $Q(x)$ ” is really $\forall$ -intro combined with $\rightarrow$ -intro: introduce a fresh variable, assume the hypothesis, derive the conclusion. A counterexample to a universal claim is $\exists$ -intro applied to the negation $\neg(\forall x. P(x))$. A proof by cases is $\vee$ -elimination in disguise. The formal rules tell us *why* these techniques are valid; the informal style is how mathematicians actually write. We’ll keep pointing out these connections as we go, so you can see the formal machinery underneath the prose.

5.1 Direct proof

The most basic proof technique is the *direct proof*. The shape of a direct proof for a universal conditional statement — something of the form “for all x , if $P(x)$ then $Q(x)$ ” — is: you assume $P(x)$ for an *arbitrary* x , and then you show that $Q(x)$ follows. That’s it. The whole game is figuring out how to get from $P(x)$ to $Q(x)$, and the trick is almost always: *unpack the definitions*.

Even and odd

Let’s nail down what “even” and “odd” actually mean, precisely enough to use in proofs.

Definition 5.1.1 (Even and odd). An integer n is *even* if $n = 2k$ for some integer k . An integer n is *odd* if $n = 2k + 1$ for some integer k .

These definitions are what we actually *use* in proofs — when we know something is even, we unpack the definition to get the k , and then we do algebra with $2k$. When we need to *show* something

is even, we exhibit a k and show the equation holds. This is the “there exists” hidden inside these definitions: “ n is even” really means “ $\exists k \in \mathbb{Z}. n = 2k$.”

Let’s do a worked example.

Claim: If a and b are both odd integers, then $a + b$ is even.

Proof: Let a and b be arbitrary odd integers. By the definition of odd, we have $a = 2j + 1$ for some integer j , and $b = 2k + 1$ for some integer k . Then

$$a + b = (2j + 1) + (2k + 1) = 2j + 2k + 2 = 2(j + k + 1).$$

Since $j + k + 1$ is an integer, $a + b = 2m$ where $m = j + k + 1$, so $a + b$ is even by definition. $\square$

See the pattern? We assumed the hypothesis (both odd), unpacked the definitions (got j and k), did some algebra, and then repacked the result into the definition of even. See the pattern? Assume the hypothesis, unpack the definitions, do the algebra, repack the result. That’s the core of direct proof.

Let’s do another one.

Claim: The sum of any even integer and any odd integer is odd.

Proof: Let a be an arbitrary even integer and b be an arbitrary odd integer. Then $a = 2j$ for some integer j and $b = 2k + 1$ for some integer k . So

$$a + b = 2j + 2k + 1 = 2(j + k) + 1.$$

Since $j + k$ is an integer, $a + b$ has the form $2m + 1$ where $m = j + k$, so $a + b$ is odd. $\square$

Same pattern, different details.

Divisibility

Here’s another definition that works exactly the same way.

Definition 5.1.2 (Divisibility). For integers a and d with $d \neq 0$, we say d *divides* a (written $d \mid a$) if $a = dk$ for some integer k .

So “ $3 \mid 12$ ” because $12 = 3 \cdot 4$, and “ $5 \nmid 7$ ” because there’s no integer k with $7 = 5k$.

Notice this has the same structure as evenness: it’s an existential claim buried inside a definition. So proofs about divisibility work the same way — unpack, do algebra, repack.

Claim: For all integers a, b, c : if $a \mid b$ and $b \mid c$, then $a \mid c$. (Transitivity of divisibility.)

Proof: Let a, b, c be arbitrary integers with $a \mid b$ and $b \mid c$. By definition, $b = aj$ for some integer j , and $c = bk$ for some integer k . Then

$$c = bk = (aj)k = a(jk).$$

Since jk is an integer, we have $c = am$ where $m = jk$, so $a \mid c$. $\square$

Rational numbers

One more definition in this style.

Definition 5.1.3 (Rational number). A real number r is *rational* if $r = a/b$ where a, b are integers and $b \neq 0$.

Showing that a number is rational is really just an existence proof — you need to exhibit the integers a and b . For example, 0.75 is rational because $0.75 = 3/4$, and we’ve exhibited $a = 3$, $b = 4$ ¹.

Exercise 1. Prove: the product of two rational numbers is rational. (Use the same unpack-algebra-repack pattern from the proofs above.)

5.2 Counterexample

Direct proofs let you *prove* universal statements. But what if a universal statement is *false*? How do you disprove it?

A universal statement “for all x , $P(x)$ ” is false if there exists *even one* x where $P(x)$ is false. That one example is called a *counterexample*. This is just the negation of a universal: $\neg(\forall x. P(x)) \equiv \exists x. \neg P(x)$. We saw this equivalence back in Module 4!

So to disprove a universal, you just need to find one concrete value that breaks it. Let’s see this in action.

Claim to disprove: For every integer n , if n is odd then $(n^2 - 1)/2$ is odd.

Disproof: Counterexample: let $n = 1$. Then n is odd, and $(1^2 - 1)/2 = 0/2 = 0$, which is even, not odd. So the claim is false. $\square$

That’s the whole proof. One example. That’s all you need.

Here’s the important asymmetry: to **prove** a universal statement you need to work with an *arbitrary* element — you can’t pick a specific one, because you need the argument to work for all of them. But to **disprove** a universal you just need *one specific* counterexample. This asymmetry shows up everywhere and is worth internalizing.

Exercise 2. Disprove: for every integer n , $n^2 > n$. (Find a counterexample and verify it.)

If you’re a programmer, think of it this way: proving a function is correct means showing it works *for all inputs*. Finding a bug means finding *one bad input*. Testing is the art of finding counterexamples; formal verification is the art of proving universals. That’s why testing can show the presence of bugs but never their absence².

5.3 Existence proofs

Counterexamples are about *disproving* universals, but what about *proving* existentials? If you want to show $\exists x : A. P(x)$, how do you actually do it?

The most straightforward way is a *constructive existence proof*: you produce a concrete witness and verify that it satisfies the predicate. This is exactly $\exists$ -introduction from Module 4! You hand over a specific element $c : A$ along with a proof that $P(c)$ holds, and you’re done.

Claim: There exists a real number $x > 1$ such that $2^x > x^{10}$.

Proof: Let $x = 100$. Then $x > 1$ and we need to check that $2^{100} > 100^{10}$. Well, $100^{10} = (10^2)^{10} = 10^{20}$. And $2^{10} = 1024 > 10^3$, so $2^{100} = (2^{10})^{10} > (10^3)^{10} = 10^{30}$. Since $10^{30} > 10^{20}$, we have $2^{100} > 100^{10}$. $\square$

¹Technically you could also use $a = 6$, $b = 8$, or any other representation. There’s no uniqueness requirement in the definition, though you can get uniqueness if you insist on lowest terms.

²This is usually attributed to Dijkstra, and it’s one of those quotes that sounds deep until you realize it’s just the logical asymmetry between $\forall$ and $\exists$ wearing a trench coat.

That’s it! We picked a value, plugged it in, and checked. The hard part isn’t the proof itself — it’s figuring out *which* value to try. But once you’ve found one that works, the proof is just computation.

A couple of things to note about existence proofs:

- You don’t need to find the *best* or *smallest* witness. Any witness that works is fine. We used $x = 100$ above, but $x = 59$ also works.³ The goal is existence, not optimization.
- The witness doesn’t have to come from anywhere in particular. You can use trial and error, intuition, a calculator, whatever. As long as you *verify* it rigorously in the proof, nobody cares how you found it.
- This is a *constructive* proof because we actually built the thing we claimed exists. There are also *non-constructive* existence proofs that show something must exist without ever telling you what it is—proof by contradiction can sometimes do this—but constructive proofs are generally preferred when you can get them, especially in computer science where you usually want to actually *compute* the thing.⁴

It’s worth making the computational significance of this distinction more explicit. A constructive existence proof hands you a *witness*—a concrete object satisfying the property—and in programming terms that means it’s a function that actually returns a value. A non-constructive proof, by contrast, tells you something exists without giving you any way to compute it: you know it’s out there, but you can’t write a program to find it. This is a real practical distinction, not a philosophical one. If you prove “there exists an optimal schedule” constructively, you have an algorithm; if you prove it non-constructively, you have a theorem but no code. This is the core reason constructive mathematics appeals to computer scientists—every constructive proof is, in a precise sense, a program. (This idea can be made fully rigorous via the Curry–Howard correspondence, which we’ll gesture at later in the course, but even informally it’s a useful way to think about what your proofs are actually *doing*.)

Now that we have the basic direct proof toolkit for number theory, let’s practice these same techniques in a completely different mathematical setting. The point is to show that the proof patterns we just learned — unpack definitions, do algebra, repack — are *general*. They don’t just work for even/odd and divisibility; they work for any mathematical structure with precise definitions.

I’m going to talk about *groups*.

5.4 What is a group?

A group is one of the simplest structures in abstract algebra⁵—an incredibly useful but very simple concept in mathematics that shows up over and over again. It’s also simple enough to start with if you’ve never seen any abstract algebra before.

Here’s the definition:

³In fact the crossover point is somewhere around $x \approx 58.77$, but nobody asked us for the exact boundary. One working example is all we need.

⁴This is the constructive mathematician in your instructor peeking through again. Hi.

⁵I sometimes think of this as “My First Abstract Algebra”—it’s the training-wheels version of a much larger zoo of algebraic structures.

Definition 5.4.1 (Group). A *group* is a set G , an element $e : G$, and a binary operation $* : G \rightarrow G \rightarrow G$ that satisfies the following axioms:

(associativity) $\forall a, b, c : G. (a * b) * c = a * (b * c)$

(left-identity) $\forall g : G. e * g = g$

(right-identity) $\forall g : G. g * e = g$

(inverses) $\forall g : G. \exists g^{-1} : G$ such that both

1. (left-inverse) $g^{-1} * g = e$, and

2. (right-inverse) $g * g^{-1} = e$.

So what is this saying? A group is a triple of data: a base set G , a binary operation $*$ on the set, and an element e that acts as the identity for the operation. Associativity tells us that repeated applications of the operation don't depend on how we parenthesize them, the identity axioms guarantee that e really acts as an identity, and the inverse axioms guarantee that every element has an inverse that can be used to cancel it out.

Let's start by proving that some things **are** or are **not** groups.

5.5 The integers under addition form a group

Proposal: the set of integers, $\mathbb{Z}$, is a group with addition and an identity element of 0.

Proof: To prove this we need to show all of the axioms hold for the integers. We're allowed to assume the standard definitions of addition, negation, and 0 since we're proving that these form a group not defining them formally. If you were doing this *inside* a logic you'd have to first use the tools of the logic to give definitions for $\mathbb{Z}$ and for all the operations on it. This is essentially what the Natural Number Game requires.

Proof of associativity: Let $a, b, c : \mathbb{Z}$. Then $(a + b) + c = a + (b + c)$ by the associativity of integer addition.

Proof of left-identity: Let $n : \mathbb{Z}$, an arbitrary element, we need to show that $0 + n = n$. But this follows by the definition of addition on $\mathbb{Z}$!

Proof of right-identity: Let $n : \mathbb{Z}$, an arbitrary element, we need to show that $n + 0 = n$. But this follows by the definition of addition on $\mathbb{Z}$!

Now to show the inverses we need to show that for every element of $\mathbb{Z}$ we have a proposed inverse-under-addition for this element, and *then* we prove that the element really is both the left and right inverse.

The obvious candidate for every $n : \mathbb{Z}$ would be $-n$!

So now the theorems we need to prove become

$\forall n. (-n) + n = 0$

$\forall n. n + (-n) = 0$

but this is just a funnily written subtraction! And we already have defined subtraction to work this way, so our sub-theorems work automatically.

Thus we've shown that $(\mathbb{Z}, +, 0)$ is a group. □

Exercise 3. Is $(\mathbb{Z}, -, 0)$ a group (the integers with subtraction as the binary operation and 0 as the proposed identity)? Either verify all four axioms or find one that fails.

Exercise 4. Find the error in the following “proof” that every integer is even:

“Let n be an integer. Suppose $n = 2k$ for some integer k . Then $n = 2k$, which is even by definition. Therefore n is even.”

What proof technique error does this commit?

5.6 The integers under multiplication do *not* form a group

We can *disprove* that the integers with multiplication as the binary operation and 1 as the identity are a group easily. We just need to show one of the theorems cannot hold.

Let's show that inverses don't exist!

Consider the integer 2. For $(\mathbb{Z}, \cdot, 1)$ to be a group, there would need to be some integer n such that $2 \cdot n = 1$. But for any integer n , the product $2 \cdot n$ is always even⁶, and 1 is odd. So there is no integer that can act as the multiplicative inverse of 2.

Since we've found even one element without an inverse, the inverse axiom fails and it's not a group! $\square$

5.7 Proof of uniqueness of identity

We can also prove properties about *all* groups, not just a specific group. Like let's prove the uniqueness of identity elements. In other words, that if there are two elements in a group that satisfy the identity axioms then they must be **equal**.

Theorem: Let $(G, *, e)$ be a group and let $e' : G$ satisfy the left and right identity properties. Then $e = e'$.

Proof: We'll prove this by calculation with the properties of identity.

$$\begin{array}{ll} e = e * e' & \text{(since } e' \text{ is a right-identity: } e * e' = e) \\ e * e' = e' & \text{(since } e \text{ is a left-identity: } e * e' = e') \end{array}$$

and thus $e = e'$. $\square$

This result tells us something important: the identity element of a group isn't just convenient notation—it's *forced*. Any element that behaves like an identity must be the same as the one we started with. This kind of uniqueness result is typical in algebra: the axioms pin things down more tightly than you might expect.

⁶If you want a bonus challenge, prove this property of multiplication with natural induction! It's not too hard but is a good exercise

Chapter 6

Modular Arithmetic and Proofs by Cases

6.1 The modulus operation

An operation that's important in both mathematics and computer science is the modulus operator, sometimes called `mod` or `%`. This is the operation that gives you the remainder after integer division.

Examples:

```
5 % 2 = 1
9 % 3 = 0
59 % 4 = 3
```

The modulus operation takes the natural numbers—which are NOT a group under addition, because there are no additive inverses—and gives them really nice properties!

6.2 The quotient-remainder theorem

Let's make this precise.

Theorem 6.2.1 (Quotient-Remainder Theorem). For any integer n and any positive integer d , there exist **unique** integers q and r such that

$$n = d \cdot q + r \quad \text{and} \quad 0 \leq r < d.$$

We call q the *quotient* (this is integer division) and r the *remainder* (this is the mod operation).

This theorem formalizes the quotient-remainder decomposition: the decomposition into quotient and remainder always exists and is always unique.

If you're a programmer you already know this in your bones. In Python, `//` gives you q and `%` gives you r . In C, `/` on ints gives you q and `%` gives you r —at least for non-negative values. For negative numbers, C's division truncates toward zero rather than rounding down, so the remainder can be negative; Python's `//` and `%` always match the theorem. So the quotient-remainder theorem is literally just this:

```
n == d * (n // d) + (n % d) # always True!
```

That's it. That's the theorem, in code.

Let's do a worked example. Take $n = 67$ and $d = 4$. Then $67 = 4 \cdot 16 + 3$, so $q = 16$ and $r = 3$. You can check: $4 \times 16 = 64$, and $64 + 3 = 67$. And $0 \leq 3 < 4$, so the remainder condition is satisfied.

Why does this matter? The theorem guarantees that every integer falls into exactly one *residue class* mod d : the remainder is always one of $0, 1, 2, \dots, d-1$. This is why proof by cases using mod works! For example, every integer is either even ($r = 0 \bmod 2$) or odd ($r = 1 \bmod 2$)—there’s no third option. The quotient-remainder theorem is what makes that airtight.

6.3 Floor and ceiling

Two functions that show up constantly in both math and programming are *floor* and *ceiling*.

Definition 6.3.1 (Floor and Ceiling). The *floor* of a real number x , written $\lfloor x \rfloor$, is the largest integer that is less than or equal to x —“rounding down.” In Python this is `math.floor(x)`, or for positive numbers just `int(x)`.

The *ceiling* of x , written $\lceil x \rceil$, is the smallest integer that is greater than or equal to x —“rounding up.” In Python: `math.ceil(x)`.

Some examples:

$$\lfloor 3.7 \rfloor = 3, \quad \lceil 3.7 \rceil = 4, \quad \lfloor 5 \rfloor = 5, \quad \lceil 5 \rceil = 5, \quad \lfloor -2.3 \rfloor = -3, \quad \lceil -2.3 \rceil = -2.$$

One key property: $\lfloor x \rfloor = x$ if and only if x is an integer.¹ If x has any fractional part at all, the floor will be strictly less than x .

There’s a nice connection to the quotient-remainder theorem too. For positive integers n and d , the quotient q is exactly $\lfloor n/d \rfloor$. So floor and integer division are really the same thing.

Fun fact: the floor function shows up all over computer science. Array indexing, hash functions, binary search—anywhere you need to convert a real-valued quantity into an integer index, you’re probably using floor.

Exercise 1. Compute $\lfloor -3.7 \rfloor$, $\lceil -3.7 \rceil$, $\lfloor 4 \rfloor$, and $\lceil 4 \rceil$. (Be careful with the negative value—remember that floor rounds *down*, not toward zero.)

6.4 $\mathbb{Z}/n\mathbb{Z}$ is a group

In Module 5 we proved that $(\mathbb{Z}, +, 0)$ is a group. Now we’ll see that modular arithmetic gives us *finite* groups—same axioms, but with a finite set of residues instead of all the integers.

For any positive integer n , we can define the set of residues $\mathbb{Z}/n\mathbb{Z} = \{0, 1, \dots, n-1\}$. This set **is** actually a group under addition. We’ll write $\oplus$ for the modular addition to distinguish it from ordinary $+$:

```
q, r : Z % n
q (+) r = (q + r) % n
```

In other words, we perform ordinary addition and then take the modulus. (In practice, most texts just write $+$ for both and let context disambiguate, but the distinction is worth noting.)

The group axioms work the way you’d hope: associativity comes from ordinary integer addition, the identity is still 0, and every residue has an additive inverse.

For every $q \in \mathbb{Z}/n\mathbb{Z}$, the inverse is the element $r = (n - q) \bmod n$.

Why is this the inverse? Because

¹This sounds obvious but you’ll need to prove it carefully on the assignment!

$$\begin{aligned}
 q (+) (n - q) &= (q + n - q) \% n \\
 &= n \% n \\
 &= 0
 \end{aligned}$$

Exercise 2. In $\mathbb{Z}/7\mathbb{Z}$, find the additive inverse of 3. Verify your answer by checking that the sum of 3 and your proposed inverse is 0 under modular addition.

6.5 $\mathbb{Z}/p\mathbb{Z}$, where p is prime

When the number you're taking the modulus by is prime, something **really** special happens. It isn't just a group under addition—if you exclude 0, the nonzero residues also form a group under multiplication. That is, $((\mathbb{Z}/p\mathbb{Z})^\times, \cdot, 1)$ satisfies exactly the same group axioms we checked for the integers in Module 5, just with a different set and a different operation.

$$\begin{aligned}
 q, r : \mathbb{Z} \% p, q \neq 0, r \neq 0 \\
 q * r = (q * r) \% p
 \end{aligned}$$

Theorem 6.5.1 (Bézout's Identity). For any integers a and b , there exist integers x and y such that $ax + by = \gcd(a, b)$.

We won't prove this here, but Module 7 shows how the extended Euclidean algorithm computes these coefficients.

It's maybe not obvious that multiplicative inverses exist, but they do: if q is a nonzero residue mod p and p is prime, then $\gcd(q, p) = 1$, so Bézout's identity gives integers x, y with $qx + py = 1$. Reducing mod p then shows $qx \equiv 1 \pmod{p}$, so x is a multiplicative inverse of q modulo p .

Distributivity of multiplication over addition follows from distributivity in $\mathbb{Z}$, so we won't reprove it here.

The importance of this is that it makes $\mathbb{Z}/p\mathbb{Z}$ a *finite field*: addition works on all residues, and multiplication works on the nonzero residues. This is actually really important in cryptography!

Exercise 3. In $\mathbb{Z}/5\mathbb{Z}$, find the multiplicative inverse of 3. Verify your answer by checking that $3 \cdot x \equiv 1 \pmod{5}$.

For further reading on finite fields and their applications:

https://en.wikipedia.org/wiki/Finite_field_arithmetic

https://en.wikipedia.org/wiki/Finite_field#Discrete_logarithm

6.6 Proofs by cases and the connection to or-elimination

Proof by cases is $\vee$ -elimination in disguise. If you have an exhaustive set of cases for the data of your problem—like a number is either 0 or non-zero, or a number is positive, zero, or negative, or a number is either odd or even, &c.—then you can treat the disjunction of those cases as a tautology and apply $\vee$ -elimination to it.

Let's do a concrete example. We'll prove that for every integer n , the quantity $n^2 - n + 3$ is odd. The quotient-remainder theorem tells us every integer is either even or odd (the two residue classes mod 2), so those are our two cases.

Case 1: n is even, so $n = 2k$ for some integer k . Then

$$n^2 - n + 3 = (2k)^2 - 2k + 3 = 4k^2 - 2k + 3 = 2(2k^2 - k + 1) + 1,$$

which is odd.

Case 2: n is odd, so $n = 2k + 1$ for some integer k . Then

$$n^2 - n + 3 = (2k + 1)^2 - (2k + 1) + 3 = 4k^2 + 4k + 1 - 2k - 1 + 3 = 4k^2 + 2k + 3 = 2(2k^2 + k + 1) + 1,$$

which is odd.

So in both cases we get something of the form $2m + 1$, which is odd. Since every integer is either even or odd, we're done. Notice how the quotient-remainder theorem is doing the heavy lifting here—it's the reason we know these two cases are *exhaustive*. We don't have to worry about some integer that's neither even nor odd, because the theorem guarantees there isn't one.

Intermezzo: What Does It Mean for Things to Be “The Same”?

In Module 1 we defined equivalence relations—relations that are reflexive, symmetric, and transitive—and noted that equality is the canonical example. In Module 6 we met modular congruence: two integers are “the same” mod n when they leave the same remainder. We built $\mathbb{Z}/n\mathbb{Z}$ out of this idea without pausing to ask a deeper question: *what are we really doing when we declare two different things to be “the same”?*

The answer turns out to be one of the most important ideas in mathematics and computer science, and it’s worth making explicit.

6.7 Equivalence as abstraction

Every equivalence relation is a *choice* about which differences to ignore. When you write $17 \equiv 3 \pmod{7}$, you’re saying: “I don’t care about multiples of 7.” The numbers 17 and 3 are genuinely different integers, but for your purposes—working in $\mathbb{Z}/7\mathbb{Z}$ —the difference between them is irrelevant. You’ve chosen to ignore it.

This pattern is everywhere:

- **Modular arithmetic mod n :** ignore multiples of n . Two integers are “the same” if their difference is divisible by n .
- **Case-insensitive string comparison:** ignore capitalization. “Hello” and “hello” are different strings, but if you’re searching a database and don’t care about case, you declare them equivalent.
- **Graph isomorphism:** ignore vertex labels, keep structure. The graph with vertices $\{1, 2, 3\}$ and edges $\{(1, 2), (2, 3)\}$ is “the same” as the graph with vertices $\{a, b, c\}$ and edges $\{(a, b), (b, c)\}$ —they’re both paths of length 2.
- **Extensional equality of programs:** ignore implementation, keep input-output behavior. A bubble sort and a merge sort are “the same function” if you only care about what output they produce for each input.

Each of these is genuinely an equivalence relation—you can verify that each one is reflexive, symmetric, and transitive. (Try it for graph isomorphism if you want a small challenge.) And each one amounts to the same conceptual move: deciding that certain differences don’t matter for the question you’re asking.

This is precisely the idea that the mathematician Hermann Weyl championed. Weyl argued that the essence of mathematical abstraction is *identifying things that differ only in irrelevant ways*. In

his view, a mathematical concept isn't a specific thing—it's what remains after you've stripped away everything you've chosen to ignore. The number 3 isn't any particular collection of three objects (three apples, three dots, three stars). It's what *all* collections of three objects have in common. The concept “three” is an equivalence class.

6.8 Quotient sets

If every equivalence relation is a choice about what to ignore, then we should be able to form a new set that *only remembers what we chose to keep*. That's exactly what a quotient set is.

Definition 6.8.1 (Equivalence Class). Given a set A and an equivalence relation $\sim$ on A , the *equivalence class* of an element $a \in A$ is the set of all elements equivalent to a :

$$[a] = \{b \in A \mid a \sim b\}.$$

Definition 6.8.2 (Quotient Set). The *quotient set* $A/\sim$ is the set of all equivalence classes:

$$A/\sim = \{[a] \mid a \in A\}.$$

The equivalence classes partition A : every element belongs to exactly one class, and distinct classes don't overlap. (This is a theorem you can prove from the reflexivity, symmetry, and transitivity axioms—it's a good exercise.)

Now, concrete examples:

Modular arithmetic. Define $\sim$ on $\mathbb{Z}$ by $a \sim b$ iff $n \mid (a - b)$. The equivalence classes are exactly the residue classes $[0], [1], \dots, [n - 1]$. The quotient set $\mathbb{Z}/\sim$ is $\mathbb{Z}/n\mathbb{Z}$ —exactly the set we worked with in Module 6. This is not a coincidence or an analogy. The notation $\mathbb{Z}/n\mathbb{Z}$ literally means “the integers, quotiented by the equivalence relation of congruence mod n .” We were building a quotient set all along.

Case-insensitive strings. Let A be the set of all strings over the English alphabet, and define $s_1 \sim s_2$ iff s_1 and s_2 differ only in capitalization. Then $[\text{“Hello”}] = \{\text{“Hello”}, \text{“hello”}, \text{“HELLO”}, \text{“hELLO”}, \dots\}$ —all $2^5 = 32$ capitalizations of a five-letter string. The quotient set $A/\sim$ is the set of case-insensitive strings. When Python's `str.lower()` method converts a string to lowercase before comparing, it's choosing a *representative* from each equivalence class.

Constructing the integers. Here's a deeper example. Suppose you only have the natural numbers $\mathbb{N} = \{0, 1, 2, \dots\}$ and you want to *build* the integers. You can't just “add negatives”—you need to define them from what you already have. The trick: represent each integer as a *pair* (a, b) of natural numbers, where the pair is meant to stand for $a - b$. So $(5, 3)$ represents 2, and $(3, 5)$ represents -2 .

But many different pairs represent the same integer: $(5, 3)$, $(6, 4)$, $(7, 5)$, and $(100, 98)$ all represent 2. So we define an equivalence relation on $\mathbb{N} \times \mathbb{N}$:

$$(a, b) \sim (c, d) \quad \text{iff} \quad a + d = b + c.$$

(Notice the trick: we avoid writing $a - b = c - d$, which would require subtraction—the very thing we haven't defined yet. Instead we rearrange to $a + d = b + c$, which only uses addition of natural numbers.)

The integers *are* the quotient set $(\mathbb{N} \times \mathbb{N})/\sim$. The integer 2 is the equivalence class $[(5, 3)] = [(6, 4)] = [(7, 5)] = \dots$. This construction is standard in mathematics, and it's a beautiful instance of the pattern: define what you want as what remains after you've identified everything that should be “the same.”

6.9 The connection to types and programming

In programming, you make equivalence-class decisions constantly—often without realizing it.

Hash maps. When you use a dictionary in Python or a `HashMap` in Java, you’re relying on a notion of key equality. Two keys are “the same” if they produce the same hash *and* compare equal under `__eq__` or `.equals()`. The hash map doesn’t distinguish between two keys that are equal in this sense, even if they’re different objects in memory. This is an equivalence relation on the set of all possible key objects, and the hash map is organized by equivalence classes.

Database normalization. In a relational database, two rows in a table represent the same entity if their primary keys match. You might have a `users` table where the primary key is `user_id`. Two rows with the same `user_id` are “the same user” even if other columns differ (perhaps one was updated more recently). The primary key defines the equivalence relation; normalization is partly about making this choice explicit and consistent.

Interfaces and duck typing. In Python, if two objects both support `__len__` and `__getitem__`, you can use either one wherever you need a sequence. The objects might be implemented completely differently—one a list, one a database cursor—but from the perspective of code that uses the sequence interface, they’re interchangeable. “If it walks like a duck and quacks like a duck...” is an equivalence relation: two objects are equivalent if they support the same operations.

In type theory, this idea is formalized through *quotient types*: you can define a type where certain values are considered equal even though they’re constructed differently. This is the formal version of what every programmer does informally when they implement `__eq__` for a class.

And this connects back to Module 1’s discussion of extensional versus intensional equality. Extensional equality of functions—two functions are equal iff they produce the same output for every input—is a quotient. You take the set of all programs and identify those with the same input-output behavior. What remains is the set of “functions” in the mathematical sense, with implementation details erased. Every time you write a specification for a function (“it should sort the list”) and don’t care how it does it, you’re working in this quotient.

6.10 Weyl’s insight, and why it matters

Hermann Weyl (1885–1955) was one of the twentieth century’s most versatile mathematicians, making deep contributions to analysis, geometry, number theory, and mathematical physics. But some of his most lasting influence was philosophical. Weyl thought carefully about what mathematical objects *are*, and he arrived at a striking answer: we never study raw objects. We study objects *up to equivalence*.

When a geometer studies triangles, she doesn’t care about their position or orientation in the plane—she studies them up to congruence. When a topologist studies shapes, she doesn’t care about bending or stretching—she studies them up to continuous deformation. When a number theorist studies residues mod n , she doesn’t care about the particular integer—she studies them up to congruence mod n . In each case, the mathematician has *chosen an equivalence relation*, and that choice determines the level of abstraction at which the work happens.

The choice of equivalence relation is not a minor technical detail. It’s the most consequential decision you make, because it determines what your theory can see and what it’s blind to. Geometry up to congruence can see lengths and angles; geometry up to similarity can see angles but not lengths; topology can see connectedness but not angles or lengths. Each level of abstraction is useful for different questions, and each one is defined by choosing what to ignore.

This is the same insight that drives good software design. An interface defines what you care

about; everything behind the interface is an implementation detail you're choosing to ignore. The more carefully you choose what to expose and what to hide, the more robust your abstractions become. A well-designed API is, in a precise sense, a well-chosen equivalence relation on the set of possible implementations.

So the next time you declare two things “the same”—whether in a proof, a program, or a conversation—pause and ask: what am I choosing to ignore? That choice is where the real thinking lives.

Exercise 1. Define an equivalence relation on the set of all Python programs such that two programs are equivalent if and only if they produce the same output for every input. Is this the same as checking whether the source code is identical? Why or why not? (Hint: think about what the equivalence classes look like. How many programs are in the equivalence class of a program that sorts a list?)

Exercise 2. Consider the set of fractions $\{a/b \mid a, b \in \mathbb{Z}, b \neq 0\}$. Define an equivalence relation $\sim$ that captures when two fractions represent the same rational number. (That is, give a precise condition for when $a/b \sim c/d$.) Verify that your relation is reflexive, symmetric, and transitive. What are the equivalence classes? How does this connect to the construction of the integers from pairs of naturals discussed above?

Chapter 7

Indirect Proof, the Euclidean Algorithm, and Hoare Logic

7.1 More on proofs by contradiction and proofs by contrapositive

Back in Module 3 we derived proof by contradiction as a tautology, starting from modus tollens and working through a bunch of substitution and simplification. That was important for understanding *why* the technique is valid, but now we need to get good at *using* it.

7.1.1 Proof by contradiction, in practice

Let's recall the shape of proof by contradiction. You want to prove some statement P . Instead of proving it directly, you assume $\sim P$ —that is, you assume the *negation* of what you want to prove—and then you show that this assumption leads to a contradiction. A contradiction is when you can derive both some statement Q and its negation $\sim Q$ at the same time, which obviously can't happen. Since the assumption $\sim P$ led to nonsense, you conclude that P must be true after all.

The template looks like this:

```
Goal: prove P
(1) Assume  $\sim P$ 
(2) ... reasoning ...
(3) Derive  $Q$ 
(4) Derive  $\sim Q$ 
(5) Contradiction! Therefore  $P$ .
```

When should you reach for this technique? A good rule of thumb: if the statement you're trying to prove is a *negative* statement ("there is no...", "it is not the case that...", " x does not divide y ") then contradiction is often a natural fit. Why? Because assuming the negation of a negative statement gives you a *positive* assumption, which tends to be easier to work with.

Let's do a classic example.

Theorem 7.1.1. $\sqrt{2}$ is irrational.

Proof by contradiction.

Assume, for the sake of contradiction, that $\sqrt{2}$ is *rational*. Then by definition of rational, we can write $\sqrt{2} = \frac{a}{b}$ where a and b are integers with $b \neq 0$ and the fraction is in lowest terms (meaning $\gcd(a, b) = 1$ —hey, look, the stuff from the next section is already showing up).

Squaring both sides we get $2 = \frac{a^2}{b^2}$, so $a^2 = 2b^2$.

This means a^2 is even. But if a^2 is even then a must be even.¹ So we can write $a = 2c$ for some integer c .

Substituting back: $(2c)^2 = 2b^2$, so $4c^2 = 2b^2$, so $b^2 = 2c^2$. But this means b^2 is even, so b is even by the same reasoning.

Now here's the contradiction: we said the fraction $\frac{a}{b}$ was in lowest terms, meaning a and b share no common factors. But we just showed both a and b are even, so they share the common factor 2. Contradiction!

Therefore $\sqrt{2}$ is irrational.

That's a really satisfying proof and one of the oldest in all of mathematics—it goes back to the ancient Greeks, who were apparently quite disturbed by the discovery.

7.1.2 Proof by contrapositive

The other indirect technique is proof by *contrapositive*. This one is specifically for proving implications. So say you want to prove “if P then Q ”, written $P \rightarrow Q$. The contrapositive of this statement is $\sim Q \rightarrow \sim P$, and it turns out these are logically equivalent!

We actually already verified this back in Module 3 when we showed that modus tollens is a tautology. But let's just check the equivalence directly with a quick truth table:

P	Q	$P \rightarrow Q$	$\sim Q \rightarrow \sim P$
T	T	T	T
T	F	F	F
F	T	T	T
F	F	T	T

Same column, so they're equivalent. This means that whenever you want to prove $P \rightarrow Q$, you're free to instead prove $\sim Q \rightarrow \sim P$. Sometimes this is way easier because flipping the direction and negating everything gives you better things to work with.

Here's the classic example.

Theorem 7.1.2. For any integer n , if n^2 is even then n is even.

If we tried to prove this directly we'd start with “assume n^2 is even” and then... what? We know $n^2 = 2k$ for some integer k but it's not obvious how to extract information about n from that.

Instead, let's prove the contrapositive: if n is **not** even (i.e. n is odd) then n^2 is **not** even (i.e. n^2 is odd).

Proof by contrapositive.

Assume n is odd. Then $n = 2k + 1$ for some integer k . So

$$n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$$

which is odd. See how much cleaner that was? The direct proof was stuck, but the contrapositive version just fell right out. The reason is that “ n is odd” gives us a concrete form to compute with ($n = 2k + 1$), whereas “ n^2 is even” is much harder to decompose.

¹Why? Because if a were odd, say $a = 2k + 1$, then $a^2 = 4k^2 + 4k + 1$ which is odd. So a can't be odd, meaning it must be even. This is actually a proof by contrapositive hiding inside our proof by contradiction!

7.1.3 Contradiction vs. contrapositive: when to use which

You might be wondering: when should I use contradiction and when should I use contrapositive? Here's my advice:

- If you're trying to prove an *implication* ($P \rightarrow Q$) and you're stuck on the direct proof, try the **contrapositive** first. It's usually cleaner and more direct than a full contradiction argument.
- If you're trying to prove a statement that *isn't* naturally an implication—especially negative or existential statements like “there is no x such that...” or “ x is not divisible by...”—then **contradiction** is your go-to.
- If you're not sure, contradiction always works as a fallback. You can always just assume the negation and see what breaks. The worst that happens is your proof is a little longer than it needs to be.

There's a subtlety worth noting for those interested in constructive mathematics. Modus tollens—from $P \rightarrow Q$ and $\neg Q$, conclude $\neg P$ —is constructively valid and doesn't require excluded middle. But “proof by contrapositive” as a *technique*—proving $\neg Q \rightarrow \neg P$ in order to establish $P \rightarrow Q$ —relies on the equivalence between an implication and its contrapositive, which does require excluded middle. So strictly speaking, both contradiction and contrapositive are classical techniques. However, contrapositive proofs tend to be more transparent and informative, which is why they're generally preferred when available. In proof assistants like Lean, Agda, or Coq, you'd need to explicitly invoke a classical axiom for either technique, but a contrapositive proof usually requires it in only one place (at the very end, to flip the direction) while a contradiction proof may obscure the structure more thoroughly.

In your homework this week you'll get to practice both techniques. One tip for the contradiction proofs: when you assume the negation, make sure you actually *use* the assumption somewhere to derive the contradiction. If your proof never references the assumption, something has gone wrong—you've probably actually found a direct proof in disguise!

Exercise 1. Prove by contrapositive: for any integer n , if n^3 is even then n is even. (Hint: the structure is very similar to the n^2 proof above. Start by assuming n is odd and show that n^3 is odd.)

7.2 GCD, Euclid's algorithm, &c.

In this section, we're going to talk a little bit about Euclid's algorithm, the *extended* version of Euclid's algorithm, and how it relates to finding multiplicative inverses for modular arithmetic $\mathbb{Z}/m$.

First, let's nail down the definition of the “greatest common divisor.”

Definition 7.2.1 (Greatest Common Divisor). For integers a and b that are not both zero, the *greatest common divisor* of a and b , denoted $\gcd(a, b)$, is the greatest *positive* integer that divides both a and b .

For example

$$\gcd(12, 24) = 12$$

$$\gcd(21, 35) = 7$$

```
gcd(1024,1536) = 512
```

```
gcd(18,0) = 18
```

More generally, if $a \neq 0$ then $\gcd(a, 0) = |a|$. The reason is that every integer divides 0, so the common divisors of a and 0 are exactly the divisors of a , and the greatest positive one is $|a|$. (We leave $\gcd(0, 0)$ undefined.)

How do you *calculate* the gcd when it isn't obvious? The classic solution is *Euclid's algorithm*.

Euclid's algorithm is based on the observation that if $a = n \cdot b + r$ then $\gcd(a, b) = \gcd(b, r)$, or in other words $\gcd(a, b) = \gcd(b, a \% b)$. We keep doing that until the right-hand argument is 0; if it hits 1 first, then we already know the gcd is 1 and can stop early.

Here's a quick implementation in Python:

```
def gcd(a,b):
    if b == 1:
        return 1
    elif b == 0:
        return abs(a)
    else:
        return gcd(b,a%b)
```

What about the *extended* Euclidean algorithm? The idea is that we can simultaneously calculate the gcd *and* coefficients x, y such that

```
a*x + b*y = gcd(a,b)
```

What's the use of this? It helps us calculate multiplicative inverses in arithmetic modulo n . (See Module 6 for the modular-arithmetic background.)

How? Well let's first assume that n is prime (this is stronger than we technically need, but it implies that every **nonzero** residue class has a multiplicative inverse, and the case of arithmetic mod a prime is really the one we care about anyway).

Then we can use the extended Euclidean algorithm to calculate

```
a*x + n*y = gcd(a,n)
```

but what's the $\gcd(a, n)$? If a is not divisible by n , then since n is prime we must have $\gcd(a, n) = 1$, so we've got

```
a*x + n*y = 1
```

and reducing modulo n , the term $n \cdot y$ vanishes (since $n \cdot y$ is divisible by n), leaving us with

```
(a*x) % n = 1 % n = 1
```

which means that x is a multiplicative inverse of a modulo n . The crucial precondition here is $\gcd(a, n) = 1$; if the gcd is not 1, then no multiplicative inverse exists.

Exercise 2. Trace the Euclidean algorithm on $\gcd(48, 18)$, showing each step. That is, write out each call $\gcd(a, b)$ and the corresponding values of $a \% b$ until you reach the base case. What is $\gcd(48, 18)$?

With all of that motivation, here's the extended Euclidean algorithm—basically the same as before except we calculate a tuple instead of just one number. I'm following the treatment here (https://en.wikipedia.org/wiki/Extended_Euclidean_algorithm)

```

def egcdRec(a,b,x0,x1,y0,y1):
    if b == 0:
        return (a,x0,y0)
    else:
        q = a // b
        return egcdRec(b,a%b,x1,x0 - q*x1,y1,y0 - q*y1)

def egcd(a,b):
    return egcdRec(a,b,1,0,0,1)

def modInverse(a,m):
    g,i,_ = egcd(a,m)
    if g != 1:
        raise ValueError("inverse_does_not_exist")
    return i % m

```

7.3 Programming logic

In this section, we’re going to talk about a specific kind of logic. This material is similar to Epp section 4.10 **but** more formalized.

We’re going to be dealing with a specific logic called Hoare logic. [Hoare logic](#)—invented originally by Tony Hoare—is a formal logic for reasoning about code directly. It has its own axioms, inference rules, syntax, and semantics. It’s a fully-fledged albeit very *different* looking logic.

Each statement in Hoare logic looks like

```
{P}C{Q}
```

Where P is a proposition about the state of the program **before** your code runs—the values of variables, output, things like that— C is the code—a series of statements in your program—and Q is another proposition about the state of the program **after** the code has run.

Think of $\{P\}C\{Q\}$ as an implication with a computational step in the middle: if P holds before C runs, then Q holds afterward.

The programming language we’ll use is a simple imperative language, with just a few kinds of statements

```

Assignment:
  x := e
where e is a valid arithmetic expression (one that uses +, -, *, /, numbers, or variables
  )

Sequencing:
  c1 ; c2
where c1 and c2 are both valid statements

Conditionals:
  if b then c1 else c2
where b is a boolean valued statement that uses (>, >=, =, /=, \&c. and variables)
and c1, c2 are other valid statements in the language

While:
  while b do c

```

where b is boolean valued statement, like above, and c is a valid statement

Skip:
skip

it's a statement that does nothing

This is a deliberately stripped-down language—just enough to explain the basic concepts of proving code correct.

Now here's our basic rules:

Skip:

$\{P\}\text{skip}\{P\}$

this rule tells us that skip changes nothing. If the proposition P was true before it's true after skip.

Sequencing:

assumes: $\{P\}c1\{Q\}$
 $\{Q\}c2\{R\}$
shows: $\{P\}c1;c2\{R\}$

this rule tells us that if $c1$ takes us from P to Q , and $c2$ takes us from Q to R , then we can glue them together with sequencing. This is just transitivity— $P \rightarrow Q$ and $Q \rightarrow R$ give us $P \rightarrow R$ —but with the code steps wired in between.

Conditionals:

assumes: $\{P \wedge B\}c1\{Q\}$
 $\{P \wedge \neg B\}c2\{Q\}$
shows: $\{P\}\text{if } B \text{ then } c1 \text{ else } c2\{Q\}$

This tells us that if Q is true after $c1$ runs when P and B are true **and** Q is true after $c2$ runs when P is true and B is false, then P implies Q when the conditional statement is run. The way to think of this is that if you take the “true” branch you're not just assuming P is true but you're also allowed to assume B is true **because you took the true branch**, and conversely for when you take the false branch.

While:

assumes: $\{P \wedge B\}c\{P\}$
shows: $\{P\}\text{while } B \text{ do } c\{P \wedge \neg B\}$

The idea is to identify a *loop invariant*—a property preserved by each iteration of the loop body. You show that if the invariant and the loop condition both hold before the body executes, the invariant still holds afterward. When the loop terminates, you know the invariant holds *and* the loop condition is false.

The next one is definitely the weirdest, and one that we need to explain a bit.

Assignment:

$\{P[x/e]\}x := e\{P\}$

What this is saying is that **if** the proposition P is true after substituting the expression e for the variable x , then after assignment you can just conclude that P is true.

Let's look at an example because I think it'll make more sense:

If we take this fragment of a proposition, where we just are leaving a hole for ourselves in the pre-condition, and we **want** to conclude that after subtracting 1 from the variable x then $x > 0$

```
{??} x := x - 1 {x > 0}
```

looking at this, what does the precondition have to be? It needs to be that $x > 1$ or... another way to say this would be that $(x - 1) > 0$, which is exactly what you get if you substitute the expression e for x in P —which in this case means replacing x with $x-1$.

So the assignment rule lets us write things like this

```
{x - 1 > 0} x := x - 1 {x > 0}
```

The assignment rule as written isn't super useful on its own — it's too specific! Like it can't let us conclude an obvious fact like

```
{x > 0} x := x + 1 {x > 0}
```

in other words, that if x is greater than zero incrementing it will keep it greater than zero—exactly the kind of thing we might need to build a loop invariant.

Thankfully, there's one last rule that's super useful that helps us glue all of these bits and pieces together:

Strengthening / Weakening:

```
assumes: {P2}c{Q2}
          P1 -> P2
          Q2 -> Q1
shows:
{P1}c{Q1}
```

Read it carefully: if we know that $\{P2\}c\{Q2\}$, in other words that $P2$ implies $Q2$ **after** running the code c , and we know that $P1$ implies $P2$, and we know that $Q2$ implies $Q1$, we're allowed to glue all these facts together to make the new fact that $P1$ implies $Q1$ after c is run. Why? Because we can take the $P1$ and turn it into a $P2$, since $P1 \rightarrow P2$, and then we can take the resulting $Q2$ and turn it into a $Q1$ since $Q2 \rightarrow Q1$.

Another way of thinking of it is that you can always make the pre-condition *stronger* and the post-condition *weaker*.

So in our above example we'd have that the assignment rule gives us

```
{x + 1 > 0} x := x + 1 {x > 0}
```

but since $x > 0$ implies $x + 1 > 0$ we can use our new rule to turn this into

```
{x > 0} x := x + 1 {x > 0}
```

which is what we wanted in the first place!

7.3.1 Putting it all together

Now that we have all the rules on the table, let's do a complete worked example that uses several of them in combination. We'll prove the following Hoare triple:

```
{x = 5} x := x - 1; x := x - 1 {x = 3}
```

In other words: if x starts at 5 and we subtract 1 twice, then x ends up at 3. Not exactly a shock, but the point is to see how the formal machinery works end to end.

We'll work backwards from the postcondition, which is generally the right strategy when the assignment rule is involved. The assignment rule tells us what the precondition *must* be for a given

assignment and postcondition, so starting at the end and peeling off one assignment at a time is the natural approach.

Step 1: Apply the assignment rule to the second assignment.

We want to end up with $\{x = 3\}$ after executing $x := x - 1$. The assignment rule says:

$\{P[x/e]\} x := e \{P\}$

Here P is $x = 3$ and e is $x - 1$, so $P[x/(x - 1)]$ means we substitute $x - 1$ for x in $x = 3$, giving us $x - 1 = 3$. That produces:

$\{x - 1 = 3\} x := x - 1 \{x = 3\} \text{ [assignment]}$

So after the first assignment runs, we need $x - 1 = 3$ (equivalently, $x = 4$) to hold.

Step 2: Apply the assignment rule to the first assignment.

Now we need $\{x = 4\}$ to hold after the first $x := x - 1$. Note that $x - 1 = 3$ and $x = 4$ are logically equivalent, so we can use either form; let's use $x = 4$ since it's a bit cleaner. Applying the assignment rule again with P being $x = 4$ and e being $x - 1$:

$P[x/(x - 1)]$ gives us $x - 1 = 4$. So:

$\{x - 1 = 4\} x := x - 1 \{x = 4\} \text{ [assignment]}$

Step 3: Chain the two assignments together with sequencing.

The sequencing rule says:

```

assumes: {P}c1{Q}
         {Q}c2{R}
shows:   {P}c1;c2{R}

```

Our two results from Steps 1 and 2 fit this pattern exactly, with $P = (x - 1 = 4)$, $Q = (x = 4)$, c_1 being the first $x := x - 1$, c_2 being the second $x := x - 1$, and $R = (x = 3)$. But we need the postcondition of the first triple to match the precondition of the second triple. The postcondition of Step 2 is $x = 4$ and the precondition of Step 1 is $x - 1 = 3$, which is the same thing (since $x - 1 = 3$ iff $x = 4$). So sequencing gives us:

$\{x - 1 = 4\} x := x - 1; x := x - 1 \{x = 3\} \text{ [sequencing]}$

Step 4: Use strengthening to connect to the stated precondition.

We're almost done, but notice the precondition we derived is $x - 1 = 4$ (i.e. $x = 5$), and the precondition we *want* is $x = 5$. These are logically equivalent, so in particular $x = 5 \rightarrow x - 1 = 4$. We can use the strengthening/weakening rule:

```

assumes: {P2}c{Q2}
         P1 -> P2
         Q2 -> Q1
shows:   {P1}c{Q1}

```

with $P_1 = (x = 5)$, $P_2 = (x - 1 = 4)$, $Q_1 = Q_2 = (x = 3)$, and c being the full program. The implication $x = 5 \rightarrow x - 1 = 4$ holds by basic arithmetic, and $x = 3 \rightarrow x = 3$ is trivially true. So we conclude:

$\{x = 5\} x := x - 1; x := x - 1 \{x = 3\} \text{ [strengthening/weakening]}$

And that's the complete proof! Let's collect the whole thing in one place for reference:


```

(1) {x - 1 = 3} x := x - 1 {x = 3} [assignment]
(2) {x - 1 = 4} x := x - 1 {x = 4} [assignment]
(3) {x - 1 = 4} x := x - 1; x := x - 1 {x = 3} [sequencing, (2)+(1)]
(4) x = 5 -> x - 1 = 4 [arithmetic]
(5) {x = 5} x := x - 1; x := x - 1 {x = 3} [strengthening, (4)+(3)]

```

Notice the overall pattern: we used the assignment rule twice (working backwards from the postcondition), glued the results together with sequencing, and then used strengthening to hook up the precondition we actually wanted. This is the same pattern you'll use for the homework problem.

Exercise 3. Prove the Hoare triple $\{x = 0\} x := x + 3 \{x = 3\}$ using the assignment rule. (This is a single assignment, so you don't need sequencing—just apply the assignment rule and then use strengthening to connect the precondition you get to the one you want.)

Exercise 4. Find the error in the following “proof” by contradiction that $\sqrt{4}$ is irrational:

“Assume for contradiction that $\sqrt{4}$ is rational. Then $\sqrt{4} = a/b$ in lowest terms. Squaring: $4 = a^2/b^2$, so $a^2 = 4b^2$. This means a^2 is even, so a is even. Write $a = 2c$. Then $4c^2 = 4b^2$, so $c^2 = b^2$, so $c = b$. But then $a = 2b$ and the fraction $a/b = 2$, so $\sqrt{4} = 2$. Contradiction!”

Where exactly does the reasoning go wrong?

That was a lot, and it's meant as an introduction to how you might prove properties about programs—not something you need to completely absorb right now. In your homework for the week you're going to prove the exciting theorem that $\{x = 0\} x := x+1; x := x+1 \{x = 2\}$ or, in other words, that $1+1 = 2$.

Intermezzo: Proofs Are Programs

We’ve now seen several different ways to write proofs: derivations in the `lstlisting` style from Module 3, Gentzen-style inference trees from the `trees` intermezzo, informal arguments with “assume... therefore,” and even machine-checked proofs in Lean via the Natural Number Game. We’ve gotten comfortable with the *mechanics* of proving things.

But here’s a question we haven’t really asked: what IS a proof? Not “how do you write one” but “what kind of mathematical object is it?” We’ve been treating proofs as sequences of steps, or as trees, or as English paragraphs. But is there something deeper going on—some underlying *structure* that all these presentations share?

The answer is yes, and it’s one of the most beautiful ideas in all of mathematics and computer science. It turns out that proofs and programs are the *same thing*.

7.4 What does a proof *look like*?

To see this, we need to think about proofs differently. Instead of asking “how do I convince someone this is true?” we ask: “what *data* does a proof carry?”

There’s a tradition in constructive mathematics called the **Brouwer–Heyting–Kolmogorov interpretation** (BHK for short) that gives a precise answer. It was developed by the Dutch mathematician L.E.J. Brouwer, the logician Arend Heyting, and the great Andrey Kolmogorov, and it says the following:

- **A proof of $A \wedge B$** is a *pair*: a proof of A together with a proof of B . You need both pieces. It’s a tuple!
- **A proof of $A \vee B$** is a *tagged value*: either a proof of A (tagged “left”) or a proof of B (tagged “right”). You have to say *which* side you’re proving, and then actually prove it. It’s a sum type—an **Either**, a tagged union!
- **A proof of $A \rightarrow B$** is a *function*: given any proof of A , it produces a proof of B . Not a specific proof for a specific A —a *method* that works for any proof of A you hand it.
- **A proof of $\forall x : S. P(x)$** is a *function*: given any element x of S , it produces a proof of $P(x)$. A machine that manufactures proofs on demand, one for each possible input.
- **A proof of $\exists x : S. P(x)$** is a *pair*: a specific witness $x \in S$ together with a proof that $P(x)$ holds for that witness. You can’t just say “something exists”—you have to *produce* the thing and *show* it works.
- **A proof of $\perp$ (false)** is... nothing. There is no proof of false. It’s the empty type—the type with no values at all.

Read that list again slowly. Does it remind you of anything? Those are *data structures*. Pairs, tagged unions, functions. The BHK interpretation is saying that proofs are made out of the same stuff as programs.

This is not a metaphor. This is not a loose analogy. This is a precise, formal correspondence, and it has a name.

7.5 The Curry–Howard dictionary

The **Curry–Howard correspondence** (sometimes called the Curry–Howard *isomorphism*, because it really is a bijection) is a dictionary that translates between logic and programming. It was discovered independently by the logician Haskell Curry in the 1930s–50s and the logician William Alvin Howard in 1969, though its full implications took decades to unfold.

Here’s the dictionary:

Logic		Programming
Proposition	$\longleftrightarrow$	Type
Proof of a proposition	$\longleftrightarrow$	Program (term) of that type
$A \rightarrow B$	$\longleftrightarrow$	Function type <code>A -> B</code>
$A \wedge B$	$\longleftrightarrow$	Pair/product type <code>(A, B)</code>
$A \vee B$	$\longleftrightarrow$	Sum type <code>Either A B</code>
$\top$ (true)	$\longleftrightarrow$	Unit type <code>()</code> (one trivial value)
$\perp$ (false)	$\longleftrightarrow$	Empty type (no values at all)
$\forall x : S. P(x)$	$\longleftrightarrow$	Dependent function / polymorphism
$\exists x : S. P(x)$	$\longleftrightarrow$	Dependent pair / existential type
Modus ponens	$\longleftrightarrow$	Function application
Implication introduction	$\longleftrightarrow$	Lambda abstraction (defining a function)
Conjunction introduction	$\longleftrightarrow$	Constructing a pair
Conjunction elimination	$\longleftrightarrow$	Projecting from a pair (<code>fst</code> , <code>snd</code>)
Disjunction introduction	$\longleftrightarrow$	Injecting into a sum (<code>Left</code> , <code>Right</code>)
Disjunction elimination	$\longleftrightarrow$	Pattern matching / case analysis
Ex falso ($\perp$ -elimination)	$\longleftrightarrow$	Handling the empty type (vacuously)

The left column is everything we’ve been doing since Module 3. The right column is everything you’ve been doing since your first programming class. They were the same subject the whole time.

Let that sink in for a moment. Every time you wrote a function that takes a pair and returns one component, you were doing conjunction elimination. Every time you pattern-matched on a tagged union, you were doing proof by cases. Every time you defined a function, you were introducing an implication.

7.6 A worked example: the same proof, four ways

Let’s make this concrete. Consider the logical tautology

$$A \wedge B \rightarrow B \wedge A$$

which says that conjunction is commutative: if A and B are both true, then B and A are both true. Not exactly shocking, but it’s a perfect example because the proof is simple enough to see the correspondence clearly.

7.6.1 As a derivation (Module 3 style)

Theorem: Show $A \wedge B \rightarrow B \wedge A$

Proof:

(a) proof that assuming $A \wedge B$, shows $B \wedge A$:

subproof: (i) $A \wedge B$, by assumption

(ii) B , by $\wedge$ -elimination on (i)

(iii) A , by $\wedge$ -elimination on (i)

show $B \wedge A$, by $\wedge$ -introduction with (ii) and (iii)

shows $A \wedge B \rightarrow B \wedge A$ by introduction of implication

7.6.2 As a Gentzen tree

$$\frac{\frac{[A \wedge B]}{B} \wedge E_2 \quad \frac{[A \wedge B]}{A} \wedge E_1}{B \wedge A} \wedge I \quad \rightarrow I$$

Read it bottom-up: we want $A \wedge B \rightarrow B \wedge A$, so we use $\rightarrow I$, which means we temporarily assume $A \wedge B$ (in brackets) and need to derive $B \wedge A$. To build $B \wedge A$ we use $\wedge I$, so we need B and A separately. We get B by $\wedge E_2$ on our assumption and A by $\wedge E_1$.

7.6.3 As a Python function

```
def swap(pair):  
    return (pair[1], pair[0])
```

That's it. We take in a pair, extract the second element and the first element, and build a new pair with them in the other order. The function `swap` is the proof. Its type signature—"takes a pair (A, B) , returns a pair (B, A) "—is the theorem.

7.6.4 As Lean 4

```
fun ⟨a, b⟩ => ⟨b, a⟩
```

If you've played the Natural Number Game, this syntax might not look too alien. The angle brackets $\langle \cdot, \cdot \rangle$ are Lean's notation for pairs (and for the `And.intro` constructor). The `fun` keyword introduces a function—implication introduction. Pattern matching on $\langle a, b \rangle$ does both conjunction eliminations at once.

Now here's the punchline: **these are all the same thing**. The derivation, the Gentzen tree, the Python function, and the Lean term are four presentations of *one* underlying mathematical object. The proof IS the program. The program IS the proof. The type of the program IS the theorem statement. A well-typed program IS a valid proof.

Go back and trace the correspondence step by step. In the Python version, `pair` is the assumption $A \wedge B$. Indexing with `pair[1]` is $\wedge$ -elimination (extracting the second component). Building the tuple `(pair[1], pair[0])` is $\wedge$ -introduction. The `def` itself is $\rightarrow$ -introduction. Every single step in the proof corresponds to a programming operation, and vice versa.

7.7 Why this matters

This correspondence is not just a curiosity. It is the theoretical foundation of an entire field.

Constructive proofs are executable. Under the BHK interpretation, every constructive proof of a proposition carries enough information to actually *compute*. A proof that $\exists x. P(x)$ doesn't just tell you that such an x exists somewhere out there in the platonic realm—it hands you a specific x and shows you why it works. A proof that $A \vee B$ doesn't just say “one of them is true, I'm not telling you which”—it tells you *which one* and proves it. This is why constructive mathematics has always appealed to computer scientists: constructive proofs are programs you can run.

Proof assistants are type checkers. This is the foundation of tools like Lean, Agda, and Coq. When you write a proof in Lean, you're really writing a program. When Lean checks your proof, it's really type-checking your program. The kernel of a proof assistant—the thing you ultimately have to trust—is a type checker. If the program has the right type, the proof is valid. Period. That's why these tools can be so reliable: type checking is a well-understood, mechanizable process.

Excluded middle is non-constructive—and now we can see *why*. Remember all those careful remarks in Modules 3 and 7 about how proof by contradiction and excluded middle are “classical” and not constructively valid? The Curry–Howard correspondence makes the reason visceral. Under BHK, a proof of $A \vee \neg A$ would be a program that, for *any* proposition A whatsoever, either produces a proof of A or produces a proof of $\neg A$. Think about what that would mean: a program that can decide *any* mathematical question. That's absurd! There's no algorithm that can look at an arbitrary proposition and tell you whether it's true or false—that's essentially the halting problem.

So if you add excluded middle as an axiom, what you're doing, computationally, is adding a primitive that magically produces a value without computing it. In programming-language terms, it corresponds to something like `call/cc` (call-with-current-continuation) in Scheme—a control operator that captures the rest of the computation and can jump back to it later. It's computationally meaningful, but it's not a *constructive* program in the ordinary sense. This is the precise sense in which classical logic has “more” theorems than constructive logic: it has a more powerful (and stranger) programming language.

This is the beginning, not the end. You'll see much more of Curry–Howard in CS 251, where we'll work with dependent types and use proof assistants not just for number games but for verifying real mathematical theorems. For now, the takeaway is this: the logical infrastructure you've been building all quarter—conjunction, disjunction, implication, quantifiers, proof rules—is not separate from programming. It *IS* programming, seen from a different angle.

Exercise 1. Under the Curry–Howard correspondence, what programming construct corresponds to proof by cases (disjunction elimination, $\vee E$)? Hint: think about what you do when you have an `Either` value in a language like Haskell or Python. You need to handle both the `Left` case and the `Right` case. What programming pattern is that?

Exercise 2. The formula $A \rightarrow (B \rightarrow A)$ is a tautology (you can check it with a truth table if you like). Under Curry–Howard, this corresponds to a type: a function that takes an `A` and returns a function from `B` to `A`. Write a Python function with this type signature. What does it do? Why does it make sense that this is a tautology?

Intermezzo: Other Logics, Other Worlds

Propositional logic and first-order logic are powerful, but they are not the only games in town. Over the past century, logicians and computer scientists have invented dozens of specialized logics, each designed to capture a particular kind of reasoning. In this intermezzo we'll take a quick tour of the zoo and then spend some time with one especially useful species: *temporal logic*, the logic of time.

The punchline is this: you already know how to learn a new logic. The methodology is always the same—define a syntax (usually with BNF), define a semantics (what the formulas mean), and then prove things using inference rules. You've done this for propositional logic in Modules 2–3, for first-order logic in Module 4, and for Hoare logic in Module 7. Every logic in this chapter follows exactly the same pattern. The toolkit transfers; only the domain changes.

7.8 The zoo of logics

Here is a very incomplete field guide to some logics you might encounter in the wild.

- **Modal logic** adds two operators to propositional logic: $\Box p$ (“necessarily p ”) and $\Diamond p$ (“possibly p ”). The idea is that there are many possible “worlds,” and $\Box p$ means p is true in *all* of them, while $\Diamond p$ means p is true in *at least one*. If that sounds like $\forall$ and $\exists$ wearing different hats, you're not wrong—modal logic can be seen as a fragment of first-order logic where the quantification is over possible worlds rather than over elements of a set.
- **Temporal logic** is modal logic where the “possible worlds” are moments in time. Instead of “necessarily” and “possibly” you get “always” and “eventually.” We'll spend most of this chapter here.
- **Linear logic** is a logic of *resources*. In classical logic, if you know p you can use that fact as many times as you want—it's free to duplicate. In linear logic, each fact is a resource that gets *consumed* when you use it. If you have one dollar and you spend it, you don't have a dollar anymore. This sounds exotic, but it turns out to be exactly the right logic for reasoning about things like memory management, concurrent access to shared data, and protocol compliance. It's also deeply connected to quantum computing, where the no-cloning theorem says you literally can't duplicate a quantum state.
- **Hoare logic** (Module 7) is a logic of programs. Its “propositions” are Hoare triples $\{P\}C\{Q\}$, its “connectives” are the programming constructs (sequencing, conditionals, while loops), and its “inference rules” are the axioms we learned: assignment, sequencing, the while rule.

The point isn't to learn all of these right now. The point is that you've already internalized the *methodology*. Propositional logic had a BNF for its syntax (Module 2), truth tables and valuations for its semantics (Module 2), and inference rules for its proof system (Module 3). Every logic on

this list has the same three components. Once you see the pattern, picking up a new logic is like picking up a new programming language when you already know how to program—the surface syntax changes, but the deep structure is familiar.

7.9 Temporal logic—logic over time

Let’s zoom in on temporal logic, because it has a beautiful connection to the material we’ve already covered.

In Module 7 we used Hoare logic to reason about programs. One of the key concepts there was the *loop invariant*: a property P that holds before the loop body executes and still holds afterward. The while rule said that if P is preserved by each iteration, then P holds when the loop terminates.

Temporal logic takes this idea and runs with it. Instead of reasoning about a single loop, temporal logic lets you make statements about *entire execution traces*—infinite sequences of states that a system passes through over time. Think of a web server that starts up, handles requests, maybe crashes, restarts, handles more requests, and so on forever. Temporal logic is the language for expressing properties of such systems.

The basic temporal operators are:

Definition 7.9.1 (Temporal operators). Let p be a proposition about the current state of a system.

- $\Box p$ (read “always p ”): p holds at every point in time from now on. This is the temporal version of “necessarily.” Think of it as an invariant that never, ever breaks—not just through one loop iteration, but forever.
- $\Diamond p$ (read “eventually p ”): p holds at some point in the future (possibly right now). Something good is going to happen—you just might have to wait.
- $\bigcirc p$ (read “next p ”): p holds in the very next state. If we think of time as a sequence of discrete steps (like clock ticks, or loop iterations, or program states), this says “one step from now, p is true.”

These operators can be combined with all the usual propositional connectives ($\wedge$, $\vee$, $\rightarrow$, $\neg$), and that’s where things get expressive.

Notice how $\Box$ and $\Diamond$ are related: $\Diamond p \equiv \neg \Box \neg p$. “Eventually p ” means “it’s not the case that p is always false.” And $\Box p \equiv \neg \Diamond \neg p$ —“always p ” means “ p never fails.” This is exactly the duality between $\forall$ and $\exists$ from Module 4, just wearing a temporal costume.

Connection to Hoare logic. Remember the while rule from Module 7?

assumes: $\{P \wedge B\} c \{P\}$
shows: $\{P\} \text{while } B \text{ do } c \{P \wedge \neg B\}$

The invariant P is preserved by every iteration of the loop body. In temporal logic terms, the while rule is establishing something like $\Box P$ *during the loop*: at every step of execution, the invariant holds. When we prove a loop invariant, we’re really doing temporal reasoning in disguise—we’re showing that a property is stable across an entire sequence of state transitions.

The difference is that Hoare logic reasons about programs that terminate (or at least, individual program fragments), while temporal logic reasons about systems that might run forever. A server, an operating system, a network protocol—these aren’t programs that finish and return a value. They’re ongoing processes, and temporal logic is built for them.

7.10 Expressing real properties

The real power of temporal logic shows up when you use it to express properties that practitioners actually care about. Here are some classic examples.

“The server eventually responds to every request.”

$$\Box (request \rightarrow \Diamond response)$$

Read it carefully: *always* (at every point in time), if a request is made, then *eventually* a response will follow. The outer $\Box$ says this must hold at every moment, not just right now. The inner $\Diamond$ says the response doesn’t have to be immediate—it just has to happen at some point in the future. This is called a *liveness* property: something good eventually happens.

“A lock is never held by two threads simultaneously.”

$$\Box \neg (held_1 \wedge held_2)$$

At every point in time, it is not the case that both thread 1 and thread 2 hold the lock. This is a *safety* property: something bad never happens.

“After every login, the user is eventually logged out.”

$$\Box (login \rightarrow \Diamond logout)$$

Same shape as the server example. Every login is eventually followed by a logout. No session lasts forever.

“The system never deadlocks.”

$$\Box \Diamond progress$$

Always, eventually, progress is made. The system might pause temporarily, but it can’t get stuck forever.

These are exactly the kinds of properties that *model checkers* verify automatically. Model checking is one of the most successful applications of formal methods in industry. Amazon Web Services uses TLA+ (a temporal logic) to verify the correctness of their distributed systems. Intel uses model checking to verify chip designs before fabrication—finding a bug after you’ve manufactured a million processors is somewhat more expensive than finding it in a formula. Microsoft has used temporal logic tools to verify device drivers in the Windows kernel. The 2007 Turing Award went to Clarke, Emerson, and Sifakis for their work on model checking.

The reason model checking works so well is precisely the methodology you’ve learned in this course: write down a formal specification (in temporal logic), build a formal model of the system, and then let a computer check that the model satisfies the specification. Syntax, semantics, proof—the same three steps, applied at industrial scale.

7.11 The pattern

Let’s make the pattern explicit by writing down the BNF for (a simple version of) temporal logic. If p ranges over atomic propositions (basic facts about the current state), then the formulas of linear temporal logic (LTL) are:

$\varphi ::= p \mid \neg \varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi \mid \varphi \rightarrow \varphi \mid \Box \varphi \mid \Diamond \varphi \mid \bigcirc \varphi$
--

Look familiar? It's the BNF from Module 2, with three new operators bolted on. The propositional connectives are still there—we've just extended the language.

The semantics is defined over infinite sequences of states, sometimes called *Kripke structures* (after the philosopher and logician Saul Kripke). A Kripke structure is a sequence $s_0, s_1, s_2, \dots$ where each s_i is a state—an assignment of truth values to the atomic propositions. Given a sequence and a position i , we define when a formula holds:

- $s_i \models p$ iff p is true in state s_i
- $s_i \models \neg\varphi$ iff $s_i \not\models \varphi$
- $s_i \models \varphi \wedge \psi$ iff $s_i \models \varphi$ and $s_i \models \psi$
- $s_i \models \Box\varphi$ iff $s_j \models \varphi$ for all $j \geq i$
- $s_i \models \Diamond\varphi$ iff $s_j \models \varphi$ for some $j \geq i$
- $s_i \models \bigcirc\varphi$ iff $s_{i+1} \models \varphi$

And there are proof rules for temporal logic, just as there are for propositional and first-order logic. For instance, $\Box$ distributes over $\wedge$ (if something is always true and something else is always true, then their conjunction is always true), and from $\Box\varphi$ you can conclude φ (if something is always true, it's true right now). These rules can be presented as Gentzen-style inference trees, exactly as in the trees intermezzo.

Same methodology. Same pattern. New domain. The toolkit from Modules 2–3 transfers directly—you just need to learn the new operators and their semantics. If you can read a BNF, define a semantics, and apply inference rules, you can learn any logic.

7.12 Exercises

Exercise 1. Express the following properties in temporal logic (using $\Box$, $\Diamond$, $\bigcirc$, and the usual propositional connectives):

- (a) “The program eventually terminates.” (Let *halted* be an atomic proposition meaning “the program has halted.”)
- (b) “The counter is always non-negative.” (Let $counter \geq 0$ be an atomic proposition.)
- (c) “Every request is eventually followed by a response.” (Let *request* and *response* be atomic propositions.)

Chapter 8

Sequences and Recursion

8.1 Definition by recursion

We’ve been building up our toolkit for a while now—logic, sets, functions, induction—and now we’re going to put a lot of it together by talking about *sequences*.

What is a sequence? Informally it’s just an ordered list of values. You’ve seen these before! Arrays in C, lists in Python, those are sequences. The values come in a definite order: there’s a first element, a second element, a third, and so on.

Definition 8.1.1 (Sequence). A *sequence* is a function from the natural numbers (or some subset of them) to some set. If we write $a_1, a_2, a_3, \dots$ what we really mean is that there’s a function $a : \mathbb{N} \rightarrow S$ for some set S , and a_n is just the value of that function at input n . The subscript is the index, the “position” in the list.

If you’re a programmer, think of a_n as `a[n]`. It’s literally the same idea: indexing into a collection.

How do we actually *define* a sequence? There are two main approaches.

8.1.1 Explicit formulas

The first way is to just give a formula that lets you compute the n th term directly from n . This is called an *explicit formula* or a *closed-form definition*.

For example, $a_n = 2n + 1$ for $n \geq 0$. That gives us the sequence 1, 3, 5, 7, 9, $\dots$ — the odd numbers. If you want the 100th term you just plug in $n = 99$ and get $a_{99} = 199$. You don’t need to know any other terms.

Let’s work through a couple more examples from the textbook.

Consider $b_j = \frac{5-j}{5+j}$ for every integer $j \geq 1$. Let’s compute the first four terms:

$$b_1 = \frac{5-1}{5+1} = \frac{4}{6} = \frac{2}{3}$$

$$b_2 = \frac{5-2}{5+2} = \frac{3}{7}$$

$$b_3 = \frac{5-3}{5+3} = \frac{2}{8} = \frac{1}{4}$$

$$b_4 = \frac{5-4}{5+4} = \frac{1}{9}$$

And here's another one: $d_m = 1 + \left(\frac{1}{2}\right)^m$ for every integer $m \geq 0$:

$$d_0 = 1 + \left(\frac{1}{2}\right)^0 = 1 + 1 = 2$$

$$d_1 = 1 + \left(\frac{1}{2}\right)^1 = 1 + \frac{1}{2} = \frac{3}{2}$$

$$d_2 = 1 + \left(\frac{1}{2}\right)^2 = 1 + \frac{1}{4} = \frac{5}{4}$$

$$d_3 = 1 + \left(\frac{1}{2}\right)^3 = 1 + \frac{1}{8} = \frac{9}{8}$$

See how the terms are getting closer and closer to 1? That's a *convergent* sequence, but we'll leave that idea for a different class.

8.1.2 Recursive definitions

The other way to define a sequence is *recursively*: you give the first term (or first few terms) and then a rule that builds each subsequent term from the ones that came before.

Instead of the explicit formula $a_n = 2n + 1$ we could equivalently say: $a_0 = 1$ and $a_n = a_{n-1} + 2$ for $n \geq 1$. Same sequence! But a very different way of thinking about it.

The classic example of a recursively defined sequence is the Fibonacci sequence:

```
fib(0) = 0
fib(1) = 1
fib(n) = fib(n-1) + fib(n-2) for n >= 2
```

This should look very familiar! It's *exactly* the same structure as writing a recursive function in Python or C. That's not a coincidence. As we discussed back in Module 4 when we introduced BNF syntax and inductive definitions, recursive definitions are really just another case of defining data inductively. The base case gives you the starting point and the recursive case tells you how to build new data from old data.

In fact, remember how in Module 4 we defined the natural numbers as $\text{Nat} := 0 \mid \text{succ Nat}$? A recursively defined sequence is doing the same thing: defining a_n by cases on whether n is the base case or a successor.

Exercise 1. Write a recursive definition for the sequence $a_n = 3n$ (i.e., give a_0 and a rule for a_n in terms of a_{n-1}). Verify your definition by computing the first four terms and checking that they match 0, 3, 6, 9.

8.1.3 Summation and product notation

There's an important piece of notation we need before we go further: *summation notation*. You've probably seen the big sigma before:

$$\sum_{i=1}^n a_i = a_1 + a_2 + a_3 + \cdots + a_n$$

This is just a compact way of writing "add up all the terms a_i where i goes from 1 to n ." The variable i is the *index* of summation, the bottom tells you where to start, the top tells you where to stop.

If you're a programmer, this is literally a for-loop:

```
total = 0
for i in range(1, n+1):
    total += a[i]
```

That's what $\sum$ means. It's a for-loop.¹

For example:

$$\sum_{i=1}^4 i^2 = 1^2 + 2^2 + 3^2 + 4^2 = 1 + 4 + 9 + 16 = 30$$

And you can also nest these things or get creative with the bounds. The expression

$$\sum_{k=0}^n (-1)^k (k^3 - 1)$$

is just alternating addition and subtraction of the terms $(k^3 - 1)$ because $(-1)^k$ flips sign.

Similarly, there's *product notation* with the big pi:

$$\prod_{i=1}^n a_i = a_1 \cdot a_2 \cdot a_3 \cdots a_n$$

The most famous example of this is the factorial: $n! = \prod_{i=1}^n i$. Again, same deal as summation but you're multiplying instead of adding.

8.2 Induction on sequences

Here's where the fun starts. We introduced the induction principle on natural numbers in Module 4 and worked through proofs about addition in the Intermezzo on Natural Induction. Here we use the same tool to prove things about sequences and sums.

Let's recall the rule:

```
natural induction
  assumes P(0)
    subproof:
      assumes P(n)
      -----
      shows P(succ n)
  -----
shows forall n : Nat. P(n)
```

Same rule as before. Nothing new here! The only thing that's changing is what $P(n)$ is going to be. In the Intermezzo on Natural Induction, $P(n)$ was something like “ $0 + n = n$ ” or “addition is associative.” Now $P(n)$ is going to be equations about sums and sequences.

8.2.1 The standard template

Here's how an induction proof on a sum identity typically goes. You want to prove that some formula holds for all $n \geq \text{base}$. The template is:

¹More precisely, the body is pure—no side effects, no mutation beyond the accumulator. If you want to be fancy about it, it's a *fold*.

1. **State $P(n)$:** write down exactly what you're trying to prove. This is usually an equation like "the sum of the first n whatever equals some closed-form expression."
2. **Base case:** verify that $P(\text{base})$ is true by direct computation.
3. **Inductive hypothesis:** assume $P(k)$ is true for some arbitrary $k \geq \text{base}$.
4. **Inductive step:** prove $P(k+1)$ using the assumption that $P(k)$ holds. This is where the actual work happens.

Let's see this in action.

8.2.2 Worked example: sum of the first n integers

We want to prove: for all $n \geq 1$,

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

or in summation notation, $\sum_{i=1}^n i = \frac{n(n+1)}{2}$.

Let $P(n)$ be the statement: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$.

Base case ($n = 1$): The left side is just 1. The right side is $\frac{1 \cdot 2}{2} = 1$. They're equal, so $P(1)$ is true.

Inductive hypothesis: Assume $P(k)$ is true for some $k \geq 1$. That is, assume

$$\sum_{i=1}^k i = \frac{k(k+1)}{2}$$

Inductive step: We need to show $P(k+1)$, which says $\sum_{i=1}^{k+1} i = \frac{(k+1)(k+2)}{2}$.

Here's the key trick, and it's the same every time: split the sum into the part we already know about and the new term.

$$\begin{aligned} \sum_{i=1}^{k+1} i &= \left(\sum_{i=1}^k i \right) + (k+1) \\ &= \frac{k(k+1)}{2} + (k+1) && \text{by the inductive hypothesis!} \\ &= \frac{k(k+1)}{2} + \frac{2(k+1)}{2} \\ &= \frac{k(k+1) + 2(k+1)}{2} \\ &= \frac{(k+1)(k+2)}{2} \end{aligned}$$

And that last expression is exactly what $P(k+1)$ claims. We're done!

See the pattern? You peel off the last term of the sum, apply the inductive hypothesis to the rest, and then do algebra to show you get the right answer. That "peel off the last term" move is the heart of every induction proof on sums.

Exercise 2. Prove by induction: for all $n \geq 1$, $\sum_{i=1}^n (2i - 1) = n^2$. (This says the sum of the first n odd numbers is n^2 .) Follow the same template as the worked example above: state $P(n)$, check the base case, assume $P(k)$, and prove $P(k + 1)$ by peeling off the last term.

8.2.3 Setting up: sum of squares

Here's another one that you'll see on the assignment. For all $n \geq 1$:

$$1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

So $P(n)$ is the statement $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$.

The base case is easy: $P(1)$ says $1 = \frac{1 \cdot 2 \cdot 3}{6} = 1$. Check.

For the inductive step, you'd assume $P(k)$ and then look at

$$\sum_{i=1}^{k+1} i^2 = \left(\sum_{i=1}^k i^2 \right) + (k+1)^2 = \frac{k(k+1)(2k+1)}{6} + (k+1)^2$$

and then the game is to do the algebra to show this equals $\frac{(k+1)(k+2)(2k+3)}{6}$. It's the same strategy—peel, substitute, simplify—just with hairier algebra. I'll leave the details to you for the assignment but the key insight is: factor out $(k+1)$ from both terms and then simplify what's left.

8.2.4 Induction is recursion

I want to close with something that I think is really important and ties together a lot of what we've been doing in this course.

Induction and recursion are *the same thing*. Seriously. They're two sides of the same coin.

When you write a recursive function you compute a value by reducing to a smaller case:

```
def sum_to(n):
    if n == 0:
        return 0 # base case
    else:
        return sum_to(n-1) + n # recursive case
```

When you do an induction proof you prove a statement by reducing to a smaller case: you prove $P(k+1)$ by using $P(k)$. The inductive hypothesis is playing *exactly* the role of the recursive call. The base case of the proof is the base case of the recursion. The inductive step is the recursive step.

Another way to say this is: a recursive function computes a *value* for each natural number by building up from the base case. An inductive proof computes a *proof* for each natural number by building up from the base case. The structure is identical. Induction *is* recursion on proofs.

This is why, back in Module 4, we introduced induction right alongside BNF definitions and recursive structure. They're not separate topics that happen to be in the same chapter—they're the same idea wearing different hats. And once you see that connection, both recursive programming and inductive proofs become a lot less mysterious. You just ask yourself: what's the base case, and how do I build the next one from the previous one?

Exercise 3. Write a recursive Python function that computes $\sum_{i=1}^n i^2$. Then compare its structure to the induction proof of the sum-of-squares formula from this module. Which part of your code corresponds to the base case? Which part corresponds to the inductive step? Where does the “inductive hypothesis” show up?

Chapter 9

Strong Induction and Recurrences

9.1 Strong induction

Remember the natural induction rule from Module 4? The one where you prove a base case $P(0)$ and then show that $P(n)$ implies $P(\text{succ } n)$? That rule is great, but sometimes it's not quite convenient enough.

Here's the thing: ordinary induction lets you assume *only* the immediately previous case. You're proving $P(k+1)$ and all you get to work with is $P(k)$. But what if your problem doesn't just depend on the previous case? What if you need to reach back further—to $P(k-1)$, or $P(k-3)$, or even $P(0)$?

This is where *strong induction* comes in. The idea is that instead of just assuming $P(k)$, you get to assume $P(j)$ for **all** j from the base case up to k . You get the whole history, not just the last step.

Here's the formal rule:

Strong Induction:

```
assumes P(0)
  subproof:
    assumes P(j) for all j <= k
    -----
    shows P(k+1)
  -----
shows forall n : Nat. P(n)
```

Now you might be wondering: is this *really* a valid thing to do? Doesn't assuming more stuff make the proof weaker somehow? Nope! It turns out that strong induction is actually *equivalent* to ordinary induction. You can derive one from the other.¹ The two principles have the same logical strength, but strong induction is often way more convenient when the recursive structure of your problem doesn't just depend on the immediately previous case.

Let's connect this to programming because I think it makes the idea really clear. Ordinary induction is like a recursive function that can *only* call itself on $n-1$:

```
def f(n):
  if n == 0:
    return base_case
```

¹The trick is that if you have a proof by strong induction you can convert it to an ordinary induction proof by changing your predicate from $P(n)$ to $Q(n) = "P(j) \text{ for all } j \leq n"$. Then the ordinary induction step for Q is exactly the strong induction step for P .


```

else:
    return something(f(n-1))

```

Strong induction, on the other hand, is like a recursive function that can call itself on *any* smaller input:

```

def f(n):
    if n == 0:
        return base_case
    else:
        # can use f(n-1), f(n-2), ..., f(0), whatever you need!
        return something(f(n-3), f(n//2))

```

Think of merge sort or binary search! In merge sort you split the list roughly in half and recurse on both halves—you’re not just peeling off one element at a time. That’s the spirit of strong induction.

Let’s do a worked example. Consider the sequence defined by

$$c_0 = 2, \quad c_1 = 2, \quad c_2 = 6, \quad c_k = 3c_{k-3} \text{ for } k \geq 3$$

We want to prove that *all* terms of this sequence are even, i.e. that $2 \mid c_n$ for all $n \geq 0$.

Why do we need strong induction here? Well, the recurrence reaches back *three* steps: c_k depends on c_{k-3} , not c_{k-1} . Ordinary induction would only hand us c_{k-1} which isn’t directly useful.

Base cases: We need to check $P(0)$, $P(1)$, and $P(2)$. We have $c_0 = 2$, $c_1 = 2$, $c_2 = 6$, and all of these are even. Good.²

Inductive step: Assume $P(j)$ for all $j \leq k$ where $k \geq 2$. We need to show $P(k+1)$, i.e. that c_{k+1} is even.

Since $k+1 \geq 3$, we can use the recurrence: $c_{k+1} = 3 \cdot c_{(k+1)-3} = 3 \cdot c_{k-2}$.

Now $k-2 \leq k$, so by our strong induction hypothesis c_{k-2} is even. That means $c_{k-2} = 2m$ for some integer m . Therefore $c_{k+1} = 3 \cdot 2m = 6m = 2(3m)$, which is even. Done!

See how we needed to reach back to c_{k-2} and the strong induction hypothesis let us do that? With ordinary induction we’d have been stuck.

Here’s a second example that shows off a different flavor of strong induction—one where you don’t reach back a fixed number of steps, but an *unknown* number of steps. This is more like the merge sort analogy: you split the problem roughly in half, and the pieces could land anywhere below you.

Theorem 9.1.1. Every integer $n \geq 2$ is either prime or a product of primes.

Proof. By strong induction on n .

Base case: $n = 2$ is prime. Done.

Inductive step: Assume that every integer j with $2 \leq j \leq k$ is either prime or a product of primes. We need to show this for $k+1$ (where $k \geq 2$).

There are two cases. If $k+1$ is prime, we’re done immediately. If $k+1$ is composite, then by definition $k+1 = a \cdot b$ for some integers a, b with $2 \leq a, b \leq k$. Since both a and b are at most k , our strong induction hypothesis applies to each of them: a is either prime or a product of primes, and so is b . Therefore $k+1 = a \cdot b$ is a product of primes. $\square$

²When you use strong induction, you often need multiple base cases. The number of base cases typically matches how far back the recurrence reaches.

Notice why ordinary induction would be hopeless here. If $k + 1 = a \cdot b$ is composite, the factors a and b could be *anywhere* between 2 and k —they’re not necessarily equal to k or $k - 1$ or any particular value. We need the full strength of the strong induction hypothesis to handle all of them at once. This is a case where the “binary search” analogy is apt: the subproblems aren’t one step back, they’re roughly half the size, and strong induction is exactly the tool that lets us recurse on arbitrary smaller values.

Exercise 1. Prove by strong induction: every integer amount of postage ≥ 12 cents can be made using 4-cent and 5-cent stamps. (Hint: you will need several base cases—check 12, 13, 14, 15 individually. For the inductive step, use the fact that the amount $k + 1 - 4 = k - 3$ is at least 12 when $k + 1 \geq 16$.)

9.2 Defining sequences recursively

We’ve been using recursive sequence definitions informally and it’s worth being a little more careful about what that means. A *recursively defined sequence* (sometimes called a *recurrence relation*) has two parts:

- **Initial conditions:** explicit values for one or more of the first terms
- **A recurrence relation:** a rule that defines each subsequent term in terms of earlier terms

This is exactly like defining a recursive function in code: you need base cases and a recursive case. If you leave out the base cases your recursion never bottoms out and you get a stack overflow (or, in the math world, an ill-defined sequence).

Let’s compute a few terms of a sequence to make sure we’re comfortable with the mechanics. Consider

$$d_0 = 3, \quad d_k = k \cdot (d_{k-1})^2 \text{ for } k \geq 1$$

Let’s just crank out some values:

$$\begin{aligned} d_0 &= 3 \\ d_1 &= 1 \cdot (d_0)^2 = 1 \cdot 9 = 9 \\ d_2 &= 2 \cdot (d_1)^2 = 2 \cdot 81 = 162 \\ d_3 &= 3 \cdot (d_2)^2 = 3 \cdot 26244 = 78732 \end{aligned}$$

These numbers get big fast! That’s the power of squaring in the recurrence. The point is that even though we don’t have a nice closed-form formula, we can always compute any term we want as long as we know the previous ones.

Now here’s a useful skill: given a proposed *explicit* (closed-form) formula, how do we *verify* that it actually satisfies a recurrence?

For instance, suppose someone hands you the formula

$$s_n = \frac{(-1)^n}{n!}$$

and claims it satisfies the recurrence $s_k = -s_{k-1}/k$ for $k \geq 1$ with $s_0 = 1$.

Let's check. First, $s_0 = (-1)^0/0! = 1/1 = 1$. Good, initial condition checks out. Now we compute $-s_{k-1}/k$:

$$\begin{aligned} -\frac{s_{k-1}}{k} &= -\frac{1}{k} \cdot \frac{(-1)^{k-1}}{(k-1)!} \\ &= \frac{(-1) \cdot (-1)^{k-1}}{k \cdot (k-1)!} \\ &= \frac{(-1)^k}{k!} \\ &= s_k \end{aligned}$$

And that's it! The formula checks out. The key idea is that verifying a closed form is usually way easier than *finding* it in the first place. You just plug the formula into both sides of the recurrence and show they're equal.

9.3 Solving recurrences by iteration

How *do* you find a closed form in the first place? One really useful technique is called *iteration* (or “unrolling”). The idea is simple: you just keep substituting the recurrence into itself until you see a pattern, and then you guess the closed form.

Before we do an example of this, let's recall a formula that shows up everywhere: the *geometric series*. If $r \neq 1$ then

$$1 + r + r^2 + \cdots + r^n = \frac{r^{n+1} - 1}{r - 1}$$

This is going to come in handy when we unroll recurrences because geometric sums pop up all the time.

Let's try iterating a concrete recurrence. Consider the “number of regions” problem: you have a plane, and you're drawing lines through it. Let P_k be the number of regions created by k lines (where no two lines are parallel and no three meet at a point). The k -th line crosses all $k-1$ existing lines, creating k new regions, so:

$$P_1 = 2, \quad P_k = P_{k-1} + k \text{ for } k \geq 2$$

Let's unroll this:

$$\begin{aligned} P_n &= P_{n-1} + n \\ &= (P_{n-2} + (n-1)) + n \\ &= ((P_{n-3} + (n-2)) + (n-1)) + n \\ &= \cdots \\ &= P_1 + 2 + 3 + \cdots + n \\ &= 2 + 2 + 3 + \cdots + n \\ &= 1 + (1 + 2 + 3 + \cdots + n) \\ &= 1 + \frac{n(n+1)}{2} \end{aligned}$$

where we used the familiar formula $1 + 2 + \cdots + n = n(n+1)/2$ at the end. So the closed form is $P_n = 1 + n(n+1)/2$.

Let's sanity check: $P_1 = 1 + 1 \cdot 2/2 = 2$. $P_2 = 1 + 2 \cdot 3/2 = 4$. And indeed if you draw two crossing lines you get 4 regions. Looks right!

Now, to be rigorous, once you've *guessed* the formula by iteration you should really *prove* it by induction. The iteration gives you the conjecture; induction gives you the proof. But in practice, the iteration is often the hard part and the induction proof is pretty mechanical after that.

Exercise 2. Use iteration to find a closed form for $T_1 = 1$, $T_k = T_{k-1} + 3$ for $k \geq 2$. That is, unroll the recurrence a few times until you see the pattern, then write down a formula for T_n . Once you have your conjecture, verify it by induction.

9.4 Second-order linear homogeneous recurrences

Now we're going to talk about a really important special class of recurrences.

Definition 9.4.1 (Second-order linear homogeneous recurrence). A *second-order linear homogeneous recurrence with constant coefficients* is a recurrence of the form

$$a_k = A \cdot a_{k-1} + B \cdot a_{k-2}$$

where A and B are constants (they don't depend on k).

Let's break down the terminology because every word in that name actually means something:

- **Second-order:** the recurrence depends on the *two* previous terms (a_{k-1} and a_{k-2})
- **Linear:** the previous terms show up to the first power only—no a_{k-1}^2 or $a_{k-1} \cdot a_{k-2}$ business
- **Homogeneous:** there's no extra added constant term. Compare $a_k = 2a_{k-1} + 3$ (not homogeneous) vs. $a_k = 2a_{k-1} + a_{k-2}$ (homogeneous)
- **Constant coefficients:** A and B are just numbers, they don't change with k

Why do we care about this specific class? Because there's a *systematic method* for solving them! We don't have to guess and check. Here's the trick.

We're going to *guess* that the solution has the form $a_n = r^n$ for some constant r . Why is this a reasonable guess? Well, if you think about it, the recurrence says that each term is a linear combination of the previous two. Exponential functions have exactly this property: r^k is a constant times r^{k-1} which is a constant times r^{k-2} , and so on.

So let's substitute $a_n = r^n$ into $a_k = A \cdot a_{k-1} + B \cdot a_{k-2}$:

$$r^k = A \cdot r^{k-1} + B \cdot r^{k-2}$$

Divide both sides by r^{k-2} (assuming $r \neq 0$):

$$r^2 = Ar + B$$

This is the *characteristic equation* of the recurrence. It's just a quadratic! We know how to solve quadratics.

Now here's where it gets interesting. If the characteristic equation has two *distinct* roots r_1 and r_2 , then the general solution is

$$a_n = \alpha \cdot r_1^n + \beta \cdot r_2^n$$

where α and β are constants determined by the initial conditions a_0 and a_1 . In other words, any linear combination of the two solutions is also a solution,³ and we use the initial conditions to pin down which particular linear combination we want.

If the characteristic equation has a *repeated* root r (i.e. $r_1 = r_2 = r$), then the general solution is slightly different:

$$a_n = \alpha \cdot r^n + \beta \cdot n \cdot r^n$$

The extra factor of n shows up because we need two linearly independent solutions and r^n alone only gives us one. But the repeated root case is rarer in practice so let's focus on the distinct roots case.

Let's do the most famous example: the *Fibonacci sequence*.

$$F_0 = 0, \quad F_1 = 1, \quad F_k = F_{k-1} + F_{k-2} \text{ for } k \geq 2$$

This is a second-order linear homogeneous recurrence with $A = 1$ and $B = 1$. The characteristic equation is

$$r^2 = r + 1 \quad \implies \quad r^2 - r - 1 = 0$$

By the quadratic formula:

$$r = \frac{1 \pm \sqrt{5}}{2}$$

So $r_1 = \frac{1+\sqrt{5}}{2}$ (this is the *golden ratio*, often written φ) and $r_2 = \frac{1-\sqrt{5}}{2}$.

The general solution is $F_n = \alpha \cdot \varphi^n + \beta \cdot \left(\frac{1-\sqrt{5}}{2}\right)^n$. Now we use the initial conditions to find α and β :

$$F_0 = 0 : \quad \alpha + \beta = 0 \quad \implies \quad \beta = -\alpha$$

$$F_1 = 1 : \quad \alpha \cdot \varphi + \beta \cdot \frac{1-\sqrt{5}}{2} = 1$$

Substituting $\beta = -\alpha$:

$$\alpha \left(\varphi - \frac{1-\sqrt{5}}{2} \right) = \alpha \cdot \sqrt{5} = 1 \quad \implies \quad \alpha = \frac{1}{\sqrt{5}}$$

So the closed form for the Fibonacci numbers is

$$F_n = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^n - \left(\frac{1-\sqrt{5}}{2} \right)^n \right]$$

That's pretty remarkable, isn't it? You start with a simple recurrence and end up with the golden ratio and $\sqrt{5}$. The Fibonacci numbers are integers, but the formula involves $\sqrt{5}$ all over the place. The irrational parts cancel out perfectly every time. Also, since $\left| \frac{1-\sqrt{5}}{2} \right| < 1$, that second term shrinks to zero as n grows, which means F_n is approximately $\varphi^n / \sqrt{5}$ for large n . So the Fibonacci numbers grow exponentially at the rate of the golden ratio!

It all comes from the characteristic equation method: substitute a guess and solve a quadratic.

³This is one of those things where "linear" in the name really matters. It's exactly the same principle as in linear algebra where solutions to homogeneous systems form a vector space.

The repeated-root case

We said earlier that when the characteristic equation has a repeated root r , the general solution picks up an extra factor of n : $a_n = \alpha \cdot r^n + \beta \cdot n \cdot r^n$. Let's see this in action with a quick example.

Consider the recurrence

$$a_0 = 1, \quad a_1 = 4, \quad a_k = 4a_{k-1} - 4a_{k-2} \text{ for } k \geq 2$$

The characteristic equation is $r^2 - 4r + 4 = 0$, which factors as $(r - 2)^2 = 0$. So we have a repeated root $r = 2$, and the general solution is

$$a_n = \alpha \cdot 2^n + \beta \cdot n \cdot 2^n$$

Using the initial conditions:

$$\begin{aligned} a_0 = 1 : \quad \alpha \cdot 1 + \beta \cdot 0 &= \alpha = 1 \\ a_1 = 4 : \quad \alpha \cdot 2 + \beta \cdot 1 \cdot 2 &= 2 + 2\beta = 4 \implies \beta = 1 \end{aligned}$$

So the closed form is $a_n = 2^n + n \cdot 2^n = (1 + n) \cdot 2^n$.

Sanity check: $a_2 = 4 \cdot 4 - 4 \cdot 1 = 12$, and $(1 + 2) \cdot 2^2 = 3 \cdot 4 = 12$. It works.

Exercise 3. Find the closed form for $a_0 = 1$, $a_1 = 3$, $a_k = 4a_{k-1} - 3a_{k-2}$ using the characteristic equation method. That is: write down the characteristic equation, find its roots, write the general solution, and then use the initial conditions to solve for the constants.

Chapter 10

Structural Induction

10.1 The pattern so far

We’ve arrived at the last module and I want to name something that’s been hiding in plain sight all quarter. Let’s take a quick tour of what we’ve done.

Back in Module 2 we introduced BNF as a way to define the syntax of propositional logic:

$$L := T \mid F \mid L \wedge L \mid L \vee L \mid \sim L \mid L \rightarrow L \mid x$$

Then in Module 4 we used the exact same idea to define the natural numbers:

$$\text{Nat} := 0 \mid \text{succ Nat}$$

and we got *natural induction* out of it—for free! Two cases in the BNF, two obligations in the proof: handle zero, handle successor. And in Module 8 we saw that induction is really just recursion on proofs. A recursive function computes a value by reducing to a smaller case; an inductive proof proves a statement by reducing to a smaller case. Same thing, different hat.

Here’s what I want to make explicit now. The connection between BNF definitions and induction principles isn’t some special coincidence about natural numbers. It’s a **general pattern**: *any* BNF definition gives you an induction principle. Every single time. The base cases of the BNF become base cases of your proof, and the recursive cases of the BNF become inductive steps where you get to assume the property holds for the sub-parts.

We’ve actually been using this pattern all along without naming it. Now let’s name it—*structural induction*—and demonstrate it on two new datatypes that show up constantly in computer science: lists and binary trees.

10.2 Lists

10.2.1 Defining lists inductively

What is a list? You’ve used lists in every programming class you’ve ever taken. Python’s `[1, 2, 3]`, Java’s `ArrayList`, Haskell’s `[a]`—they’re everywhere. But what *is* a list, really, when you strip away all the implementation details?

Here’s the answer, in the same BNF style we’ve been using all quarter:

Definition 10.2.1 (List). $\vdash$

$$\text{List } A := \text{Nil} \mid \text{Cons}(A, \text{List } A)$$

A list of A 's is either *empty* (which we call `Nil`) or it's an element of type A stuck onto the front of another list (which we call `Cons`).¹

Two cases. Sound familiar?

Let's make sure this connects to what you already know. In Python, the empty list `[]` is our `Nil`. And the list `[1, 2, 3]` is really:

```
# [1, 2, 3] is secretly:
Cons(1, Cons(2, Cons(3, Nil)))
```

In Haskell this is even more explicit. The empty list is `[]` and the cons operator is `:`, so you can write `1 : 2 : 3 : []` which is exactly `Cons(1, Cons(2, Cons(3, Nil)))`. This is why the colon operator is so central in Haskell. It's the recursive case of the datatype.

Now here's the payoff. Just like the BNF for natural numbers gave us natural induction, the BNF for lists gives us *structural induction on lists*:

```
assumes P(Nil)
  subproof:
    assumes P(xs)
    -----
    shows P(Cons(x, xs))
  -----
shows  $\forall$  xs : List A. P(xs)
```

Look at how perfectly this mirrors the natural induction rule from Module 4! The structure is identical:

- The **base case** corresponds to the base case of the BNF (`Nil`, just like 0 for `Nat`).
- The **inductive step** corresponds to the recursive case of the BNF (`Cons`, just like `succ` for `Nat`), and you get to assume the property holds for the tail `xs` (just like you assumed $P(n)$ when proving $P(\text{succ } n)$).

And this makes sense! A list is basically a natural number with data attached to each successor. `Nil` is like zero, and `Cons` is like successor but it also carries a value. So of course the induction principle looks the same.

10.2.2 Recursive functions on lists

Before we can prove anything interesting we need some functions to talk about. Let's define two standard ones: `length` and `append`.

Length. The length of a list is defined recursively, following the two cases of the BNF:

```
length(Nil) = 0
length(Cons(x, xs)) = 1 + length(xs)
```

Or in Python, if you were implementing it from scratch:²

```
def length(lst):
    if lst == []: # Nil case
        return 0
    else: # Cons case
        return 1 + length(lst[1:])
```

¹The name "Cons" comes from Lisp, one of the oldest programming languages, where it was short for "construct." It's been the standard name in functional programming ever since.

²Yes, Python has a built-in `len()` function. We're defining it ourselves because we need to reason about it.

Append. The append function (which we'll write as ++ because that's what Haskell uses and it's nice and short) concatenates two lists:

```
Nil ++ ys = ys
Cons(x, xs) ++ ys = Cons(x, xs ++ ys)
```

So appending `ys` onto an empty list just gives you `ys`, and appending `ys` onto `Cons(x, xs)` sticks `x` on front and then appends `ys` onto the tail. For example:

```
Cons(1, Cons(2, Nil)) ++ Cons(3, Nil)
= Cons(1, Cons(2, Nil) ++ Cons(3, Nil))
= Cons(1, Cons(2, Nil ++ Cons(3, Nil)))
= Cons(1, Cons(2, Cons(3, Nil)))
```

which is $[1, 2] ++ [3] = [1, 2, 3]$, exactly what you'd expect.

Notice how both of these functions follow the structure of the BNF. Two cases in the datatype, two cases in the function. This is the “recursion is induction” idea from Module 8 showing up again: the shape of your data determines the shape of your code.

Exercise 1. Define a recursive function `reverse` on lists, following the two cases of the BNF. (Hint: you will need to use `append (++)` in the recursive case.) Then compute `reverse(Cons(1, Cons(2, Cons(3, Nil))))` step by step, showing each unfolding of the definition.

10.2.3 Worked proof: length distributes over append

Now let's *prove* something using structural induction on lists. We'll show that:

For all lists `xs` and `ys`:

$$\text{length}(xs ++ ys) = \text{length}(xs) + \text{length}(ys)$$

In other words, the length of two lists concatenated together is the sum of their individual lengths.

We'll do structural induction on `xs`. Our property $P(xs)$ is:

$$\text{for all } ys, \text{length}(xs ++ ys) = \text{length}(xs) + \text{length}(ys).$$

Base case (`xs = Nil`): We need to show that $\text{length}(\text{Nil} ++ ys) = \text{length}(\text{Nil}) + \text{length}(ys)$ for all `ys`.

The left side:

$$\text{length}(\text{Nil} ++ ys) = \text{length}(ys) \quad (\text{by definition of } ++)$$

The right side:

$$\begin{aligned} \text{length}(\text{Nil}) + \text{length}(ys) &= 0 + \text{length}(ys) && (\text{by definition of length}) \\ &= \text{length}(ys) \end{aligned}$$

They're equal. Base case done.

Inductive step (`xs = Cons(x, xs')`): Assume $P(xs')$ holds, i.e. assume that for all `ys`,

$$\text{length}(xs' ++ ys) = \text{length}(xs') + \text{length}(ys) \quad (\text{IH})$$

We need to show $\text{length}(\text{Cons}(x, xs') ++ ys) = \text{length}(\text{Cons}(x, xs')) + \text{length}(ys)$.
Let's work on the left side:

$$\begin{aligned}
 & \text{length}(\text{Cons}(x, xs') ++ ys) \\
 &= \text{length}(\text{Cons}(x, xs' ++ ys)) && \text{(by definition of ++)} \\
 &= 1 + \text{length}(xs' ++ ys) && \text{(by definition of length)} \\
 &= 1 + \text{length}(xs') + \text{length}(ys) && \text{(by IH!)}
 \end{aligned}$$

And the right side:

$$\begin{aligned}
 & \text{length}(\text{Cons}(x, xs')) + \text{length}(ys) \\
 &= (1 + \text{length}(xs')) + \text{length}(ys) && \text{(by definition of length)} \\
 &= 1 + \text{length}(xs') + \text{length}(ys)
 \end{aligned}$$

They're the same! So $P(\text{Cons}(x, xs'))$ holds, and by structural induction we conclude $P(xs)$ for all lists xs . $\square$

See the pattern? It's the exact same game as the induction proofs we did on natural numbers in Modules 4 and 8. Unfold the definitions, apply the inductive hypothesis, watch things simplify. The only difference is that instead of peeling off a `succ` we're peeling off a `Cons`.

10.3 Binary trees

10.3.1 Defining binary trees inductively

Now let's look at a slightly more interesting datatype. You've likely seen binary trees in a data structures class (or you will soon). Binary search trees, heaps, parse trees, &c. They're one of the most important data structures in all of computer science.

Here's the BNF definition:

Definition 10.3.1 (Binary Tree).
`BinTree := Leaf | Node(BinTree, BinTree)`

A binary tree is either a *leaf* (an empty tree, a dead end) or it's a *node* with a left subtree and a right subtree.³

Again, two cases. But notice something new: the recursive case `Node` has **two** recursive sub-parts, not one. A cons cell had one tail; a node has two children. This is going to affect the induction principle.

In Haskell you'd write this as:

```
data BinTree = Leaf | Node BinTree BinTree
```

And in Python, maybe something like:

```
class Leaf:
    pass

class Node:
```

³Some definitions of binary trees store data at the nodes, like `Node(A, BinTree, BinTree)`. We're keeping it simple here and just tracking the shape of the tree. Everything we prove about the shape applies equally well when you add data.

```
def __init__(self, left, right):
    self.left = left
    self.right = right
```

What’s the induction principle? Here’s where the two subtrees make things interesting:

```
assumes P(Leaf)
subproof:
  assumes P(left)
    P(right)
  -----
  shows P(Node(left, right))
-----
shows  $\forall t : \text{BinTree}. P(t)$ 
```

See the difference from list induction? In the inductive step you get **two** inductive hypotheses—one for the left subtree and one for the right subtree—because the BNF has two recursive occurrences. Compare this to lists, where **Cons** had one recursive sub-part so you got one inductive hypothesis, which was the same as natural induction where **succ** had one recursive sub-part. The number of inductive hypotheses always matches the number of recursive sub-parts in the BNF. Always.

10.3.2 Recursive functions on binary trees

Let’s define a function that counts the number of internal *nodes* in a binary tree (i.e. the non-leaf nodes):

```
size(Leaf) = 0
size(Node(left, right)) = 1 + size(left) + size(right)
```

A leaf contributes nothing, and a node contributes itself plus whatever’s in its subtrees. Simple enough. In Python:

```
def size(t):
    if isinstance(t, Leaf):
        return 0
    else: # Node
        return 1 + size(t.left) + size(t.right)
```

We’ll also want to count the number of *internal edges*—edges connecting one node to another node or a node to a leaf. Each node is connected to its two children, so let’s define:

```
edges(Leaf) = 0
edges(Node(left, right)) = 2 + edges(left) + edges(right)
```

Why 2? Because each **Node** creates exactly two edges: one going down to its left child and one going down to its right child.⁴

10.3.3 Worked proof: counting edges in a binary tree

Now let’s prove something about binary trees using structural induction. We’ll show that:

⁴If you draw a binary tree on paper, every line you draw corresponds to one of these edges. A node with two children means two lines going down from that node.

For all binary trees t :

$$\text{edges}(t) = 2 \cdot \text{size}(t)$$

In other words, a binary tree with n internal nodes has exactly $2n$ edges (counting edges to leaf children). This makes intuitive sense: every node contributes exactly 2 edges (to its children), and leaves contribute 0.

We'll do structural induction on t . Our property $P(t)$ is the statement " $\text{edges}(t) = 2 \cdot \text{size}(t)$."

Base case ($t = \text{Leaf}$): We need $\text{edges}(\text{Leaf}) = 2 \cdot \text{size}(\text{Leaf})$. The left side is 0 and the right side is $2 \cdot 0 = 0$. Done.

Inductive step ($t = \text{Node}(\text{left}, \text{right})$): Assume $P(\text{left})$ and $P(\text{right})$, i.e. assume:

$$\text{edges}(\text{left}) = 2 \cdot \text{size}(\text{left}) \quad (\text{IH for left})$$

$$\text{edges}(\text{right}) = 2 \cdot \text{size}(\text{right}) \quad (\text{IH for right})$$

We need to show $\text{edges}(\text{Node}(\text{left}, \text{right})) = 2 \cdot \text{size}(\text{Node}(\text{left}, \text{right}))$.

Left side:

$$\begin{aligned} \text{edges}(\text{Node}(\text{left}, \text{right})) &= 2 + \text{edges}(\text{left}) + \text{edges}(\text{right}) && (\text{by definition of edges}) \\ &= 2 + 2 \cdot \text{size}(\text{left}) + 2 \cdot \text{size}(\text{right}) && (\text{by both IHs!}) \end{aligned}$$

Right side:

$$\begin{aligned} 2 \cdot \text{size}(\text{Node}(\text{left}, \text{right})) &= 2 \cdot (1 + \text{size}(\text{left}) + \text{size}(\text{right})) && (\text{by definition of size}) \\ &= 2 + 2 \cdot \text{size}(\text{left}) + 2 \cdot \text{size}(\text{right}) \end{aligned}$$

They're equal! Both sides give us $2 + 2 \cdot \text{size}(\text{left}) + 2 \cdot \text{size}(\text{right})$, so $P(\text{Node}(\text{left}, \text{right}))$ holds. By structural induction, $P(t)$ for all binary trees t . $\square$

Notice how we used **both** inductive hypotheses in the inductive step—one for each subtree. That's the characteristic feature of tree induction: you get two IHs and you typically need both.

Exercise 2. Define a function `leaves(t)` that counts the number of leaves in a binary tree (a `Leaf` counts as 1, a `Node` counts the leaves in both subtrees). Then prove by structural induction: $\text{leaves}(t) = \text{size}(t) + 1$ for all binary trees t . (You will need both inductive hypotheses in the inductive step, just like in the edges proof above.)

10.4 The general recipe

Let's spell out the pattern once and for all, because at this point you've seen it enough times that it should click.

Here's the recipe. Given **any** BNF definition of a datatype, you automatically get an induction principle. The correspondence is:

- Each **base case** of the BNF gives you a **base case** of the proof. You need to show your property holds for each non-recursive constructor.
- Each **recursive case** of the BNF gives you an **inductive step**. For each recursive constructor, you get to *assume* the property holds for all recursive sub-parts, and you need to *show* it holds for the whole thing.

That's it. That's the whole recipe. Let's verify it against everything we've seen this quarter:

Natural numbers: $\text{Nat} := 0 \mid \text{succ Nat}$. One base case (0), one recursive case (**succ**) with one sub-part. So you get one base case and one inductive step with one IH. That's natural induction from Module 4.

Lists: $\text{List } A := \text{Nil} \mid \text{Cons}(A, \text{List } A)$. One base case (**Nil**), one recursive case (**Cons**) with one recursive sub-part (the tail). So you get one base case and one inductive step with one IH. Structurally identical to natural induction! And that makes sense: a list is basically a natural number that carries data.

Binary trees: $\text{BinTree} := \text{Leaf} \mid \text{Node}(\text{BinTree}, \text{BinTree})$. One base case (**Leaf**), one recursive case (**Node**) with **two** recursive sub-parts. So you get one base case and one inductive step with **two** IHs. That's the new thing we saw in this module.

And you could keep going! Imagine a datatype with two base cases and three recursive cases. You'd get two base cases and three inductive steps in your proof. Or consider propositional logic formulas from Module 2: T and F are base cases, $\sim L$ is a recursive case with one sub-part, and $L \wedge L$, $L \vee L$, $L \rightarrow L$ are recursive cases with two sub-parts each. So to prove something about all propositional formulas you'd handle the base cases (T , F , variables) and then do inductive steps for each connective, getting the appropriate number of inductive hypotheses each time.⁵

This extends to any datatype you can dream up—rose trees, abstract syntax trees, regular expressions, whatever. If you can write down a BNF for it, you get a structural induction principle for it. You'll see more of this in CS 251 when we work with graphs, automata, and other structures.

10.4.1 Example: structural induction on propositional formulas

Let's actually carry out that propositional-logic example instead of just waving at it. Recall the BNF from Module 2:

$L := T \mid F \mid x \mid \sim L \mid L \wedge L \mid L \vee L \mid L \rightarrow L$

Define a function **size** that counts the number of connectives in a formula:

```
size(T) = 0
size(F) = 0
size(x) = 0
size(~L) = 1 + size(L)
size(L1 ^ L2) = 1 + size(L1) + size(L2)
size(L1 v L2) = 1 + size(L1) + size(L2)
size(L1 -> L2) = 1 + size(L1) + size(L2)
```

Seven cases, following the seven cases of the BNF—exactly the recipe we just described. Now let's prove a simple property to see the machinery work on a grammar with multiple base cases and multiple recursive cases.

Claim. For all formulas f , $\text{size}(f) \geq 0$.

We proceed by structural induction on f .

Base cases. If $f = T$, $f = F$, or $f = x$ (a variable), then $\text{size}(f) = 0 \geq 0$. Done.

Inductive step (negation): $f = \sim L$. Assume $\text{size}(L) \geq 0$ (IH). Then:

$$\begin{aligned} \text{size}(\sim L) &= 1 + \text{size}(L) && \text{(by definition of size)} \\ &\geq 1 + 0 = 1 \geq 0 && \text{(by IH)} \end{aligned}$$

⁵If you go on to study programming languages or compilers, this is *exactly* how you prove theorems about programs: by structural induction on the syntax of the language. The BNF of the programming language gives you the induction principle, and then you grind through the cases. It's this same recipe, just with bigger grammars.

Inductive step (binary connectives): $f = L_1 \star L_2$ where $\star$ is any of $\wedge, \vee, \rightarrow$. Assume $\text{size}(L_1) \geq 0$ (IH1) and $\text{size}(L_2) \geq 0$ (IH2). Then:

$$\begin{aligned} \text{size}(L_1 \star L_2) &= 1 + \text{size}(L_1) + \text{size}(L_2) && \text{(by definition of size)} \\ &\geq 1 + 0 + 0 = 1 \geq 0 && \text{(by IH1 and IH2)} \end{aligned}$$

All cases handled. By structural induction, $\text{size}(f) \geq 0$ for every formula f . $\square$

The proof itself is almost trivially easy—and that’s the point. What matters is the *shape*: three base cases (matching the three non-recursive alternatives in the BNF), one inductive case with one IH (for $\sim$), and three inductive cases with two IHs each (for $\wedge, \vee, \rightarrow$). That shape was determined entirely by the BNF. We just followed the recipe.

Exercise 3. Define a function $\text{vars}(f)$ that returns the number of variable occurrences in a propositional formula (following the BNF from Module 2). For instance, $\text{vars}(x \wedge y) = 2$ and $\text{vars}(x \wedge x) = 2$ (we’re counting occurrences, not distinct variables). Then state—but you don’t need to prove—what the structural induction principle would require you to show in order to prove some property $P(f)$ for all formulas f . How many base cases? How many inductive steps? How many IHs in each?

10.5 Wrapping up: the big picture

Let’s take a step back and look at what we’ve actually built over this whole course. There’s been a single thread running through every module, and now we can finally state it cleanly.

The thread is a three-step pattern:

1. **Define your data inductively** using BNF. Specify the base cases and the recursive cases.
2. **Get an induction principle for free** that mirrors the structure of your definition. Base cases in the BNF become base cases in the proof. Recursive cases become inductive steps.
3. **Use that principle to prove things** about all instances of your data by handling each case.

We did this for propositional logic formulas in Module 2. We did this for natural numbers in Module 4. We did this for sequences in Module 8. In Module 9, we saw how *strong* induction generalizes the basic induction principle to handle recurrences that reach back more than one step, and the characteristic equation technique gave us tools for solving those recurrences in closed form. And now in this module we’ve done it for lists and binary trees.

The specific data changes but the *method* is always the same. Define inductively, get an induction principle, prove by cases. And as we saw in Module 8, this is the exact same thing as writing recursive programs: the structure of your data determines the structure of your code, which determines the structure of your proofs. Recursion, induction, and case analysis are facets of a single method depending on whether you’re computing a value or proving a theorem.

If there’s one thing I want you to take away from this course, it’s this: when you encounter some new kind of structured data, the first question you should ask is “what’s the BNF?” Once you have that, you know how to define functions on it (by recursion following the cases), and you know how to prove things about it (by induction following the cases). Everything else is details.

This is the foundation for CS 251 and beyond—type theory, programming language theory, algorithm analysis, formal verification. The datatypes get bigger and the proofs get harder, but the recipe stays the same. That recipe will carry you further than you might expect.⁶

⁶Okay, I lied a little. In CS 251 you’ll see datatypes where the recursion is more complicated—mutual recursion,

nested datatypes, indexed families—and the induction principles get correspondingly fancier. But the core idea really is the same. Promise.

Intermezzo: The Algebra of Types

Something strange has been happening throughout this course, and it's time we talked about it.

In Module 1 we learned that $|A \times B| = |A| \cdot |B|$ —the cardinality of a Cartesian product is the product of the cardinalities. In the sets intermezzo we learned that $|B^A| = |B|^{|A|}$ —function spaces have exponential cardinality. And in Module 10 we defined data types using BNF, building lists and trees out of simpler pieces.

These facts live in different chapters, but they are secretly the same idea. Types obey the laws of algebra. Not metaphorically, not “sort of”—they obey the *actual* laws of high-school algebra, the ones with $+$ and $\times$ and exponents. Once you see this, a lot of things click into place.

10.6 Types are like numbers

Let's build the dictionary.

Product types are multiplication. A pair (a, b) where $a \in A$ and $b \in B$ has type $A \times B$. How many inhabitants does $A \times B$ have? For each choice of a (there are $|A|$ many) and each choice of b (there are $|B|$ many), we get a distinct pair. So $|A \times B| = |A| \cdot |B|$. A tuple is multiplication.

Sum types are addition. Suppose we have a type $A + B$ that means “either a value of type A , tagged with `Left`, or a value of type B , tagged with `Right`.” In Haskell this is `Either A B`; in Python you might use a tagged union or a class hierarchy. How many inhabitants? Every element of A gives one (tagged `Left`), and every element of B gives another (tagged `Right`). So $|A + B| = |A| + |B|$. A tagged union is addition.

Function types are exponentiation. A function $f : A \rightarrow B$ assigns to each element of A an element of B . That's $|B|$ choices for each of the $|A|$ inputs, so $|A \rightarrow B| = |B|^{|A|}$. We saw this in the sets intermezzo—it's why the function space is written B^A . A function is exponentiation.

The unit type is 1. Let 1 denote a type with exactly one element (in Haskell, `()`; in Python, `None`). Then $A \times 1 \cong A$ —a pair of an A and a “nothing” is just an A , since the second component carries no information. And $1^A \cong 1$ —there's only one function from A to a one-element set (the function that sends everything to the single element).

The empty type is 0. Let 0 denote a type with *no* elements (in Haskell, `Void`; in Python, a class you can never instantiate). Then $A \times 0 \cong 0$ —you can't build a pair if the second component has no possible values. And $A + 0 \cong A$ —“either an A or an impossible thing” is just an A . And for any nonempty A , $0^A \cong 0$ —there are no functions from A to an empty set (you'd have to send the elements of A somewhere, but there's nowhere to send them).

Look at these equations again. $A \times 1 = A$. $A + 0 = A$. $A \times 0 = 0$. Those are the rules of arithmetic you learned in grade school, but now they're facts about data types. Types form an algebra, and it's the same algebra you already know.

10.7 The laws of algebra hold

Let’s verify that the familiar identities from arithmetic actually hold for types. In each case, “ $\cong$ ” means “there’s a bijection between the two types”—they have the same number of inhabitants, and you can convert back and forth without losing information.

Multiplicative identity: $A \times 1 \cong A$. A pair where one component is trivial is just the other component. In Haskell: `(a, ())` is isomorphic to `a`.

Additive identity: $A + 0 \cong A$. Either `A Void` is isomorphic to `A`—the `Right` case is impossible (you can’t construct a `Void`), so you’re always in the `Left` case.

Distributivity: $A \times (B + C) \cong (A \times B) + (A \times C)$. A pair of an `A` with “either a `B` or a `C`” is the same as “either a pair of `A` and `B`, or a pair of `A` and `C`.” This is the distributive law $a(b + c) = ab + ac$, in type form.

Exponent of a product: $A \rightarrow (B \times C) \cong (A \rightarrow B) \times (A \rightarrow C)$. A function that returns a pair is the same as a pair of functions. If I ask you for a machine that takes an input and produces two outputs, you can either build one machine with two output slots or build two separate machines—same thing.

Exponent of a sum: $(A + B) \rightarrow C \cong (A \rightarrow C) \times (B \rightarrow C)$. A function from “either an `A` or a `B`” is the same as a pair of functions: one that handles `A` inputs and one that handles `B` inputs. In Python:

```
# A function from Either is a pair of handlers:
def handle_either(x):
    if isinstance(x, Left):
        return handle_a(x.value) # the A -> C part
    else:
        return handle_b(x.value) # the B -> C part
```

Wait a moment. A function from a sum is a pair of handlers—one for each case. Isn’t that exactly *proof by cases*? In Module 3 we learned the disjunction elimination rule: if you want to prove something from $A \vee B$, you handle the `A` case and handle the `B` case separately. That’s the logical version of $(A + B) \rightarrow C \cong (A \rightarrow C) \times (B \rightarrow C)$ —or-elimination *is* the fact that a function from a sum is a pair of functions. The algebra is telling us something about logic.

10.8 Data types as polynomials

Here is where things get a little bit magical.

In Module 10 we defined lists with a BNF:

```
List A := Nil | Cons(A, List A)
```

In our algebraic notation, `Nil` is the unit type `1` (one way to be empty) and `Cons(A, List A)` is a product $A \times L$ where `L` is the type of lists. The `|` in the BNF is a sum. So the definition of lists is:

$$L = 1 + A \cdot L$$

That’s a polynomial equation! Let’s be reckless and solve it like one. Rearranging:

$$L - A \cdot L = 1 \quad \implies \quad L(1 - A) = 1 \quad \implies \quad L = \frac{1}{1 - A}$$

Now, you might be having a reasonable objection right now, which is: what does it even *mean* to divide types or subtract types? Fair enough. But let's push ahead and see what happens. From Module 9, we know the formula for a geometric series:

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + \dots$$

So if we just blindly expand:

$$L = 1 + A + A^2 + A^3 + A^4 + \dots$$

And what is this saying? A list of A 's is:

- 1: the empty list (one way), or
- A : a single element (one A), or
- $A^2 = A \times A$: two elements (a pair of A 's), or
- $A^3 = A \times A \times A$: three elements, or
- ...and so on forever.

That's *exactly right*. A list is either empty, or has one element, or has two, or three, or any finite number of elements. The algebra of types—this insane-looking manipulation where we divided by $(1 - A)$ as if types were numbers—told us the truth. The geometric series from Module 9 is secretly a statement about data structures.

Let's try another one. Binary trees from Module 10 were defined as:

```
Tree := Leaf | Node(Tree, Tree)
```

In algebraic notation:

$$T = 1 + T^2$$

This is a quadratic equation! Rearranging: $T^2 - T + 1 = 0$, and the quadratic formula gives

$$T = \frac{1 \pm \sqrt{1-4}}{2} = \frac{1 \pm \sqrt{-3}}{2}$$

which involves complex numbers and seems completely unhinged. But remarkably, if you take the right root and expand it as a power series, you get the *Catalan numbers*—which are exactly the number of distinct binary trees with n internal nodes. The algebra of types knows about combinatorics.

We won't push this further here, but the point is: these aren't parlor tricks. There is a rigorous mathematical theory—the theory of *combinatorial species*—that makes all of this precise. The equation $L = 1 + A \cdot L$ is a legitimate algebraic equation in that theory, the “division” is a real operation, and the geometric series expansion is a theorem, not a coincidence.

What we're seeing is a remarkable convergence: the sets intermezzo (cardinality of function spaces), Module 9 (geometric series), and Module 10 (BNF definitions of data types) are all aspects of a single algebraic framework. Types are numbers, data definitions are polynomial equations, and the power series you learned in Module 9 enumerate the shapes of data structures.

10.9 The Curry–Howard connection

There’s one more layer. In the Curry–Howard intermezzo we learned that types correspond to propositions: a type is “true” if it’s inhabited (has at least one value) and “false” if it’s empty. Under this correspondence:

Algebra	Types	Logic
$A \cdot B$	$A \times B$ (product)	$A \wedge B$ (conjunction)
$A + B$	$A + B$ (sum)	$A \vee B$ (disjunction)
B^A	$A \rightarrow B$ (function)	$A \rightarrow B$ (implication)
1	Unit	$\top$ (true)
0	Void	$\perp$ (false)

So the algebraic identities we verified in Section 2 are also *logical tautologies*:

- $A \times 1 \cong A$ is $A \wedge \top \equiv A$ (“ A and true” is just A)
- $A + 0 \cong A$ is $A \vee \perp \equiv A$ (“ A or false” is just A)
- $A \times (B + C) \cong (A \times B) + (A \times C)$ is the distribution of $\wedge$ over $\vee$
- $(A + B) \rightarrow C \cong (A \rightarrow C) \times (B \rightarrow C)$ is or-elimination
- $A \times 0 \cong 0$ is $A \wedge \perp \equiv \perp$ (“ A and false” is false)

Three parallel worlds—arithmetic, type theory, and logic—and they’re all obeying the same laws. The algebra of numbers, the algebra of types, and the algebra of propositions are three views of one structure.

That structure has a name, by the way. It’s called a *rig* (a “ring without negation”—because you can’t subtract types or negate propositions in general). But you don’t need to know the name to appreciate the fact: the math you already know, from all across this course, fits together like a puzzle. Cardinality is counting. Types are numbers. BNF definitions are polynomial equations. Proofs are programs. And the geometric series from Module 9 secretly counts the shapes of lists.

10.10 Exercises

Exercise 1. Using the algebra of types, simplify the type $(A \rightarrow B) \times (A \rightarrow C)$. What single type is it isomorphic to? What does this isomorphism correspond to as a logical proposition (under the Curry–Howard correspondence)?

Exercise 2. The BNF for natural numbers is $\text{Nat} = \text{Zero} \mid \text{Succ}(\text{Nat})$, which in our algebraic notation is $N = 1 + N$. Write this as a “polynomial equation” and “solve” for N by expanding as a series. Does the series $1 + 1 + 1 + 1 + \dots$ make sense as a description of the natural numbers? (Hint: what does $1^n = 1$ mean for the n th term?)

Intermezzo: The Lambda Calculus

We’ve reached the end of CS 250. Over the past ten modules you’ve learned how to define data with BNF, how to prove things about data with induction, and—in the intermezzi on Curry–Howard and the algebra of types—how proofs and programs are secretly the same thing. But there’s a question we left hanging all the way back in Module 1 that I want to come back to now.

Remember the question? We asked: if we want to understand what computers can do *ever*—not just what today’s chips can handle—we need an abstract model of computation, one that doesn’t depend on hardware or programming language. We said we’d leave the answer for CS 251. But we can give a preview now, and it turns out that almost everything you need to understand the answer has been hiding in this course the whole time.

10.11 A mathematical model of computation

In the 1930s, before electronic computers existed, a mathematician named Alonzo Church asked a foundational question: what does it mean to *compute* something? His answer was the **lambda calculus**—the simplest possible programming language. Not a practical language you’d use to build a web app, but a theoretical one, stripped down to the absolute minimum needed to express any computation at all.

How minimal? The entire language has exactly three constructs. Here’s the BNF:

$$M := x \mid \lambda x. M \mid M M$$

That’s it. Three things:

- **Variables** (x): names that stand for values. You’ve seen these since Module 2.
- **Abstraction** ($\lambda x. M$): a function definition. The λ says “I’m defining a function whose parameter is x and whose body is M .” If you’ve written `def f(x): return M` in Python or `fun x -> M` in OCaml, you already know what this is. The λ is just the world’s oldest anonymous function syntax.
- **Application** ($M M$): a function call. Put a function next to its argument and it runs.

No numbers. No booleans. No if-statements. No loops. No data structures. Just variables, function definitions, and function calls. You’re probably thinking: how can you compute *anything* with that? The answer is one of the most surprising facts in all of computer science. You can build everything from functions alone.

10.12 Computing with pure functions

The lambda calculus has one rule of computation, called **beta reduction**. It says: when you apply a function to an argument, substitute the argument for the parameter in the body. In symbols:

$$(\lambda x. M) N \longrightarrow M[N/x]$$

where $M[N/x]$ means “ M with every occurrence of x replaced by N .” That’s it. That’s the entire execution model.

Example 10.12.1 (The identity function). The term $\lambda x. x$ is a function that takes its argument and returns it unchanged. Apply it to 5:

$$(\lambda x. x) 5 \longrightarrow x[5/x] = 5$$

The identity function applied to 5 gives 5. Not very exciting, but it illustrates the rule.

Example 10.12.2 (Applying a function twice). Consider the term $\lambda f. \lambda x. f (f x)$. This takes a function f and a value x , and applies f twice. If f is “add 1” and x is 0:

$$\begin{aligned} (\lambda f. \lambda x. f (f x)) \text{ add1 } 0 &\longrightarrow (\lambda x. \text{add1 } (\text{add1 } x)) 0 \\ &\longrightarrow \text{add1 } (\text{add1 } 0) \\ &= \text{add1 } 1 \\ &= 2 \end{aligned}$$

Does this remind you of anything? In Module 4, we defined the natural numbers with a successor function `succ` and a base case 0. Applying `succ` twice to 0 gives 2. The idea of “iterate a function n times” is about to become very important.

Notice something familiar about the BNF for lambda terms. It has a recursive structure: the body of an abstraction is itself a lambda term, and both parts of an application are lambda terms. By the pattern from Module 10, this BNF gives us a principle of *structural induction on lambda terms*—three cases, one for each construct. This is exactly how programming language theorists prove properties of languages: by induction on the syntax. The tools you learned in Module 10 are the same tools the researchers use.

10.13 Church encodings—building everything from nothing

Here is where things get genuinely mind-bending. Since the lambda calculus has no built-in data types, Church figured out how to *encode* data as functions. The strategy is elegant: represent a piece of data by what you can *do with it*.

Church booleans

What can you do with a boolean? You use it to choose between two alternatives. So let’s define booleans as choosers:

$$\begin{array}{ll} \text{true} = \lambda t. \lambda f. t & \text{(given two options, pick the first)} \\ \text{false} = \lambda t. \lambda f. f & \text{(given two options, pick the second)} \end{array}$$

A boolean *is* an if-statement. Given two branches, it picks one. In fact, the “if” function is almost trivial:

$$\text{if} = \lambda b. \lambda t. \lambda f. b \ t \ f$$

It just hands the two branches to the boolean and lets the boolean decide. Let’s verify:

$$\text{if true } a \ b \longrightarrow \text{true } a \ b \longrightarrow (\lambda t. \lambda f. t) \ a \ b \longrightarrow (\lambda f. a) \ b \longrightarrow a$$

It works. And this connects directly to Module 2: we’re encoding T and F—the truth values of propositional logic—as pure functions. The logical constants have become computational objects.

Church numerals

What can you do with a natural number n ? You can use it to *repeat* something n times. So let’s define a natural number as a function that iterates its argument:

$$\begin{aligned} \mathbf{0} &= \lambda f. \lambda x. x && (\text{apply } f \text{ zero times}) \\ \mathbf{1} &= \lambda f. \lambda x. f \ x && (\text{apply } f \text{ once}) \\ \mathbf{2} &= \lambda f. \lambda x. f \ (f \ x) && (\text{apply } f \text{ twice}) \\ \mathbf{3} &= \lambda f. \lambda x. f \ (f \ (f \ x)) && (\text{apply } f \text{ three times}) \\ &\vdots \\ \mathbf{n} &= \lambda f. \lambda x. f^n \ x && (\text{apply } f \ n \text{ times}) \end{aligned}$$

Stop and look at this. A Church numeral $\mathbf{n}$ takes a function f and a starting value x , and applies f to x exactly n times. Zero applies f zero times (just returns x). One applies f once. And so on.

Now here’s the connection I want you to see. In Module 4 we defined the natural numbers by BNF:

$\text{Nat} ::= 0 \mid \text{succ}(\text{Nat})$

Two cases: a base case (0) and a recursive case (apply succ one more time). Church numerals are *the same definition in disguise*. The base case is x (the “zero” of the iteration). The recursive case is “apply f one more time.” Church numeral $\mathbf{n}$ is literally $\text{succ}^n(0)$ with succ renamed to f and 0 renamed to x . The BNF and the Church encoding are two faces of the same coin.

With this insight, the successor function writes itself:

$$\text{succ} = \lambda n. \lambda f. \lambda x. f \ (n \ f \ x)$$

Given a Church numeral n , this produces a new numeral that applies f one extra time: n already applies f n times to x , and then we wrap one more f around the result. That’s “apply f one more time”—exactly the recursive case of the BNF.

And addition? Apply f a total of $m + n$ times by first applying it n times, then m more:

$$\text{add} = \lambda m. \lambda n. \lambda f. \lambda x. m \ f \ (n \ f \ x)$$

The term $n \ f \ x$ applies f n times to x . Then $m \ f$ applies f m more times to that result. Total: $m + n$ applications.

10.14 Why this matters

We started this course by asking whether Python and C can solve the same problems, and whether some future computer might solve problems that today’s computers can’t. The lambda calculus gives us the tools to answer these questions.

The Church–Turing thesis. In the same decade that Church invented the lambda calculus, Alan Turing independently invented a different model of computation—the Turing machine, a kind of abstract tape-reading automaton. The two models look nothing alike. But Church and Turing proved that they compute exactly the same class of functions. Every function computable by a Turing machine can be computed in the lambda calculus, and vice versa. This result has been replicated for every “reasonable” model of computation anyone has ever proposed: register machines, cellular automata, Post systems, RAM machines, quantum computers. They all compute the same things. The **Church–Turing thesis** is the claim that this class—the class of functions computable by any of these models—is *the* right notion of “computable.”

This is the answer to Module 1’s question. Python and C can solve the same problems because they are both models of computation at least as powerful as the lambda calculus (and no more powerful). No future chip will change this. The limits of computation are not engineering constraints—they are mathematical facts, and the lambda calculus is one of the cleanest ways to see them.

The lambda calculus is the ancestor of every functional language. Haskell, ML, OCaml, Lisp, Scheme, Clojure, Erlang, F#—all of them are, at their core, the lambda calculus with syntactic sugar on top. When you write a lambda expression in Python (`lambda x: x + 1`), you are writing a Church-style abstraction. The notation is literally named after Church’s λ .

Under Curry–Howard, lambda terms are proofs. In the previous intermezzo, we saw that proofs correspond to programs and propositions correspond to types. The programs in that correspondence are lambda terms. So the simplest possible model of computation is also the simplest possible language of mathematical proof. A lambda term of type $A \rightarrow B$ is simultaneously a function from A to B and a proof that A implies B . Defining a function *is* introducing an implication. Applying a function *is* modus ponens. The act of computing and the act of reasoning are, at bottom, the same act.

BNF, induction, Curry–Howard, and the lambda calculus form a single story. The BNF for lambda terms ($M := x \mid \lambda x. M \mid M M$) gives you structural induction on terms. Structural induction lets you prove properties of programs. Curry–Howard tells you those programs are proofs. And the Church–Turing thesis tells you those programs capture all of computation. The entire course—from the first BNF in Module 2 to the structural induction of Module 10, from the proof rules of Module 3 to the Curry–Howard correspondence—was building toward this point. Logic, proof, computation, and data are not four separate subjects. They are four views of one subject. CS 251 will make that precise.

10.15 Exercises

Exercise 1. Evaluate $(\lambda x. \lambda y. x) 3 7$ by applying beta reduction step by step. What does this function do? (Hint: the expression $(\lambda x. \lambda y. x) 3 7$ means $((\lambda x. \lambda y. x) 3) 7$ —application is left-associative.)

Exercise 2. Write a Church encoding for the boolean operation AND. Hint: $\text{and} = \lambda a. \lambda b. a b \text{ false}$. Verify that this definition is correct by evaluating it on all four input combinations: and true true ,

and true false, and false true, and and false false. For each case, show the beta reduction steps and confirm the result matches the expected truth value.

Appendix A

Notation Summary

Symbol	Name	Introduced in
<i>Propositional Logic</i>		
$\wedge$	Conjunction (and)	Module 2
$\vee$	Disjunction (or)	Module 2
$\neg$ or $\sim$	Negation (not)	Module 2
$\rightarrow$	Implication (if... then)	Module 2
$\leftrightarrow$	Biconditional (if and only if)	Module 2
$\equiv$	Logical equivalence	Module 2
$\oplus$	Exclusive or (XOR)	Module 2
T, F	Truth values	Module 2
$\perp$	Absurdity / falsum	Gentzen Intermezzo
<i>Quantifiers and Predicates</i>		
$\forall$	For all (universal quantifier)	Module 4
$\exists$	There exists (existential quantifier)	Module 4
$P(x)$	Predicate applied to x	Module 4
<i>Sets</i>		
$\{ \}$	Set braces	Module 1
$\in, \notin$	Element of, not element of	Module 1
$\subseteq$	Subset	Module 1
$\cap$	Intersection	Module 1
$\cup$	Union	Module 1
$\setminus$	Set difference	Module 1
$\overline{A}$	Complement	Module 1
$\emptyset$	Empty set	Module 1
$ A $	Cardinality	Module 1
$\mathcal{P}(A)$	Power set	Module 1
$A \times B$	Cartesian product	Module 1
B^A	Function space ($A \rightarrow B$)	Cardinality Intermezzo
$\{x \mid P(x)\}$	Set-builder notation	Module 1
<i>Functions and Relations</i>		
$f : A \rightarrow B$	Function from A to B	Module 1
$a R b$	a is related to b by R	Module 1
$\cong$	Isomorphism / bijection exists	Cardinality Intermezzo

Symbol	Name	Introduced in
<i>Number Theory</i>		
$\mathbb{N}$	Natural numbers	Module 1
$\mathbb{Z}$	Integers	Module 5
$\mathbb{Q}$	Rationals	Module 5
$\mathbb{R}$	Reals	Module 4
$a \mid b$	a divides b	Module 5
$\gcd(a, b)$	Greatest common divisor	Module 7
$\mathbb{Z}/n\mathbb{Z}$	Integers mod n	Module 6
$\lfloor x \rfloor, \lceil x \rceil$	Floor, ceiling	Module 6
<i>Sequences and Sums</i>		
a_n	n th term of a sequence	Module 8
$\sum$	Summation	Module 8
$\prod$	Product	Module 8
$n!$	Factorial	Module 8
<i>Data Structures</i>		
Nil, Cons	List constructors	Module 10
Leaf, Node	Binary tree constructors	Module 10
<i>Proof Notation</i>		
$\therefore$	Therefore	Module 3
$\square$ or QED	End of proof	Throughout
$\{P\} C \{Q\}$	Hoare triple	Module 7

Appendix B

Solutions to Exercises

Module 1

Exercise 1. Let $A = \{1, 2, 3, 4\}$ and $B = \{3, 4, 5, 6\}$.

$$\begin{aligned}A \cap B &= \{3, 4\}, \\A \cup B &= \{1, 2, 3, 4, 5, 6\}, \\|A \times B| &= |A| \cdot |B| = 4 \times 4 = 16.\end{aligned}$$

Exercise 2. The set $\{1, \{1\}, \{1, \{1\}\}\}$ has three distinct elements: 1, $\{1\}$, and $\{1, \{1\}\}$. Therefore $|\{1, \{1\}, \{1, \{1\}\}\}| = 3$.

Exercise 3.

(a) $a R b$ iff $a + b$ is even.

- *Reflexive:* $a + a = 2a$, which is always even. ✓
- *Symmetric:* $a + b = b + a$, so if $a + b$ is even then $b + a$ is even. ✓
- *Transitive:* Suppose $a + b$ and $b + c$ are both even. Then $a + c = (a + b) + (b + c) - 2b$ is a sum of even numbers minus an even number, hence even. ✓

All three properties hold, so R is an equivalence relation.

(b) $a R b$ iff $a \leq b$.

- *Reflexive:* $a \leq a$. ✓
- *Symmetric:* Fails. $1 \leq 2$ but $2 \not\leq 1$. ✗
- *Transitive:* If $a \leq b$ and $b \leq c$ then $a \leq c$. ✓

Not an equivalence relation (symmetry fails).

(c) $a R b$ iff $|a - b| \leq 1$.

- *Reflexive:* $|a - a| = 0 \leq 1$. ✓
- *Symmetric:* $|a - b| = |b - a|$. ✓
- *Transitive:* Fails. $|0 - 1| = 1 \leq 1$ and $|1 - 2| = 1 \leq 1$, but $|0 - 2| = 2 > 1$. ✗

Not an equivalence relation (transitivity fails).

Intermezzo: Cardinality and Function Spaces

Exercise 1.

$$\mathcal{P}(\{1, 2, 3\}) = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}.$$

That is $2^3 = 8$ elements.

Exercise 2. There are $2^3 = 8$ functions from $\{a, b, c\}$ to $\{0, 1\}$. Each function is the characteristic function of a subset of $\{a, b, c\}$:

Subset	$f(a)$	$f(b)$	$f(c)$
$\emptyset$	0	0	0
$\{a\}$	1	0	0
$\{b\}$	0	1	0
$\{c\}$	0	0	1
$\{a, b\}$	1	1	0
$\{a, c\}$	1	0	1
$\{b, c\}$	0	1	1
$\{a, b, c\}$	1	1	1

Exercise 3.

Proof. Define $f : \mathbb{N} \rightarrow E$ by $f(n) = 2n$, where $E = \{0, 2, 4, 6, \dots\}$.

Injective: If $f(n) = f(m)$, then $2n = 2m$, so $n = m$.

Surjective: Let $e \in E$. Then $e = 2k$ for some $k \in \mathbb{N}$, and $f(k) = 2k = e$.

Since f is both injective and surjective, it is a bijection, proving that the even natural numbers are countably infinite. $\square$

Module 2

Exercise 1. Truth table for $(p \rightarrow q) \wedge (q \rightarrow p)$:

p	q	$p \rightarrow q$	$q \rightarrow p$	$(p \rightarrow q) \wedge (q \rightarrow p)$
T	T	T	T	T
T	F	F	T	F
F	T	T	F	F
F	F	T	T	T

This matches the biconditional $p \leftrightarrow q$, which is true exactly when p and q have the same truth value.

Exercise 2. Truth table for $\neg(p \vee q)$:

p	q	$p \vee q$	$\neg(p \vee q)$
T	T	T	F
T	F	T	F
F	T	T	F
F	F	F	T

Truth table for $(\neg p) \wedge (\neg q)$:

p	q	$\neg p$	$\neg q$	$(\neg p) \wedge (\neg q)$
T	T	F	F	F
T	F	F	T	F
F	T	T	F	F
F	F	T	T	T

The output columns are identical, confirming De Morgan's law $\neg(p \vee q) \equiv (\neg p) \wedge (\neg q)$.

Exercise 3. We claim $p \oplus q \equiv (p \wedge \neg q) \vee (\neg p \wedge q)$. Truth table:

p	q	$p \wedge \neg q$	$\neg p \wedge q$	$(p \wedge \neg q) \vee (\neg p \wedge q)$
T	T	F	F	F
T	F	T	F	T
F	T	F	T	T
F	F	F	F	F

This matches the XOR truth table (true when exactly one of p, q is true).

Exercise 4. A unary boolean operation takes one boolean input (2 possible values) and produces one boolean output. The number of such operations is $2^2 = 4$. They are:

1. Identity: $p \mapsto p$.
2. Negation: $p \mapsto \neg p$.
3. Constant true: $p \mapsto T$.
4. Constant false: $p \mapsto F$.

Module 3

Exercise 1. Truth table for $p \wedge q \rightarrow q$:

p	q	$p \wedge q$	$p \wedge q \rightarrow q$
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	T

Every row evaluates to T, so $p \wedge q \rightarrow q$ is a tautology.

Exercise 2. Premises: $p \rightarrow q, r \rightarrow \neg q, r$. Conclusion: $\neg p$.

From r (premise 3) and $r \rightarrow \neg q$ (premise 2), by modus ponens we obtain $\neg q$. From $\neg q$ and $p \rightarrow q$ (premise 1), by modus tollens we obtain $\neg p$.

Thus the argument is **valid**. (Equivalently, one can build the full truth table and verify that in every critical row—where all three premises are true—the conclusion $\neg p$ is also true.)

Exercise 3. The rows with output 1 are $(P, Q, R) = (1, 0, 1)$ and $(P, Q, R) = (0, 1, 1)$. The DNF expression is:

$$(P \wedge \neg Q \wedge R) \vee (\neg P \wedge Q \wedge R).$$

Circuit description: Two AND gates, one for each minterm. The first AND gate takes P , $\neg Q$ (via a NOT gate on Q), and R . The second AND gate takes $\neg P$ (via a NOT gate on P), Q , and R . The outputs of both AND gates feed into a single OR gate, which produces the final output.

Exercise 4. The “proof” only checks the rows of the truth table where p and q have the same truth value (both T or both F). In those rows, $p \rightarrow q$ and $q \rightarrow p$ do happen to agree. But the proof ignores the two rows where p and q differ.

When $p = T$ and $q = F$: $p \rightarrow q = F$ but $q \rightarrow p = T$. When $p = F$ and $q = T$: $p \rightarrow q = T$ but $q \rightarrow p = F$.

Since the two formulas differ on these rows, they are **not** logically equivalent. The error is an incomplete case analysis—the proof conveniently skipped exactly the cases that would reveal the formulas are different.

Module 4

Exercise 1. In the formula $\exists x. \forall y. P(x, y, z)$:

- x is **bound** by $\exists$.
- y is **bound** by $\forall$.
- z is **free** (it does not appear in the scope of any quantifier binding z).

Exercise 2. Original statement in symbols:

$$\forall s : \text{CS250}. \exists h : \text{HW}. \text{OnTime}(s, h).$$

Negation (pushing $\neg$ inward, flipping quantifiers):

$$\exists s : \text{CS250}. \forall h : \text{HW}. \neg \text{OnTime}(s, h).$$

In English: “There is a student in CS 250 who did not complete any homework on time.”

Exercise 3. Let $P(x, y)$ mean “ $x + y = 0$ ” over $\mathbb{Z}$.

- (a) $\forall x. \exists y. P(x, y)$. **True.** For any $x \in \mathbb{Z}$, pick $y = -x$. Then $x + (-x) = 0$.
- (b) $\exists y. \forall x. P(x, y)$. **False.** This claims there is a single y such that $x + y = 0$ for *all* x . That would require $y = -x$ for every x simultaneously, which is impossible.

Intermezzo: Quantifiers as Games

Exercise 1. The game has three rounds, corresponding to the three quantifiers $\forall \epsilon, \exists \delta, \forall x$:

1. **Round 1** (Challenger’s move, $\forall \epsilon$): Challenger picks any $\epsilon > 0$. Challenger’s goal is to make the task as hard as possible, so they will try to pick a very small ϵ .
2. **Round 2** (Prover’s move, $\exists \delta$): Prover sees the chosen ϵ and picks a $\delta > 0$ in response. Prover’s choice of δ can depend on ϵ .

3. **Round 3** (Challenger’s move, $\forall x$): Challenger picks any x satisfying $|x - 3| < \delta$, trying to find an x that breaks Prover’s claim.

Prover wins if, for the chosen x , $|x^2 - 9| < \epsilon$. The fact that Prover has a winning strategy—pick δ small enough based on ϵ —is precisely what it means for $f(x) = x^2$ to be continuous at $x = 3$.

Exercise 2. Prover moves first (this is the $\exists x$) and picks $x = 0$. Then Challenger picks any y they like (this is the $\forall y$). Prover wins if $x + y = y$, which becomes $0 + y = y$ —true for every y . So no matter what Challenger picks, Prover wins.

The winning move is $x = 0$ because 0 is the additive identity.

Module 5

Exercise 1.

Proof. Let r_1 and r_2 be rational numbers. By definition there exist integers a_1, b_1, a_2, b_2 with $b_1 \neq 0$ and $b_2 \neq 0$ such that $r_1 = a_1/b_1$ and $r_2 = a_2/b_2$. Then

$$r_1 \cdot r_2 = \frac{a_1}{b_1} \cdot \frac{a_2}{b_2} = \frac{a_1 a_2}{b_1 b_2}.$$

Since $a_1 a_2$ and $b_1 b_2$ are integers and $b_1 b_2 \neq 0$, the product $r_1 \cdot r_2$ is rational. □

Exercise 2. The claim “for every integer n , $n^2 > n$ ” is **false**.

Counterexample: $n = 0$. Then $n^2 = 0$ and $n = 0$, so $n^2 = n$, not $n^2 > n$.

(Another counterexample: $n = 1$, since $1^2 = 1 = 1$.)

Exercise 3. $(\mathbb{Z}, -, 0)$ is **not** a group. Associativity fails: $(a - b) - c \neq a - (b - c)$ in general.

Concrete example:

$$\begin{aligned}(1 - 2) - 3 &= -1 - 3 = -4, \\ 1 - (2 - 3) &= 1 - (-1) = 2.\end{aligned}$$

Since $-4 \neq 2$, the operation is not associative, so $(\mathbb{Z}, -, 0)$ is not a group.

Exercise 4. The “proof” that every integer is even begins by writing “Suppose $n = 2k$ for some integer k ”—but $n = 2k$ is exactly the conclusion that n is even. The proof assumes what it is trying to prove. This is *begging the question* (circular reasoning).

A correct proof would need to show that *every* integer n can be written as $n = 2k$ for some integer k . But this is false: odd integers exist (e.g., $n = 3$ cannot be written as $2k$ for any integer k). The flaw is not a small gap in an otherwise valid argument; the claim itself is false, and the “proof” hides this by assuming the conclusion as its starting point.

Module 6

Exercise 1.

$$\begin{aligned}\lfloor -3.7 \rfloor &= -4, & \lceil -3.7 \rceil &= -3, \\ \lfloor 4 \rfloor &= 4, & \lceil 4 \rceil &= 4.\end{aligned}$$

(Note: $\lfloor -3.7 \rfloor = -4$, not -3 , because floor rounds *down*, not toward zero.)

Exercise 2. The additive inverse of 3 in $\mathbb{Z}/7\mathbb{Z}$ is $(7 - 3) \bmod 7 = 4$.
 Verification: $3 + 4 = 7 \equiv 0 \pmod{7}$. ✓

Exercise 3. We need $3x \equiv 1 \pmod{5}$. Try $x = 2$:

$$3 \cdot 2 = 6 \equiv 1 \pmod{5}.$$

So the multiplicative inverse of 3 in $\mathbb{Z}/5\mathbb{Z}$ is 2.

Intermezzo: What Does It Mean for Things to Be “The Same”?

Exercise 1. Define $P_1 \sim P_2$ iff for every input x , $P_1(x) = P_2(x)$ (they produce the same output on every input).

This is **not** the same as checking whether the source code is identical. Two programs with completely different code can produce the same outputs. For example:

```
def f(x): return x + x
def g(x): return 2 * x
```

These are equivalent under the relation $\sim$ (they produce the same output for every input), but their source code is different. The equivalence class of a sorting program contains every correct sorting implementation—bubble sort, merge sort, quicksort, insertion sort, etc.—all distinct programs, but all “the same” under $\sim$.

This is *extensional equality* (Module 1): two functions are equal if they agree on all inputs, regardless of how they compute their results.

Exercise 2. Define $a/b \sim c/d$ iff $ad = bc$ (cross-multiplication).

Reflexive: $a/b \sim a/b$ iff $ab = ba$, which holds by commutativity of multiplication. ✓

Symmetric: If $a/b \sim c/d$ then $ad = bc$, so $cb = da$ (the same equation rearranged), hence $c/d \sim a/b$. ✓

Transitive: Suppose $a/b \sim c/d$ and $c/d \sim e/f$. Then $ad = bc$ and $cf = de$. Multiply the first equation by f : $adf = bcf$. Multiply the second by b : $cfb = deb$. So $adf = bcf = deb$, which gives $af \cdot d = be \cdot d$. Since $d \neq 0$, we can cancel to get $af = be$, hence $a/b \sim e/f$. ✓

The equivalence class of $1/2$ is $\{1/2, 2/4, 3/6, -1/(-2), -2/(-4), \dots\}$ —all fractions that reduce to the same value. The rational numbers $\mathbb{Q}$ are these equivalence classes: just as the integers can be constructed as equivalence classes of pairs of natural numbers (with $(a, b) \sim (c, d)$ iff $a + d = b + c$), the rationals are constructed as equivalence classes of pairs of integers.

Module 7

Exercise 1.

Proof. We prove the contrapositive: if n is odd, then n^3 is odd.

Assume n is odd. Then $n = 2k + 1$ for some integer k . Compute:

$$\begin{aligned} n^3 &= (2k + 1)^3 \\ &= 8k^3 + 12k^2 + 6k + 1 \\ &= 2(4k^3 + 6k^2 + 3k) + 1. \end{aligned}$$

Since $4k^3 + 6k^2 + 3k$ is an integer, n^3 has the form $2m + 1$ and is therefore odd.

By contrapositive, if n^3 is even then n is even. □

Exercise 2. Trace of the Euclidean algorithm on $\text{gcd}(48, 18)$:

$$\begin{aligned}\text{gcd}(48, 18) : \quad & 48 = 2 \cdot 18 + 12, \quad \text{so } \text{gcd}(48, 18) = \text{gcd}(18, 12). \\ \text{gcd}(18, 12) : \quad & 18 = 1 \cdot 12 + 6, \quad \text{so } \text{gcd}(18, 12) = \text{gcd}(12, 6). \\ \text{gcd}(12, 6) : \quad & 12 = 2 \cdot 6 + 0, \quad \text{so } \text{gcd}(12, 6) = 6.\end{aligned}$$

Answer: $\text{gcd}(48, 18) = 6$.

Exercise 3. We want to derive $\{x = 0\} x := x + 3 \{x = 3\}$.

Step 1 (assignment rule). The assignment rule says: to derive $\{?\} x := x + 3 \{x = 3\}$, substitute $x + 3$ for x in the postcondition $x = 3$. This yields the precondition $x + 3 = 3$, giving:

$$\{x + 3 = 3\} x := x + 3 \{x = 3\}.$$

Step 2 (strengthening). We need to show $x = 0 \implies x + 3 = 3$. This holds by elementary arithmetic. By the rule of consequence (precondition strengthening):

$$\{x = 0\} x := x + 3 \{x = 3\}.$$

Exercise 4. The “proof” that $\sqrt{4}$ is irrational follows the structure of the classic $\sqrt{2}$ proof by contradiction. It assumes $\sqrt{4} = a/b$ with $\text{gcd}(a, b) = 1$, squares both sides to get $4b^2 = a^2$, and derives that a is even, say $a = 2c$. Substituting gives $4b^2 = 4c^2$, so $b^2 = c^2$, hence $b = c$ (since $b, c > 0$). The proof then claims “Contradiction!”

But there is no contradiction. The derivation $b = c$ together with $a = 2c$ gives $a = 2b$, so $\sqrt{4} = a/b = 2b/b = 2$. This is a perfectly true statement— $\sqrt{4}$ really is 2—not a contradiction. The proof claims a true result is contradictory.

(A secondary issue: the step from $c^2 = b^2$ to $c = b$ requires $c, b \geq 0$, which happens to hold here but isn’t justified. However, the main error is that the argument produces no contradiction at all, because $\sqrt{4}$ is rational.)

Intermezzo: Proofs Are Programs

Exercise 1. Disjunction elimination ($\vee E$) corresponds to **pattern matching** on a sum type, i.e., case analysis on an **Either** value. Given a value of type **Either** **A** **B**, you provide two handlers:

- One for the **Left** **a** case (handling a proof of **A**), and
- One for the **Right** **b** case (handling a proof of **B**).

This is exactly proof by cases: to prove C from $A \vee B$, you show $A \rightarrow C$ and $B \rightarrow C$ separately, then use whichever case applies. In Haskell:

```
either :: (a -> c) -> (b -> c) -> Either a b -> c
either f g (Left a) = f a
either f g (Right b) = g b
```

Exercise 2. The type $A \rightarrow (B \rightarrow A)$ corresponds to a function that takes an a and returns a function that ignores its argument and returns a :

```
def f(a):
    return lambda b: a
```

This is the “constant function factory”—given any value of type A , it produces a function from B to A that ignores its input and always returns the original a .

As a logical tautology, $A \rightarrow (B \rightarrow A)$ says: “if A is true, then no matter what B is, A is still true.” This is obvious logically—having a proof of A means you have a proof of A regardless of what else is going on—and the program makes it computationally concrete: given evidence for A , just hold onto it and ignore whatever B -evidence someone hands you.

Intermezzo: Other Logics, Other Worlds

Exercise 1.

- (a) “The program eventually terminates”: $\Diamond \text{halt}$.
- (b) “The counter is always non-negative”: $\Box (\text{counter} \geq 0)$.
- (c) “Every request is eventually followed by a response”: $\Box (\text{request} \rightarrow \Diamond \text{response})$.

Module 8

Exercise 1. Recursive definition of $a_n = 3n$:

$$a_0 = 0, \quad a_n = a_{n-1} + 3 \quad \text{for } n \geq 1.$$

Verification: $a_0 = 0$, $a_1 = 0 + 3 = 3$, $a_2 = 3 + 3 = 6$, $a_3 = 6 + 3 = 9$. These match 0, 3, 6, 9. ✓

Exercise 2.

Proof. Let $P(n)$ be the statement $\sum_{i=1}^n (2i - 1) = n^2$.

Base case ($n = 1$): $\sum_{i=1}^1 (2i - 1) = 2(1) - 1 = 1 = 1^2$. ✓

Inductive step. Assume $P(k)$: $\sum_{i=1}^k (2i - 1) = k^2$.

Then:

$$\begin{aligned} \sum_{i=1}^{k+1} (2i - 1) &= \sum_{i=1}^k (2i - 1) + (2(k+1) - 1) \\ &= k^2 + (2k + 1) \quad (\text{by the inductive hypothesis}) \\ &= k^2 + 2k + 1 \\ &= (k + 1)^2. \end{aligned}$$

This establishes $P(k + 1)$, completing the induction. □

Exercise 3.

```
def sum_squares(n):
    if n == 0:
        return 0
    else:
        return sum_squares(n-1) + n*n
```

- The `if n == 0: return 0` branch corresponds to the **base case** of the induction.
- The recursive call `sum_squares(n-1)` corresponds to the **inductive hypothesis**: we assume the function already computes the correct answer for $n - 1$.
- Adding `n*n` to the result of `sum_squares(n-1)` mirrors the **inductive step**, where we peel off the last term n^2 and use the IH for the remaining sum.

Module 9

Exercise 1.

Proof. We prove by strong induction that every integer amount of postage $n \geq 12$ can be made using 4-cent and 5-cent stamps.

Base cases:

$$\begin{aligned} 12 &= 3 \times 4, \\ 13 &= 4 + 4 + 5, \\ 14 &= 4 + 5 + 5, \\ 15 &= 3 \times 5. \end{aligned}$$

Inductive step. Let $n \geq 16$ and assume (strong IH) that every integer amount from 12 to $n - 1$ can be made with 4-cent and 5-cent stamps.

Since $n \geq 16$, we have $n - 4 \geq 12$. By the strong inductive hypothesis, $n - 4$ cents can be made with 4-cent and 5-cent stamps. Adding one more 4-cent stamp gives n cents. $\square$

Exercise 2. The recurrence is $T_1 = 1$, $T_k = T_{k-1} + 3$ for $k \geq 2$.

Iteration:

$$\begin{aligned} T_k &= T_{k-1} + 3 \\ &= (T_{k-2} + 3) + 3 = T_{k-2} + 2 \cdot 3 \\ &= (T_{k-3} + 3) + 2 \cdot 3 = T_{k-3} + 3 \cdot 3 \\ &\vdots \\ &= T_1 + 3(k-1) \\ &= 1 + 3(k-1) = 3k - 2. \end{aligned}$$

Verification: $T_1 = 3(1) - 2 = 1 \checkmark$; $T_{k+1} = 3(k+1) - 2 = (3k - 2) + 3 = T_k + 3 \checkmark$.

Closed form: $T_k = 3k - 2$.

Exercise 3. The recurrence is $a_0 = 1$, $a_1 = 3$, $a_k = 4a_{k-1} - 3a_{k-2}$ for $k \geq 2$.

Characteristic equation:

$$r^2 - 4r + 3 = 0 \implies (r-1)(r-3) = 0 \implies r_1 = 1, r_2 = 3.$$

General solution: $a_n = \alpha \cdot 1^n + \beta \cdot 3^n = \alpha + \beta \cdot 3^n$.

Solving for constants:

$$\begin{aligned} a_0 = 1 &\implies \alpha + \beta = 1, \\ a_1 = 3 &\implies \alpha + 3\beta = 3. \end{aligned}$$

Subtracting the first from the second: $2\beta = 2$, so $\beta = 1$ and $\alpha = 0$.

Closed form: $a_n = 3^n$.

Module 10

Exercise 1. Recursive definition:

$$\begin{aligned}\text{reverse}(\text{Nil}) &= \text{Nil}, \\ \text{reverse}(\text{Cons}(x, xs)) &= \text{reverse}(xs) ++ \text{Cons}(x, \text{Nil}).\end{aligned}$$

Step-by-step computation of $\text{reverse}(\text{Cons}(1, \text{Cons}(2, \text{Cons}(3, \text{Nil}))))$:

$$\begin{aligned}\text{reverse}(\text{Cons}(1, \text{Cons}(2, \text{Cons}(3, \text{Nil})))) &= \text{reverse}(\text{Cons}(2, \text{Cons}(3, \text{Nil}))) ++ \text{Cons}(1, \text{Nil}) \\ &= (\text{reverse}(\text{Cons}(3, \text{Nil})) ++ \text{Cons}(2, \text{Nil})) ++ \text{Cons}(1, \text{Nil}) \\ &= ((\text{Nil} ++ \text{Cons}(3, \text{Nil})) ++ \text{Cons}(2, \text{Nil})) ++ \text{Cons}(1, \text{Nil}) \\ &= (\text{Cons}(3, \text{Nil}) ++ \text{Cons}(2, \text{Nil})) ++ \text{Cons}(1, \text{Nil}) \\ &= \text{Cons}(3, \text{Cons}(2, \text{Nil})) ++ \text{Cons}(1, \text{Nil}) \\ &= \text{Cons}(3, \text{Cons}(2, \text{Cons}(1, \text{Nil}))) \\ &= [3, 2, 1].\end{aligned}$$

Exercise 2. Define $\text{leaves}(t)$ by:

$$\begin{aligned}\text{leaves}(\text{Leaf}) &= 1, \\ \text{leaves}(\text{Node}(l, r)) &= \text{leaves}(l) + \text{leaves}(r).\end{aligned}$$

Proof. We prove $\text{leaves}(t) = \text{size}(t) + 1$ for all binary trees t by structural induction.

Base case ($t = \text{Leaf}$):

$$\text{leaves}(\text{Leaf}) = 1 = 0 + 1 = \text{size}(\text{Leaf}) + 1. \quad \checkmark$$

Inductive step ($t = \text{Node}(l, r)$). Assume (IH) that $\text{leaves}(l) = \text{size}(l) + 1$ and $\text{leaves}(r) = \text{size}(r) + 1$. Then:

$$\begin{aligned}\text{leaves}(\text{Node}(l, r)) &= \text{leaves}(l) + \text{leaves}(r) \\ &= (\text{size}(l) + 1) + (\text{size}(r) + 1) \quad (\text{by IH}) \\ &= \text{size}(l) + \text{size}(r) + 2 \\ &= (1 + \text{size}(l) + \text{size}(r)) + 1 \\ &= \text{size}(\text{Node}(l, r)) + 1. \quad \checkmark\end{aligned}$$

□

Exercise 3. Define vars on the BNF for propositional logic formulas:

$$\begin{aligned}\text{vars}(T) &= 0, \\ \text{vars}(F) &= 0, \\ \text{vars}(x) &= 1, \\ \text{vars}(\sim L) &= \text{vars}(L), \\ \text{vars}(L_1 \wedge L_2) &= \text{vars}(L_1) + \text{vars}(L_2), \\ \text{vars}(L_1 \vee L_2) &= \text{vars}(L_1) + \text{vars}(L_2), \\ \text{vars}(L_1 \rightarrow L_2) &= \text{vars}(L_1) + \text{vars}(L_2).\end{aligned}$$

A structural induction proof about vars would require:

- **Base cases:** three cases for the atomic formulas T , F , and a variable x .
- **Inductive case for negation** ($\sim L$): one inductive hypothesis (for L).
- **Inductive cases for binary connectives** ($\wedge, \vee, \rightarrow$): each requires two inductive hypotheses (one for L_1 and one for L_2).

Intermezzo: The Algebra of Types

Exercise 1. Using the exponent-of-a-product law from the algebra of types:

$$(A \rightarrow B) \times (A \rightarrow C) \cong A \rightarrow (B \times C).$$

A pair of functions from A is the same as a single function from A that returns a pair. Given $f : A \rightarrow B$ and $g : A \rightarrow C$, we can build $h : A \rightarrow (B \times C)$ by $h(a) = (f(a), g(a))$. Conversely, given h , we recover $f = \text{fst} \circ h$ and $g = \text{snd} \circ h$.

Under the Curry–Howard correspondence, this is the logical equivalence:

$$(A \rightarrow B) \wedge (A \rightarrow C) \equiv A \rightarrow (B \wedge C).$$

In words: “if A implies B and A implies C , then A implies B and C ”—and vice versa.

Exercise 2. The BNF $\text{Nat} = \text{Zero} \mid \text{Succ}(\text{Nat})$ translates to $N = 1 + N$. “Solving” by repeated substitution:

$$\begin{aligned} N &= 1 + N \\ &= 1 + (1 + N) \\ &= 1 + 1 + (1 + N) \\ &= 1 + 1 + 1 + (1 + N) \\ &= \dots \\ &= \underbrace{1 + 1 + 1 + \dots}_{\text{infinitely many}} \end{aligned}$$

This is an infinite sum of 1’s. It makes perfect sense as a description of the natural numbers: each term “1” in the sum represents one specific natural number (the first 1 is 0, the second is 1, the third is 2, etc.). Since each 1 contributes exactly one inhabitant, the type N has inhabitants $0, 1, 2, 3, \dots$ —exactly the natural numbers.

Intermezzo: The Lambda Calculus

Exercise 1. Application is left-associative, so $(\lambda x. \lambda y. x) 3 7 = ((\lambda x. \lambda y. x) 3) 7$.

Step 1. Apply $(\lambda x. \lambda y. x)$ to 3: substitute 3 for x in the body $\lambda y. x$:

$$(\lambda x. \lambda y. x) 3 \rightarrow \lambda y. 3.$$

Step 2. Apply $(\lambda y. 3)$ to 7: substitute 7 for y in the body 3. Since y does not appear in 3, nothing changes:

$$(\lambda y. 3) 7 \rightarrow 3.$$

Final result: 3.

This function takes two arguments and returns the first, ignoring the second—it is the “constant” function (or, in Church encoding terms, it is `true`: given two options, pick the first).

Exercise 2. Define `and` = $\lambda a. \lambda b. a \ b \text{ false}$.

The idea: if a is `true`, the result depends on b ; if a is `false`, the result is `false` regardless. Verification on all four cases:

Case 1: `and true true`:

$$\begin{aligned} \text{and true true} &= (\lambda a. \lambda b. a \ b \text{ false}) \text{ true true} \\ &\longrightarrow (\lambda b. \text{true } b \text{ false}) \text{ true} \\ &\longrightarrow \text{true true false} \\ &= (\lambda t. \lambda f. t) \text{ true false} \\ &\longrightarrow (\lambda f. \text{true}) \text{ false} \\ &\longrightarrow \text{true.} \quad \checkmark \end{aligned}$$

Case 2: `and true false`:

$$\longrightarrow \text{true false false} = (\lambda t. \lambda f. t) \text{ false false} \longrightarrow (\lambda f. \text{false}) \text{ false} \longrightarrow \text{false.} \quad \checkmark$$

Case 3: `and false true`:

$$\longrightarrow \text{false true false} = (\lambda t. \lambda f. f) \text{ true false} \longrightarrow (\lambda f. f) \text{ false} \longrightarrow \text{false.} \quad \checkmark$$

Case 4: `and false false`:

$$\longrightarrow \text{false false false} = (\lambda t. \lambda f. f) \text{ false false} \longrightarrow (\lambda f. f) \text{ false} \longrightarrow \text{false.} \quad \checkmark$$

All four cases match the expected truth table for `AND`.

Index

- argument form, 29
- assignment rule, 73
- associativity, 48

- beta reduction, 112
- Bézout’s identity, 62
- BHK interpretation, 77
- $\leftrightarrow$ (biconditional), 19
- bijection, 10
- binary tree, 101
- BNF (Backus-Naur form), 15
- bound variable, 40

- Cantor’s diagonal argument, 12
- cardinality, 7
- Cartesian product, 8
- ceiling, 61
- characteristic equation, 95
- characteristic function, 13
- Church, Alonzo, 111
- Church–Turing thesis, 114
- circuit, 32
- commutativity, 49
- $\wedge$ (conjunction), 16
- conjunctive normal form (CNF), 31
- Cons, 99
- constructive proof, 56
- contradiction, 18
- converse error, 30
- countable, 11
- countably infinite, 11
- counterexample, 56
- Curry–Howard correspondence, 78

- De Morgan’s laws, 19
- discharged assumption, 36
- disjoint, 6
- $\vee$ (disjunction), 16
- disjunctive normal form (DNF), 31
- disjunctive syllogism, 29
- divisibility, 55

- element, 5
- elimination rule, 26
- empty set, 7
- equivalence relation, 9
- Euclidean algorithm, 71
- even, 54
- existence proof, 56
- explicit formula, 85
- extended Euclidean algorithm, 71
- extensional equality, 6

- fallacy, 30
- Fibonacci sequence, 96
- finite field, 62
- first-order logic, 39
- floor, 61
- free variable, 40
- function, 8
- function space, 12
- functional completeness, 20

- Gentzen style, 35
- geometric series, 94
- golden ratio, 96
- greatest common divisor, 70
- group, 58

- Hoare logic, 72, 81
- Hoare triple, 72

- identity element, 58
- $\rightarrow$ (implication), 16
- indicator function, 13
- induction, 45
- induction on sequences, 87
- injection, 10
- intensional equality, 6
- $\cap$ (intersection), 6
- introduction rule, 26
- inverse, 58
- inverse error, 30

- irrational number, 68
- Kripke structure, 84
- lambda calculus, 111
- Leaf, 101
- linear logic, 81
- list, 99
- liveness, 83
- logic gate, 32
- logical equivalence, 19
- loop invariant, 73
- LTL, 83
- membership ($\in$), 5
- modal logic, 81
- model checking, 83
- modulus, 60
- modus ponens, 23
- modus tollens, 24
- NAND, 21
- natural deduction, 35
- natural numbers, 45
- necessary condition, 43
- $\neg$ (negation), 16
- Nil, 99
- Node, 101
- odd, 54
- partial function, 8
- postcondition, 72
- precondition, 72
- predicate, 39
- proof
 - by contradiction, 25, 68
 - by contrapositive, 69
 - direct, 54
- proof tree, 37
- propositional logic, 15
- quantifier
 - existential, 40
 - universal, 40
- quotient set, 65
- quotient-remainder theorem, 60
- rational number, 55
- recurrence relation, 93
- recursive definition, 86
- reflexive, 8
- relation, 8
- rig, 110
- safety property, 83
- semantics, 15
- sequence, 85
- set, 5
 - complement, 6
 - difference, 6
 - power, 7
- set-builder notation, 6
- strengthening, 74
- strong induction, 91
- structural induction, 98
 - on lambda terms, 112
- subset, 6
- sufficient condition, 43
- summation notation, 86
- surjection, 10
- symmetric, 8
- syntax, 15
- tautology, 18
- temporal logic, 81
- transitive, 9
- truth set, 40
- truth table, 16
- tuple, 8
- uncountable, 12
- $\cup$ (union), 6
- valid argument, 29
- weakening, 74
- XOR, 20